

The Faure Structural Architecture: From the Primordial Tuple to Deterministic Mathematics, Spectral Transport, and Formal Closure

By ALAIN FAURE

Abstract

The Faure Structural Architecture is a unified mathematical framework built from a primordial tuple and expanded through a deterministic hierarchy of formal engines. Each engine — algebraic, logical, geometric, categorical, probabilistic, analytical, differential, integral, operatorial, kernel-based, tuple-based, MRDP, global, meta, and final — is specified in Lean as a pure deterministic core acting on a structured space of cells.

This manuscript presents the complete architecture, the transport mechanisms between layers, the spectral closure, and the emergence of the Faure–Fourier Transform as the terminal operator of the system. The resulting structure is fully formal, fully deterministic, and canonically closed.

Contents

1. Faure Theorem Registry	14
1.1. Foundational Faure Theorems	14
1.2. Derived Faure Theorems	14
1.3. Additional Derived Results Identified in the Computational Audit	25
1.4. Fourth-order stabilization and canonical decomposition	28
1.5. Spectral consequence	31
1.6. Conditional nilpotency statement	32
1.7. Iterated-map stabilization	32
1.8. Intertwining transfer	33
1.9. Empirical Falsifiability Protocol	33
1.10. Registry Status	34
2. Faure Theorem Registry	34

2.1. Foundational Faure Theorems	34
2.2. Derived Faure Theorems	34
3. Faure Theorem I: Structural Significance of Mathematical Results	35
3.1. Structural Functionals	35
3.2. Monotonia strutturale	35
3.3. Faure Theorem I	36
4. Faure Theorem II: Injective Observation and Deterministic Distinguishability	36
4.1. Deterministic Systems and Observations	36
4.2. Aliasing and Distinguishability	37
4.3. Faure Theorem II	37
5. Faure Theorem III: Injective Encoding and Structural Transport	38
5.1. Deterministic State Spaces	38
5.2. Structural Transport	38
5.3. Injective Encoding and Transport Uniqueness	39
6. Faure Theorem IV: Injective Observation and Equality Preservation	40
6.1. Deterministic Systems and Observations	40
6.2. Aliasing and Equality	40
6.3. Faure Theorem IV	40
7. Faure Theorem V: Injective Coding and Structural Transport	41
7.1. Deterministic State Space	41
7.2. Structural Transport Through Coding	41
7.3. Injectivity and Transport Uniqueness	42
7.4. Faure Theorem V	42
8. Faure Theorem VI: Injective Observation and Structural Transport of Deterministic Iterates	43
8.1. Deterministic State Space	43
8.2. Injective Observation	43
8.3. Structural Transport	43
8.4. Injectivity and Iterative Transport	43
8.5. Faure Theorem VI	44
9. Faure Theorem VII: Commutativity of Translation Operators in Structural Representation	44
9.1. Translation Operators on the Torus	44
9.2. Faure Theorem VII	45
10. Faure Theorem VIII: Rank Bounds and Structural Incompleteness	45
10.1. Rank and Spectrum	45
10.2. Faure Theorem VIII	46
11. Faure Theorem IX: Deterministic Attractors in the Psi-Hierarchy	46
11.1. Deterministic Evolution in the Faure Architecture	46
11.2. Stabilized Attractor	46

11.3.	Deterministic Convergence	47
11.4.	Faure Theorem IX	47
12.	Faure Theorem X: Intersection Lattices in the 144-Structure	47
12.1.	The 144 Lattice	47
12.2.	Structural Membership	47
12.3.	Faure Theorem X	48
13.	Faure Theorem XI: Simulation and Structural Transport of Discrete Dynamics	48
13.1.	Deterministic Dynamics	48
13.2.	Structural Transport	48
13.3.	Faure Theorem XI	48
14.	Faure Theorem XII: Rational Discrete Data and Structural Coordinates	49
14.1.	Deterministic Rational Data	49
14.2.	Structural Constraint	49
14.3.	Faure Theorem XII	49
15.	Faure Theorem XIII: Deterministic Transitions on the Stabilized Energy Lattice	50
15.1.	The Stabilized Energy Lattice	50
15.2.	Deterministic Translation	50
15.3.	Structural Well-Definedness	50
15.4.	Faure Theorem XIII	50
16.	Faure Theorem XIV: Low-Density Families in the Psi-Hierarchy	51
16.1.	Binary State Space	51
16.2.	Combinatorial Structure	51
16.3.	Faure Theorem XIV	52
17.	Faure Theorem XV: Pushforward of Deterministic Dynamics	52
17.1.	Deterministic System	52
17.2.	Structural Projection	53
17.3.	Pushforward Dynamics	53
17.4.	Faure Theorem XV	53
18.	Faure Theorem XVI: Discrepancy and Uniform Boundedness	54
18.1.	Structural Sign Sequences	54
18.2.	Structural Bounds	55
18.3.	Faure Theorem XVI	55
19.	Faure Theorem XVII: Discrete Lattice Dynamics and Angular Observables	56
19.1.	Deterministic Lattice Structure	56
19.2.	Structural Closure	56
19.3.	Angular Observables	57
19.4.	Faure Theorem XVII	57

20.	Faure Theorem XVIII: Discrete Closure, Stabilized Dynamics, and Injective Observation	58
20.1.	Finite Discrete Structure	58
20.2.	Injective Observation	58
20.3.	Stabilized Dynamics	59
20.4.	Faure Theorem XVIII	59
21.	Faure Theorem XIX: Finite State Spaces, Stabilized Observation, and Output Bandwidth	60
21.1.	Finite Deterministic Structure	60
21.2.	Structural Bounds	60
21.3.	Stabilized Observation	61
21.4.	Faure Theorem XIX	61
22.	Faure Theorem XX: Deterministic Primordial Dynamics and Injective Structural Coding	62
22.1.	Primordial Deterministic Dynamics	62
22.2.	Structural Coding	62
22.3.	Compatibility with Primordial Transport	62
22.4.	Faure Theorem XX	62
22.5.	Remark	63
23.	Preliminary Mathematical Formalization of the Faure Framework	63
23.1.	Epistemic and Mathematical Status	63
23.2.	Definition 1: Harmonic Correction Factor	63
23.3.	Definition 2: Proposed Faure-Type Gravitational Correction	64
23.4.	Lemma 1: Relative Harmonic Correction	65
23.5.	Corollary 1: Numerical Evaluation at $\Theta = 23.26^\circ$	65
23.6.	Definition 3: Proposed Harmonic Correction of an Orbital Period	66
23.7.	Lemma 2: Relative Period Correction	66
23.8.	Corollary 2: Numerical Evaluation at $\Theta = 23.26^\circ$	67
23.9.	Proposition 1: General Scale-Factor Identity	67
23.10.	Corollary 3: Numerical Scale Factor	68
23.11.	Candidate Faure Theorem VIII: Strong CP Constraint	69
23.12.	Provisional Classification of Extracted Mathematical Statements	69
23.13.	Criterion for Promotion to Established Theorem	70
23.14.	Provisional Theorem Count	71
23.15.	Protocol for Subsequent Source Extraction	71
23.16.	Final Status of the Present Formalization	72
24.	The Faure Universal Tuple	72
24.1.	Universal Components	73
24.2.	Universal Relations	73
24.3.	The Universal Tuple	74
24.4.	Coherence, Stability, and Closure Structures	75

24.5.	The Iterative Component Map	75
24.6.	Image Property of Positive Iterates	76
24.7.	Positive-Iterate Stabilization	77
24.8.	Formal Verification Status	78
25.	Deterministic Structures	79
25.1.	Deterministic State Systems	79
25.2.	Fixed Points and Invariant Sets	80
25.3.	Finite-Time Stabilization	81
25.4.	Global Stabilization	82
25.5.	Idempotence	83
25.6.	Four-Step Stabilization	84
25.7.	The Stabilized State Map	85
25.8.	Linear Systems	86
25.9.	Scope of the Deterministic Results	89
26.	The Faure Spectral Engine	89
26.1.	Spectral Indices and Values	90
26.2.	Spectral Lines	90
26.3.	Spectra	91
26.4.	Spectral Operators	91
26.5.	Faure Spectral Engine	92
26.6.	The Spectral Transition Map	92
26.7.	Canonical Spectral Extraction	93
26.8.	Iterated Canonical Extraction	95
26.9.	Stabilization of the Spectral Engine	96
26.10.	Scope of the Spectral Construction	98
27.	Probability Structures	99
27.1.	Probability Indices	99
27.2.	Probability Data	99
27.3.	Probability Spaces	100
27.4.	Indexed Probability Events	100
27.5.	Indexed Probability Families	101
27.6.	Measurable Transformations	102
27.7.	Iteration of Measurable Transformations	102
27.8.	Probability Operators	103
27.9.	Probability Engines	105
27.10.	Iteration of the Probability Operator	105
27.11.	Probability Preservation Under Iteration	106
27.12.	Properties Not Implied by Indexing	107
27.13.	Scope of the Probability Construction	108
28.	The Faure Operator Engine	109
28.1.	Operator Indices	109

28.2.	Operator Data and Values	109
28.3.	Operator Cells	109
28.4.	Operator Spaces	110
28.5.	Endomorphisms	110
28.6.	Operator Structure	110
28.7.	The Faure Operator Engine	111
28.8.	Iteration of an Endomorphism	111
28.9.	Iteration of Operator Cells	111
28.10.	Well-Definedness of Operator Iteration	111
28.11.	Composition Law for Iterates	111
28.12.	Invariant Subsets	111
28.13.	Bijjective Operators	112
28.14.	Idempotence and Periodicity	112
28.15.	Structural Limitations	112
28.16.	Scope of the Operator Construction	112
29.	The Faure MRDP Generator	112
29.1.	Generator States	113
29.2.	Generator Rules	113
29.3.	MRDP-like Models	113
29.4.	Generator Links	114
29.5.	Universal Generator	114
29.6.	Generator Transitions	114
29.7.	Iterated Generator States	115
29.8.	Structural Theorem	115
30.	The Faure MRDP Engine	115
30.1.	MRDP Indices and Variables	116
30.2.	MRDP Constraints and Spaces	116
30.3.	MRDP Operators and Engines	117
30.4.	Iterated MRDP Constraints	117
30.5.	Structural Theorem	118
31.	Classical vs. Faure Diophantine Structures	118
31.1.	Classical Diophantine Structures	119
31.2.	Faure Diophantine Structures	120
31.3.	The Classical–Faure Bridge	121
31.4.	Structural Theorem	121
31.5.	Structural Limitation	122
32.	The Faure Meta Engine	122
32.1.	Meta Indices and Values	122
32.2.	Meta Cells and Meta Spaces	122
32.3.	Meta Operators and Engines	123
32.4.	Iterated Meta Cells	124

32.5. Structural Theorem	124
33. The Faure Measure Engine	124
33.1. Measure Indices and Values	125
33.2. Measure Cells and Measure Spaces	125
33.3. Measure Operators and Engines	126
33.4. Iterated Measure Cells	126
33.5. Structural Theorem	127
34. The Faure Logic Engine	127
34.1. Logical Indices and Atoms	127
34.2. Logical Clauses and Logical Spaces	128
34.3. Logic Operators and Engines	128
34.4. Iterated Logical Clauses	129
34.5. Structural Theorem	129
35. The Faure Deterministic Kernel	130
35.1. Kernel Points	130
35.2. Deterministic Kernel	130
35.3. Kernel Composition	131
35.4. Kernel Sectors	131
35.5. Kernel Stability	131
35.6. Kernel Closure	132
35.7. Iterated Kernel Points	132
35.8. Structural Theorem	132
36. The Faure Hilbert Structure	133
36.1. Hilbert Variables and Terms	133
36.2. Hilbert Equations and Problems	133
36.3. Faure Hilbert Objects, Relations, and Solutions	134
36.4. Hilbert Bridge	135
36.5. Structural Theorem	135
37. The Faure Global Engine	136
37.1. Global Indices and Values	136
37.2. Global Cells and Global Spaces	136
37.3. Global Operators and Engines	137
37.4. Iterated Global Cells	138
37.5. Structural Theorem	138
38. The Faure Final Engine	138
38.1. Final Indices and Values	139
38.2. Final Cells and Final Spaces	139
38.3. Final Operators and Engines	140
38.4. Iterated Final Cells	140
38.5. Structural Theorem	141
39. The Faure Deterministic Hierarchy	141

39.1.	Hierarchical States	141
39.2.	Deterministic Hierarchical Evolution	142
39.3.	Iteration Algebra	143
39.4.	Invariant Classes	144
39.5.	Level-Dependent Closure	145
39.6.	Deterministic Hierarchy as an Intermediate Structural Layer	146
39.7.	Scope of the Hierarchical Layer	148
40.	Concrete Specialization to the AOS144 Realization	148
40.1.	Finite-Dimensional Realization	148
40.2.	Consequences of the Rank Profile	149
40.3.	Image Filtration and Kernel Dimensions	149
40.4.	Compatibility with the Universal Operator	150
40.5.	Identification with the Universal Operator	150
40.6.	Universal Stabilization from AOS144	151
40.7.	Universal Decomposition	151
40.8.	Conclusion	151
41.	The Universal Stabilized Decomposition	151
41.1.	Universal Operator	151
41.2.	Stabilization Hypothesis	152
41.3.	The Stabilized Operator	152
41.4.	Factorization	152
41.5.	Image and Kernel Structure	153
41.6.	Universal Decomposition	153
41.7.	Explicit Component Formulas	153
41.8.	Finite Stabilization	153
41.9.	Universal Identification via AOS144	154
41.10.	Conclusion	154
42.	The Universal Stabilized Operator	154
42.1.	Universal Algebraic Framework	154
42.2.	Fourth-Order Stabilization	155
42.3.	Polynomial Coprimality	155
42.4.	Image and Kernel Relations	155
42.5.	Universal Stabilized Decomposition	156
42.6.	Idempotent Projection	156
42.7.	Explicit Component Formulas	156
42.8.	Universal Identification via AOS144	157
42.9.	Conclusion	157
43.	Faure Global Engine	158
43.1.	Global Indices, Values, and Cells	158
43.2.	Global Space	158
43.3.	Global Operator and Engine	158

43.4.	Global Evolution	159
43.5.	Structural Lemma	159
43.6.	Space Compatibility	159
43.7.	Iteration Stability	159
43.8.	Structural Interpretation	160
44.	Faure Geometry Engine	160
44.1.	Geometric Indices and Points	160
44.2.	Geometric Cells and Spaces	160
44.3.	Geometric Operators and Engines	160
44.4.	Geometric Evolution	161
44.5.	Structural Lemma	161
44.6.	Space Compatibility	161
44.7.	Geometric Iteration Stability	162
44.8.	Structural Interpretation	162
45.	Faure Final Engine	162
45.1.	Final Indices, Values, and Cells	162
45.2.	Final Space	163
45.3.	Final Operator and Engine	163
45.4.	Final Evolution	163
45.5.	Structural Lemma	163
45.6.	Space Compatibility	164
45.7.	Final Iteration Stability	164
45.8.	Structural Interpretation	164
46.	Faure Deterministic Hierarchy	164
46.1.	Deterministic Hierarchy	164
46.2.	Structural Assumptions	165
46.3.	Origin of the Stabilization Identity	165
46.4.	Master Structural Theorem	166
46.5.	Corollaries	166
46.6.	Relation to the Faure Structural Program	167
47.	Faure Deterministic Model	167
47.1.	Deterministic State Space and Operator	167
47.2.	Deterministic Evolution	168
47.3.	Universal Stabilization	168
47.4.	Deterministic Decomposition	168
47.5.	Deterministic Stabilization Time	168
47.6.	Structural Interpretation	169
48.	Faure Diophantine Lattice	169
48.1.	Diophantine Indices and Values	169
48.2.	Diophantine Lattice Structure	169
48.3.	Diophantine Evolution	170

48.4.	Universal Stabilization in the Diophantine Layer	170
48.5.	Diophantine Decomposition	171
48.6.	Finite-Time Diophantine Stabilization	171
48.7.	Structural Interpretation	171
49.	Faure Differential Engine	171
49.1.	Differential Indices and States	171
49.2.	Differential Increments and Spaces	172
49.3.	Differential Operators and Engines	172
49.4.	Iterated Differential Increments	173
49.5.	Structural Lemma	173
49.6.	Structural Theorem	173
50.	Faure Category Engine	174
50.1.	Categorical Indices and Objects	174
50.2.	Morphisms and Categorical Spaces	174
50.3.	Category Operators and Engines	174
50.4.	Iterated Morphisms	175
50.5.	Structural Lemma	175
50.6.	Structural Theorem	175
51.	Faure Analysis Engine Theorem	176
51.1.	Analytical Indices and Values	176
51.2.	Analytical Cells and Spaces	176
51.3.	Analysis Operators and Engines	176
51.4.	Iterated Analytical Cells	177
51.5.	Structural Lemma	177
51.6.	Structural Theorem	177
52.	Faure Algebra Engine Theorem	178
52.1.	Algebraic Indices and Elements	178
52.2.	Algebraic Operations and Spaces	178
52.3.	Algebra Operators and Engines	179
52.4.	Iterated Algebraic Elements	179
52.5.	Structural Lemma	179
52.6.	Structural Theorem	179
53.	Faure Idempotent Projection Theorem (Additive Form)	180
53.1.	Definitions	180
53.2.	Lemmas	181
53.3.	Idempotence	181
53.4.	Kernel–Image Structure	182
53.5.	Corollaries	183
53.6.	Linear Structure	183
54.	Faure Fourth–Fifth Idempotence Theorem	183
54.1.	Definitions	183

54.2.	Lemmas	183
54.3.	Idempotence	184
54.4.	Consequences of Idempotence	184
54.5.	Kernel–Range Decomposition	185
54.6.	Kernel–Range Structure	186
54.7.	Faure–Fourier Transform	186
55.	Faure Fourier Final Bridge	187
55.1.	Definitions	187
55.2.	Faure–Fourier Transform	187
55.3.	Basic Properties	187
55.4.	Kernel–Range Structure	188
56.	Faure B33 Kernel–Rigidity Closure	190
56.1.	Additive Structure	190
56.2.	Iterates	190
56.3.	Rigidity and Stability	190
56.4.	Faure B33.1	191
56.5.	Faure B33.2	191
56.6.	Faure B33 Final	191
57.	Faure B32 Idempotence Theorem	192
57.1.	Additive Structure	192
57.2.	Iterates	192
57.3.	Rigidity	192
57.4.	Faure B32	193
58.	The Faure Ancestral Theorem	194
59.	Faure Theorem VI: Fully Determined Transport of Relation-Valued Maps	196
60.	Faure Theorem VI: Fully Determined Transport of Relation-Valued Maps	198
61.	Theorem 5.1 — Universal Four-Step Nilpotency	200
61.1.	Reduced–complete system and intertwining	200
61.2.	Iterated intertwining	201
61.3.	Universal four-step nilpotency	201
61.4.	Interpretation	202
62.	Theorem 5.1 — Universal Four-Step Nilpotency	202
	TOE Validator – Faure AOS144 (strumentazione)	203
63.	Structural Modular Projection and Deterministic State Evaluation	203
63.1.	Modular Projection on the Faure Lattice	203
63.2.	Algebraic Structural Identity	204
63.3.	Deterministic State Evaluation	204
63.4.	Universal Faure Tuple	204
63.5.	Structural Verification Criterion	205

64.	Deterministic Numerical Verification on the Universal Tuple U_F	205
64.1.	Discrete Lattice Projection	205
64.2.	Ring Shift Inversion Identity	206
64.3.	Deterministic Metric State Evaluation	206
64.4.	Verification Criterion	206
65.	Structural–Thermodynamic Pipeline for the Faure Water Verification	207
65.1.	Canonical Structural Invariant	207
65.2.	Distinguished Structural State	207
65.3.	Structural–to–Thermodynamic Projection	207
65.4.	Independent Numerical Density Extremum	208
65.5.	Structural vs. Reference Comparison	208
65.6.	Reproducibility Artifacts	209
66.	Canonical Structural State and Thermodynamic Projection	209
66.1.	Distinguished Structural State	209
66.2.	Thermodynamic Projection	209
66.3.	Comparison with an Independent Density Extremum	210
67.	Wielandt Exponent for the AOS144 Matrix	210
68.	Stabilization on the Discrete Lattice	211
69.	Structural–Thermodynamic Separation	211
	Bibliography	212
	Bibliography	212
	References	212
	Epigraph	212
	Epigraph	212
	Author’s Claim	212
	Author’s Claim	212
	Author Identity	213
	Author Identity	213
	Author Information	213
	Author Information	213

```

1  namespace FaureMeta
2
3  /-- Abstract meta index. -/
4  structure MIndex where
5  id   : Nat
6  tag  : Nat
7
8  /-- Abstract meta value. -/

```

```

9      structure MValue where
10     idx  : MIndex
11     data : Type
12
13     -- Abstract meta cell. --
14     structure MCell where
15     id   : Nat
16     val  : MValue
17     info : Type
18
19     -- Abstract meta space. --
20     structure MSpace where
21     cell   : Nat -> MCell
22     label  : Nat
23     payload : Nat -> Type
24
25     -- Abstract meta operator. --
26     structure MOperator where
27     opValue : MValue -> MValue
28     opCell  : MCell -> MCell
29     tag     : Nat
30
31     -- Abstract deterministic meta engine. --
32     structure MEngine where
33     baseSpace : MSpace
34     op        : MOperator
35     mode      : Nat
36     channel   : Nat -> Type
37
38     -- Structural iteration on meta cells. --
39     def iterateCell (E : MEngine) : Nat -> MCell -> MCell
40     | 0, C => C
41     | n + 1, C =>
42     let i := C.id
43     let C2 := E.baseSpace.cell i
44     iterateCell E n C2
45
46     end FaureMeta
47

```

Listing 1. Faure Meta Engine: pure deterministic core

1. Faure Theorem Registry

1.1. *Foundational Faure Theorems.* The foundational Faure theorem family is recorded using the canonical numerical identifiers FT01–FT10. The corresponding Roman-numeral designation is retained as a historical identifier.

The mathematical statements of FT01–FT10 are not reconstructed here until their primary formulations and dependency chains have been independently recovered and verified.

- (1) FT01 – Deterministic Balancing Theorem (Theorem I).
- (2) FT02 – Entropic Invariance Theorem (Theorem II).
- (3) FT03 – Complex-System Coherence Theorem (Theorem III).
- (4) FT04 – Quantum Energy-Field Stabilization Theorem (Theorem IV).
- (5) FT05 – Ideal Saddle-Point Synthesis Theorem (Theorem V).
- (6) FT06 – Harmonic Homogeneity Theorem (Theorem VI).
- (7) FT07 – Angular Metric Constraint Theorem (Theorem VII).
- (8) FT08 – Isomorphic Lattice Invariance Theorem (Theorem VIII).
- (9) FT09 – Stationary-Plane / Volume-Constraint Theorem (Theorem IX).
- (10) FT10 – Higher-Order Deterministic Series and Matrices Theorem (Theorem X).

FT11 and FT12 are retained as historical identifiers in the Faure corpus. No mathematical statement is assigned to them in the present registry until their primary formulations have been recovered and verified.

1.2. *Derived Faure Theorems.* The following results are derived mathematical statements. Their status as mathematical consequences does not, by itself, establish that the hypotheses occur in any particular physical, computational, or empirical system.

THEOREM 1.1 (FDT01 – Faure Iterate Stabilization Theorem). *Let V be a vector space over a field K , and let*

$$T \in \text{End}_K(V).$$

Assume

$$T^4 = T^5.$$

Then

$$T^n = T^4 \quad \text{for every integer } n \geq 4.$$

Proof. We prove the statement by induction on n .

For $n = 4$, the assertion is immediate.

Assume that, for some integer $n \geq 4$,

$$T^n = T^4.$$

Then

$$T^{n+1} = T^n T = T^4 T = T^5 = T^4.$$

Thus the assertion holds for $n + 1$. By induction,

$$T^n = T^4$$

for every integer $n \geq 4$. □

THEOREM 1.2 (FDT02 – Faure Stabilized-Projector Theorem). *Under the hypotheses of FDT01, define*

$$P = T^4.$$

Then

$$P^2 = P.$$

Proof. Since $P = T^4$,

$$P^2 = T^8.$$

By FDT01, since $8 \geq 4$,

$$T^8 = T^4.$$

Therefore

$$P^2 = P.$$

□

THEOREM 1.3 (FDT03 – Faure Annihilating-Polynomial Theorem). *Let $T \in \text{End}_K(V)$. If*

$$T^4 = T^5,$$

then

$$T^4(I - T) = 0.$$

Equivalently, the polynomial

$$p(X) = X^4(1 - X)$$

annihilates T .

Proof. The hypothesis gives

$$T^4 - T^5 = 0.$$

Factoring on the right yields

$$T^4(I - T) = 0.$$

Hence

$$p(T) = T^4(I - T) = 0$$

for

$$p(X) = X^4(1 - X).$$

□

THEOREM 1.4 (FDT04 – Faure 0/1 Primary-Decomposition Theorem).
Let V be a vector space over a field K and let

$$T \in \text{End}_K(V).$$

Assume

$$T^4 = T^5.$$

Then

$$V = \ker(T^4) \oplus \ker(I - T).$$

Proof. By FDT03,

$$T^4(I - T) = 0.$$

We first prove that every vector in V belongs to

$$\ker(T^4) + \ker(I - T).$$

Since the polynomials

$$X^4 \quad \text{and} \quad 1 - X$$

are coprime in $K[X]$, there exist polynomials

$$a(X), b(X) \in K[X]$$

such that

$$a(X)X^4 + b(X)(1 - X) = 1.$$

Evaluating this identity at T gives

$$a(T)T^4 + b(T)(I - T) = I.$$

Therefore, for every $v \in V$,

$$v = a(T)T^4v + b(T)(I - T)v.$$

Because polynomials in T commute with T , we have

$$T^4a(T) = a(T)T^4.$$

Moreover,

$$T^4a(T)T^4v = a(T)T^8v.$$

Since $T^4(I - T) = 0$, FDT01 gives $T^8 = T^4$, and hence

$$T^4a(T)T^4v = a(T)T^4v.$$

Thus

$$a(T)T^4v \in \ker(T^4).$$

Similarly,

$$(I - T)b(T)(I - T)v = b(T)(I - T)^2v.$$

Since

$$(I - T)T^4 = 0$$

and the Bezout identity above implies the corresponding projector

$$b(T)(I - T)$$

has image contained in $\ker(I - T)$, we obtain

$$b(T)(I - T)v \in \ker(I - T).$$

Hence

$$V = \ker(T^4) + \ker(I - T).$$

It remains to prove that the sum is direct. Let

$$v \in \ker(T^4) \cap \ker(I - T).$$

Then

$$T^4v = 0$$

and

$$Tv = v.$$

Consequently,

$$T^4v = v.$$

Thus $v = 0$. Therefore

$$\ker(T^4) \cap \ker(I - T) = \{0\}.$$

Hence

$$V = \ker(T^4) \oplus \ker(I - T).$$

□

THEOREM 1.5 (FDT05 – Faure Kernel-Rigidity Theorem). *Let V be a vector space and let*

$$T \in \text{End}(V).$$

Assume that

$$T^4x = 0 \implies Tx = 0$$

for every $x \in V$. Then

$$\ker(T^4) = \ker(T).$$

Proof. If $x \in \ker(T)$, then

$$Tx = 0,$$

and therefore

$$T^4x = T^3(Tx) = 0.$$

Thus

$$\ker(T) \subseteq \ker(T^4).$$

The hypothesis gives the reverse inclusion

$$\ker(T^4) \subseteq \ker(T).$$

Therefore

$$\ker(T^4) = \ker(T).$$

□

THEOREM 1.6 (FDT06 – Faure Global Idempotence Stabilization Theorem).
Let V be a vector space and let

$$T \in \text{End}(V).$$

If

$$T^2 = T,$$

then

$$T^n = T \quad \text{for every integer } n \geq 1.$$

Proof. For $n = 1$ the assertion is immediate.

Assume that

$$T^n = T$$

for some integer $n \geq 1$. Then

$$T^{n+1} = T^n T = T T = T^2 = T.$$

Induction therefore gives

$$T^n = T$$

for every integer $n \geq 1$. □

THEOREM 1.7 (FDT07 – Faure Dual-Kernel Compatibility Theorem). *Let V be a vector space and let*

$$S, T \in \text{End}(V).$$

Assume

$$S^2 = I$$

and

$$ST = TS.$$

Then

$$S(\ker T) = \ker T.$$

Proof. Let $x \in \ker T$. Then

$$Tx = 0.$$

Using $ST = TS$, we obtain

$$T(Sx) = S(Tx) = 0.$$

Hence

$$Sx \in \ker T.$$

Therefore

$$S(\ker T) \subseteq \ker T.$$

Since

$$S^2 = I,$$

the map S is invertible and

$$S^{-1} = S.$$

Applying the same inclusion to S^{-1} gives

$$S^{-1}(\ker T) \subseteq \ker T.$$

Multiplying by S yields

$$\ker T \subseteq S(\ker T).$$

Consequently,

$$S(\ker T) = \ker T.$$

□

THEOREM 1.8 (FDT08 – Faure Dual-Projector Commutation Theorem).
Let V be a vector space and let

$$S, T \in \text{End}(V).$$

Assume

$$ST = TS$$

and define

$$P = T^4.$$

Then

$$SP = PS.$$

Proof. From

$$ST = TS,$$

an induction on n gives

$$ST^n = T^n S$$

for every integer $n \geq 1$.

Taking $n = 4$ gives

$$ST^4 = T^4 S.$$

Since $P = T^4$, it follows that

$$SP = PS.$$

□

THEOREM 1.9 (FDT09 – Faure 0/1-Spectrum Theorem). *Let V be a finite-dimensional vector space over a field K and let*

$$T \in \text{End}_K(V).$$

Assume

$$T^4 = T^5.$$

Then, with the spectrum taken over K ,

$$\sigma_K(T) \subseteq \{0, 1\}.$$

Proof. By FDT03, the polynomial

$$p(X) = X^4(1 - X)$$

satisfies

$$p(T) = 0.$$

Let

$$\lambda \in \sigma_K(T).$$

Suppose, for contradiction, that

$$\lambda \notin \{0, 1\}.$$

Then

$$p(\lambda) = \lambda^4(1 - \lambda) \neq 0.$$

Consequently, the polynomials

$$X - \lambda \quad \text{and} \quad p(X)$$

are coprime in $K[X]$.

By Bezout's identity, there exist polynomials

$$a(X), b(X) \in K[X]$$

such that

$$a(X)(X - \lambda) + b(X)p(X) = 1.$$

Evaluating at T gives

$$a(T)(T - \lambda I) + b(T)p(T) = I.$$

Since $p(T) = 0$,

$$a(T)(T - \lambda I) = I.$$

Because $a(T)$ commutes with $T - \lambda I$, it also follows that

$$(T - \lambda I)a(T) = I.$$

Thus $T - \lambda I$ is invertible, contradicting

$$\lambda \in \sigma_K(T).$$

Therefore

$$\lambda \in \{0, 1\}.$$

Hence

$$\sigma_K(T) \subseteq \{0, 1\}.$$

□

THEOREM 1.10 (FDT10 – Faure AOS144 Exact-Nilpotency Theorem). *Let $A = A_{\text{AOS144}}$ be a matrix, and let $\nu(A)$ denote its nilpotency index, defined for a nonzero nilpotent matrix as the least positive integer m such that*

$$A^m = 0.$$

Assume

$$A^4 = 0$$

and

$$A^3 \neq 0.$$

Then

$$\nu(A) = 4.$$

Proof. The identity

$$A^4 = 0$$

shows that

$$\nu(A) \leq 4.$$

The condition

$$A^3 \neq 0$$

shows that

$$\nu(A) > 3.$$

Since $\nu(A)$ is an integer,

$$\nu(A) = 4.$$

□

Remark 1.11. FDT10 is a conditional mathematical statement. It does not by itself verify that the concrete matrix A_{AOS144} satisfies

$$A^4 = 0 \quad \text{and} \quad A^3 \neq 0.$$

Verification of those identities for a specific matrix is a separate algebraic or computational task.

THEOREM 1.12 (FDT11 – Faure Spectral Restriction Theorem). *Let V be a finite-dimensional vector space over a field K and let*

$$T \in \text{End}_K(V).$$

If

$$T^4(I - T) = 0,$$

then

$$\sigma_K(T) \subseteq \{0, 1\}.$$

Proof. Let

$$p(X) = X^4(1 - X).$$

The hypothesis is

$$p(T) = 0.$$

Let

$$\lambda \in \sigma_K(T).$$

If

$$\lambda \notin \{0, 1\},$$

then

$$p(\lambda) \neq 0.$$

Hence $X - \lambda$ and $p(X)$ are coprime. By Bezout's identity, there exist $a, b \in K[X]$ such that

$$a(X)(X - \lambda) + b(X)p(X) = 1.$$

Evaluating at T and using $p(T) = 0$ gives

$$a(T)(T - \lambda I) = I.$$

Since the factors commute, $T - \lambda I$ is invertible, contradicting $\lambda \in \sigma_K(T)$.

Therefore

$$\lambda \in \{0, 1\},$$

and hence

$$\sigma_K(T) \subseteq \{0, 1\}.$$

□

Remark 1.13. FDT11 is not counted as an independent new result when FDT03 and FDT09 are already present. It is retained as a general formulation of the spectral consequence of the annihilating-polynomial identity.

COROLLARY 1.14 (FDT12 – Faure Rigidity Decomposition Corollary). *Let V be a vector space over a field K and let*

$$T \in \text{End}_K(V).$$

Assume

$$T^4 = T^5$$

and

$$\ker(T^4) = \ker(T).$$

Then

$$V = \ker(T) \oplus \ker(I - T).$$

Proof. By FDT04,

$$V = \ker(T^4) \oplus \ker(I - T).$$

Using the hypothesis

$$\ker(T^4) = \ker(T),$$

we obtain

$$V = \ker(T) \oplus \ker(I - T).$$

□

THEOREM 1.15 (FDT13 – Faure Circular Addressing Theorem). *Let*

$$N \in \mathbb{N}, \quad N > 0,$$

and let

$$a : \{0, \dots, N - 1\} \rightarrow X.$$

Define

$$C_N : \mathbb{Z} \rightarrow \{0, \dots, N - 1\}$$

by

$$C_N(i) = i \bmod N.$$

Then

$$a(C_N(i))$$

is well-defined for every

$$i \in \mathbb{Z}.$$

Proof. By the Euclidean division theorem, for every

$$i \in \mathbb{Z}$$

there exist unique integers q, r satisfying

$$i = qN + r, \quad 0 \leq r < N.$$

By definition of the Euclidean remainder,

$$C_N(i) = i \bmod N = r.$$

Therefore

$$C_N(i) \in \{0, \dots, N - 1\}.$$

Since a is defined on the entire set

$$\{0, \dots, N - 1\},$$

the value

$$a(C_N(i))$$

is well-defined for every

$$i \in \mathbb{Z}.$$

□

THEOREM 1.16 (FDT14 – Faure Periodic Addressing Theorem). *Under the hypotheses of FDT13,*

$$C_N(i + N) = C_N(i)$$

for every

$$i \in \mathbb{Z}.$$

Proof. By definition,

$$C_N(i + N) = (i + N) \bmod N.$$

Since

$$i + N \equiv i \pmod{N},$$

the two integers have the same Euclidean remainder modulo N . Therefore

$$(i + N) \bmod N = i \bmod N,$$

and hence

$$C_N(i + N) = C_N(i).$$

□

COROLLARY 1.17 (Periodic Array Access). *Under the hypotheses of FDT13,*

$$a(C_N(i + N)) = a(C_N(i))$$

for every

$$i \in \mathbb{Z}.$$

Proof. By FDT14,

$$C_N(i + N) = C_N(i).$$

Applying a to both sides gives

$$a(C_N(i + N)) = a(C_N(i)).$$

□

THEOREM 1.18 (FDT15 – Faure Complete Residue Representative Theorem). *Let*

$$N \in \mathbb{N}, \quad N > 0,$$

and define

$$R_N = \{0, 1, \dots, N - 1\}.$$

Then every congruence class in

$$\mathbb{Z}/N\mathbb{Z}$$

contains exactly one element of R_N .

Proof. Let $i \in \mathbb{Z}$. By the Euclidean division theorem, there exist unique integers q, r such that

$$i = qN + r, \quad 0 \leq r < N.$$

Thus

$$r \in R_N$$

and

$$i \equiv r \pmod{N}.$$

Hence every congruence class modulo N contains at least one element of R_N .

For uniqueness, let

$$r_1, r_2 \in R_N$$

and suppose

$$r_1 \equiv r_2 \pmod{N}.$$

Then

$$N \mid (r_1 - r_2).$$

Because

$$0 \leq r_1 < N \quad \text{and} \quad 0 \leq r_2 < N,$$

we have

$$-(N - 1) \leq r_1 - r_2 \leq N - 1.$$

The only multiple of N in this interval is 0. Therefore

$$r_1 - r_2 = 0,$$

so

$$r_1 = r_2.$$

Thus every congruence class modulo N contains exactly one element of R_N . $\square$

1.3. Additional Derived Results Identified in the Computational Audit.

THEOREM 1.19 (C-FDT01 – AOS144 Exact Primitive Exponent Theorem).

Let

$$A \in \mathbb{N}^{144 \times 144}$$

be a nonnegative matrix. Assume that

$$A^4 > 0$$

entrywise and

$$A^3 \not> 0$$

entrywise. Then A is primitive and its primitive exponent satisfies

$$\gamma(A) = 4.$$

Proof. By definition, a nonnegative matrix is primitive if some positive integer power is strictly positive entrywise. Since

$$A^4 > 0,$$

the matrix A is primitive and

$$\gamma(A) \leq 4.$$

The condition

$$A^3 \not> 0$$

means that A^3 is not strictly positive entrywise. Therefore

$$\gamma(A) > 3.$$

Since $\gamma(A)$ is an integer,

$$\gamma(A) = 4.$$

□

Remark 1.20. C-FDT01 is a conditional theorem. The theorem does not by itself establish that a particular matrix called $A_{\text{AOS}_{144}}$ satisfies the two hypotheses. That verification is a separate computational or algebraic claim.

THEOREM 1.21 (C-FDT02 – Faure Third-Iterate Stabilization Theorem).
Let X be a set and let

$$\Psi : X \rightarrow X.$$

Assume

$$\Psi^4(x) = \Psi^3(x)$$

for every

$$x \in X.$$

Then

$$\Psi^n(x) = \Psi^3(x)$$

for every

$$x \in X$$

and every integer

$$n \geq 3.$$

In particular,

$$\Psi^3(x)$$

is a fixed point of Ψ for every $x \in X$.

Proof. We first show that

$$\Psi^{n+1}(x) = \Psi^n(x)$$

for every $x \in X$ and every $n \geq 3$.

The hypothesis gives

$$\Psi^4(x) = \Psi^3(x).$$

Applying Ψ to both sides yields

$$\Psi^5(x) = \Psi^4(x).$$

Combining the two equalities gives

$$\Psi^5(x) = \Psi^3(x).$$

More generally, suppose that for some $n \geq 3$,

$$\Psi^n(x) = \Psi^3(x).$$

Applying Ψ to both sides gives

$$\Psi^{n+1}(x) = \Psi^4(x).$$

Using the hypothesis,

$$\Psi^{n+1}(x) = \Psi^3(x).$$

Thus, by induction,

$$\Psi^n(x) = \Psi^3(x)$$

for every $n \geq 3$.

Taking $n = 4$ gives

$$\Psi(\Psi^3(x)) = \Psi^4(x) = \Psi^3(x).$$

Therefore $\Psi^3(x)$ is a fixed point of Ψ . □

THEOREM 1.22 (C-FDT03 – Faure Four-Step Nilpotency Transfer Theorem). *Let X_D and R_D be vector spaces over a field K , and let*

$$\Pi_D : X_D \rightarrow R_D, \quad \Psi_D : X_D \rightarrow X_D, \quad \Psi_{\text{red}} : R_D \rightarrow R_D$$

be linear maps.

Assume

$$\Pi_D \circ \Psi_D = \Psi_{\text{red}} \circ \Pi_D,$$

that Π_D is injective, and that

$$\Psi_{\text{red}}^4 = 0.$$

Then

$$\Psi_D^4 = 0.$$

Proof. We first prove by induction on $n \geq 1$ that

$$\Pi_D \circ \Psi_D^n = \Psi_{\text{red}}^n \circ \Pi_D.$$

For $n = 1$, this is precisely the assumed intertwining relation

$$\Pi_D \circ \Psi_D = \Psi_{\text{red}} \circ \Pi_D.$$

Assume the identity holds for some $n \geq 1$. Then

$$\begin{aligned} \Pi_D \circ \Psi_D^{n+1} &= (\Pi_D \circ \Psi_D^n) \circ \Psi_D \\ &= (\Psi_{\text{red}}^n \circ \Pi_D) \circ \Psi_D \\ &= \Psi_{\text{red}}^n \circ (\Pi_D \circ \Psi_D) \\ &= \Psi_{\text{red}}^n \circ (\Psi_{\text{red}} \circ \Pi_D) \\ &= \Psi_{\text{red}}^{n+1} \circ \Pi_D. \end{aligned}$$

Thus the identity holds for every $n \geq 1$.

For $n = 4$,

$$\Pi_D \circ \Psi_D^4 = \Psi_{\text{red}}^4 \circ \Pi_D.$$

Since

$$\Psi_{\text{red}}^4 = 0,$$

we obtain

$$\Pi_D \circ \Psi_D^4 = 0.$$

Therefore, for every $x \in X_D$,

$$\Pi_D(\Psi_D^4 x) = 0.$$

Because Π_D is injective,

$$\Psi_D^4 x = 0.$$

Hence

$$\Psi_D^4 = 0.$$

□

1.4. *Fourth-order stabilization and canonical decomposition.* Let V be a vector space over a field K , and let $T: V \rightarrow V$ be a linear operator satisfying

$$(1) \quad T^4 = T^5.$$

LEMMA 1.23 (Fourth-order stabilization). *Under (1),*

$$T^n = T^4 \quad \text{for every } n \geq 4.$$

Proof. For every integer $n \geq 4$, multiplication of $T^4 = T^5$ on the left by T^{n-4} gives

$$T^n = T^{n+1}.$$

Hence

$$T^4 = T^5 = T^6 = \dots = T^n$$

for every $n \geq 4$. □

LEMMA 1.24 (The stabilized operator is a projection). *Define*

$$P := T^4.$$

Then

$$P^2 = P$$

and

$$PT = TP.$$

Proof. By the preceding lemma,

$$P^2 = T^8 = T^4 = P.$$

Moreover,

$$PT = T^5 = T^4 = TP,$$

where the first and last equalities follow from $P = T^4$ and the middle equality from $T^4 = T^5$. □

THEOREM 1.25 (Canonical decomposition). *Let V be a vector space over a field K , and let $T: V \rightarrow V$ be linear. If*

$$T^4 = T^5,$$

then

$$V = \ker(T^4) \oplus \ker(I - T).$$

Proof. The polynomial identity

$$X^4(1 - X)$$

annihilates T , since

$$T^4(I - T) = T^4 - T^5 = 0.$$

The polynomials X^4 and $1 - X$ are coprime in $K[X]$. Hence, by Bézout's identity, there exist polynomials $a, b \in K[X]$ such that

$$a(X)X^4 + b(X)(1 - X) = 1.$$

Substituting T for X yields

$$a(T)T^4 + b(T)(I - T) = I.$$

Let $v \in V$. Then

$$v = a(T)T^4v + b(T)(I - T)v.$$

Since $a(T)$ commutes with T ,

$$(I - T)a(T)T^4v = a(T)(I - T)T^4v = a(T)(T^4 - T^5)v = 0.$$

Thus

$$a(T)T^4v \in \ker(I - T).$$

Similarly,

$$T^4 b(T)(I - T)v = b(T)T^4(I - T)v = b(T)(T^4 - T^5)v = 0,$$

so

$$b(T)(I - T)v \in \ker(T^4).$$

Therefore

$$V = \ker(T^4) + \ker(I - T).$$

It remains to prove that the sum is direct. Suppose

$$v \in \ker(T^4) \cap \ker(I - T).$$

Then

$$T^4 v = 0$$

and

$$Tv = v.$$

Consequently,

$$T^4 v = v.$$

Hence $v = 0$, and therefore

$$\ker(T^4) \cap \ker(I - T) = \{0\}.$$

The sum is direct. □

COROLLARY 1.26 (Stabilized component). *Under the hypotheses of Theorem 1.25, every $v \in V$ has a unique decomposition*

$$v = v_{\text{tr}} + v_{\text{st}},$$

where

$$v_{\text{tr}} \in \ker(T^4), \quad v_{\text{st}} \in \ker(I - T).$$

Moreover,

$$T^4 v = v_{\text{st}}$$

and

$$T^n v = v_{\text{st}} \quad \text{for every } n \geq 4.$$

Proof. By the canonical decomposition,

$$v = v_{\text{tr}} + v_{\text{st}}$$

with the stated properties. Since $v_{\text{tr}} \in \ker(T^4)$ and $v_{\text{st}} \in \ker(I - T)$,

$$T^4 v = T^4 v_{\text{tr}} + T^4 v_{\text{st}} = 0 + v_{\text{st}} = v_{\text{st}}.$$

The fourth-order stabilization lemma then gives

$$T^n v = T^4 v = v_{\text{st}} \quad (n \geq 4).$$

□

PROPOSITION 1.27 (Equality of kernels under an additional hypothesis).
Suppose, in addition to $T^4 = T^5$, that

$$T^4v = 0 \implies Tv = 0 \quad \text{for every } v \in V.$$

Then

$$\ker(T^4) = \ker(T)$$

and consequently

$$V = \ker(T) \oplus \ker(I - T).$$

Proof. Since $Tv = 0$ implies $T^4v = 0$, one has

$$\ker(T) \subseteq \ker(T^4).$$

The additional hypothesis gives the reverse inclusion. Hence

$$\ker(T) = \ker(T^4).$$

The asserted decomposition follows from Theorem 1.25. $\square$

1.5. *Spectral consequence.* The following statement requires the usual spectral setting, in which $\sigma(T)$ denotes the set of scalars λ for which $T - \lambda I$ is not invertible.

THEOREM 1.28 (Spectral restriction). *Let T be a linear operator in a setting where the spectrum $\sigma(T)$ is defined by invertibility of $T - \lambda I$. If*

$$T^4(I - T) = 0,$$

then

$$\sigma(T) \subseteq \{0, 1\}.$$

Proof. Let $\lambda \in \sigma(T)$. Suppose, for contradiction, that

$$\lambda \notin \{0, 1\}.$$

Then

$$\lambda^4(1 - \lambda) \neq 0.$$

Consequently, the polynomials

$$X - \lambda \quad \text{and} \quad X^4(1 - X)$$

are coprime in $K[X]$. Hence there exist polynomials $a, b \in K[X]$ such that

$$a(X)(X - \lambda) + b(X)X^4(1 - X) = 1.$$

Evaluating at T gives

$$a(T)(T - \lambda I) + b(T)T^4(I - T) = I.$$

Since $T^4(I - T) = 0$,

$$a(T)(T - \lambda I) = I.$$

All polynomials in T commute, so also

$$(T - \lambda I)a(T) = I.$$

Thus $T - \lambda I$ is invertible, contradicting $\lambda \in \sigma(T)$.

Therefore

$$\sigma(T) \subseteq \{0, 1\}.$$

□

1.6. Conditional nilpotency statement.

PROPOSITION 1.29 (Nilpotency index). *Let A be a linear operator. If*

$$A^4 = 0 \quad \text{and} \quad A^3 \neq 0,$$

then the nilpotency index of A is equal to 4.

Proof. By definition, the nilpotency index is the least positive integer m such that $A^m = 0$. The hypotheses show that $m \leq 4$ and $m > 3$. Hence $m = 4$. □

1.7. Iterated-map stabilization.

PROPOSITION 1.30 (Stabilization of iterates). *Let X be a set and let $\Psi: X \rightarrow X$ satisfy*

$$\Psi^4(x) = \Psi^3(x) \quad \text{for every } x \in X.$$

Then

$$\Psi^n(x) = \Psi^3(x) \quad \text{for every } x \in X \text{ and every } n \geq 3.$$

In particular,

$$\Psi(\Psi^3(x)) = \Psi^3(x).$$

Proof. We proceed by induction on $n \geq 3$. The assertion for $n = 3$ is immediate.

Assume that

$$\Psi^n(x) = \Psi^3(x)$$

for every $x \in X$. Then

$$\Psi^{n+1}(x) = \Psi(\Psi^n(x)) = \Psi(\Psi^3(x)) = \Psi^4(x) = \Psi^3(x).$$

Thus the assertion holds for $n + 1$, completing the induction.

Taking $n = 4$ in the hypothesis gives

$$\Psi(\Psi^3(x)) = \Psi^4(x) = \Psi^3(x).$$

□

1.8. *Intertwining transfer.*

PROPOSITION 1.31 (Transfer of fourth-order nilpotency). *Let X_D and R_D be vector spaces, let*

$$\Psi_D: X_D \rightarrow X_D, \quad \Psi_{\text{red}}: R_D \rightarrow R_D$$

be linear maps, and let

$$\Pi_D: X_D \rightarrow R_D$$

be an injective linear map satisfying

$$\Pi_D \circ \Psi_D = \Psi_{\text{red}} \circ \Pi_D.$$

If

$$\Psi_{\text{red}}^4 = 0,$$

then

$$\Psi_D^4 = 0.$$

Proof. Iterating the intertwining relation gives

$$\Pi_D \circ \Psi_D^4 = \Psi_{\text{red}}^4 \circ \Pi_D.$$

Since $\Psi_{\text{red}}^4 = 0$,

$$\Pi_D \circ \Psi_D^4 = 0.$$

Therefore, for every $x \in X_D$,

$$\Pi_D(\Psi_D^4 x) = 0.$$

The injectivity of Π_D implies

$$\Psi_D^4 x = 0.$$

Since x was arbitrary,

$$\Psi_D^4 = 0.$$

□

1.9. *Empirical Falsifiability Protocol.*

Definition 1.32 (EMP01 – Structural–Thermodynamic Comparison Protocol). Let T^* denote a mathematically defined structural temperature and let T_{max} denote an independently measured empirical temperature. Define

$$\Delta T = T^* - T_{\text{max}}.$$

The quantity ΔT is an empirical comparison observable and is not, by this definition, a mathematical theorem.

Remark 1.33. The definition of ΔT does not establish a mathematical identity or a physical law. A falsifiable empirical prediction requires explicit definitions of the observables, estimation procedures, uncertainties, and rejection criteria.

1.10. *Registry Status.* The present registry distinguishes mathematical validity from empirical verification and from claims of originality.

- FT01–FT10 are canonical foundational identifiers whose primary mathematical formulations remain to be recovered and independently verified.
- FDT01–FDT10 are mathematically established conditional results within the stated hypotheses.
- FDT11 is a general spectral formulation substantially overlapping with FDT09 and is therefore not counted as an independent new theorem.
- FDT12 is a corollary of FDT04 and the stated kernel-rigidity hypothesis.
- FDT13–FDT15 are mathematically valid standard results of modular arithmetic and are not, by themselves, evidence of a new mathematical theorem.
- C-FDT01 is a conditional theorem; verification of the hypotheses for a specific matrix remains a separate task.
- C-FDT02 is a general stabilization theorem for a self-map.
- C-FDT03 is a transfer theorem whose hypotheses explicitly require vector-space structure and injectivity of the projection.
- EMP01 is an empirical protocol definition and is not counted as a mathematical theorem.

2. Faure Theorem Registry

2.1. *Foundational Faure Theorems.* The foundational Faure theorem family is recorded using the canonical Roman-numeral identifiers used in the source corpus.

- (1) FT-I – Deterministic Balancing Theorem.
- (2) FT-II – Entropic Invariance Theorem.
- (3) FT-III – Complex-System Coherence Theorem.
- (4) FT-IV – Quantum Energy-Field Stabilization Theorem.
- (5) FT-V – Ideal Saddle-Point Synthesis Theorem.
- (6) FT-VI – Harmonic Homogeneity Theorem.
- (7) FT-VII – Angular Metric Constraint Theorem.
- (8) FT-VIII – Isomorphic Lattice Invariance Theorem.
- (9) FT-IX – Stationary-Plane / Volume-Constraint Theorem.
- (10) FT-X – Higher-Order Deterministic Series and Matrices Theorem.

FT-XI and FT-XII are retained as canonical identifiers in the Faure corpus but are not assigned reconstructed mathematical statements until their primary formulations have been recovered.

2.2. *Derived Faure Theorems.* The derived theorem registry uses the prefix FDT.

- (1) FDT01 – Faure Iterate Stabilization Theorem.
- (2) FDT13 – Faure Circular Addressing Theorem.

- (3) FDT14 – Faure Periodic Addressing Theorem.
- (4) FDT15 – Faure Complete Residue Representative Theorem.

Capitolo 0. Struttura Prima della Manifestazione

0.1. Premessa. Il programma TOE Faure parte dalla domanda: *la realtà osservata è la struttura, oppure la sua manifestazione?* La proposta è che esista una struttura relazionale primordiale, della quale le manifestazioni costituiscono proiezioni o rappresentazioni.

0.2. Livelli della teoria. Una teoria completa richiede una catena verificabile:

IDEA $\rightarrow$ MODEL $\rightarrow$ MATHEMATICS $\rightarrow$ AXIOMS $\rightarrow$ FORMALIZATION $\rightarrow$ LEAN $\rightarrow$ COMPUTATION

0.3. Formalizzazione Lean. La formalizzazione in Lean 4 elimina ambiguità e assunzioni implicite. Un teorema è accettato solo quando il kernel verifica la prova a partire da definizioni e ipotesi esplicite.

0.4. Identità strutturale. L'identità fondamentale

$$\Psi^4 = \Psi^5$$

impone

$$\Psi^4(I - \Psi) = 0,$$

generando vincoli strutturali su iterati, decomposizioni e spettro.

0.5. Rigidità del kernel. La distinzione tra $\ker(\Psi^4)$ e $\ker(\Psi)$ è essenziale. L'implicazione

$$\Psi^4 x = 0 \implies \Psi x = 0$$

elimina settori nilpotenti non desiderati.

0.6. Stabilizzazione degli iterati. Dalla relazione $\Psi^4 = \Psi^5$ segue

$$\Psi^4 = \Psi^5 = \Psi^6 = \Psi^7 = \dots,$$

e più in generale

$$\Psi^n = \Psi^{n+1} \quad (n \geq 4).$$

0.7. Dualità con l'operatore S . Con $S^2 = I$ e la commutazione $S\Psi = \Psi S$, emergono settori congiunti

$$\text{Fix}(S) \cap \text{Fix}(\Psi).$$

0.8. *Quattro settori.* I valori naturali sono:

$$S: \{+1, -1\}, \quad \Psi: \{0, 1\},$$

generando i settori:

$$(+1, 0), (+1, 1), (-1, 0), (-1, 1).$$

0.9. *Struttura prima della manifestazione.* La struttura genera le manifestazioni. Le manifestazioni non determinano la struttura.

0.10. *AOS144-HDFP.* Il livello computazionale AOS144-HDFP realizza la struttura formale in forma eseguibile, creando la catena:

struttura $\rightarrow$ forma matematica $\rightarrow$ prova formale $\rightarrow$ kernel computazionale $\rightarrow$ output osservabile.

0.11. *Validazione.* La validazione avviene su tre livelli distinti:

- matematica (Lean 4);
- computazione (AOS144, Python `Decimal`);
- osservazione fisica.

0.12. *Obiettivo scientifico.* L'obiettivo non è una singola equazione, ma una struttura testabile, falsificabile, riproducibile.

0.13. *Il principio Faure.*

First identify the structure. Then identify its invariants. Then formalize them.

Then compute them. Then test their manifestations.

3. Faure Theorem I: Structural Significance of Mathematical Results

3.1. *Structural Functionals.* Let $\mathcal{R}$ be a class of mathematical results and let $\mathcal{P}$ be a class of problems.

Definition 3.1 (Structural Scope). A *structural scope functional* is any map

$$\Sigma : \mathcal{R} \longrightarrow \mathbb{R}_{\geq 0}.$$

No semantic or physical interpretation of Σ is assumed a priori.

Definition 3.2 (Problem Difficulty). A *difficulty functional* is any map

$$\Delta : \mathcal{P} \longrightarrow \mathbb{R}_{\geq 0}.$$

Definition 3.3 (Structural Relevance). A *structural relevance functional* is any map

$$\mathcal{I} : \mathcal{R} \longrightarrow \mathbb{R}_{\geq 0}$$

such that $\mathcal{I}$ depends only on Σ and not on Δ .

3.2. Monotonia strutturale.

Definition 3.4 (Monotonia rispetto alla struttura). A functional $\mathcal{I} : \mathcal{R} \rightarrow \mathbb{R}_{\geq 0}$ is *monotone with respect to Σ* if

$$\Sigma(R_1) \leq \Sigma(R_2) \implies \mathcal{I}(R_1) \leq \mathcal{I}(R_2)$$

for all $R_1, R_2 \in \mathcal{R}$.

LEMMA 3.5 (Canonical structural lift). *Given any structural scope functional $\Sigma : \mathcal{R} \rightarrow \mathbb{R}_{\geq 0}$, the map*

$$\mathcal{I}_\Sigma(R) := \Sigma(R)$$

is a structural relevance functional monotone with respect to Σ .

Proof. By definition,

$$\mathcal{I}_\Sigma(R_1) = \Sigma(R_1), \quad \mathcal{I}_\Sigma(R_2) = \Sigma(R_2).$$

Thus

$$\Sigma(R_1) \leq \Sigma(R_2) \implies \mathcal{I}_\Sigma(R_1) \leq \mathcal{I}_\Sigma(R_2).$$

Since $\mathcal{I}_\Sigma$ is defined solely in terms of Σ , it is independent of Δ . $\square$

3.3. Faure Theorem I.

THEOREM 3.6 (Faure I: Structural Index of Mathematical Results). *Let $\mathcal{R}$ be a class of mathematical results and let*

$$\Sigma : \mathcal{R} \rightarrow \mathbb{R}_{\geq 0}$$

be a structural scope functional. Then there exists a structural relevance functional

$$\mathcal{I} : \mathcal{R} \longrightarrow \mathbb{R}_{\geq 0}$$

such that:

(1) $\mathcal{I}$ is monotone with respect to Σ , i.e.

$$\Sigma(R_1) \leq \Sigma(R_2) \implies \mathcal{I}(R_1) \leq \mathcal{I}(R_2);$$

- (2) $\mathcal{I}$ is independent of any difficulty functional Δ on $\mathcal{P}$;
- (3) $\mathcal{I}$ can be chosen canonically as $\mathcal{I} = \mathcal{I}_\Sigma$.

Proof. Define

$$\mathcal{I}(R) := \Sigma(R).$$

By Lemma 3.5, $\mathcal{I}$ is monotone with respect to Σ and depends only on Σ . No reference to Δ is used in the definition, hence $\mathcal{I}$ is independent of any difficulty functional. $\square$

Remark 3.7. Faure Theorem I isolates a purely structural index of mathematical results: it depends only on the chosen scope functional Σ and is insensitive to any notion of problem difficulty. In the Faure program, this separates the *structural weight* of a result from its *perceived hardness*, allowing the hierarchy and the universal operator to act on structural invariants rather than subjective measures of difficulty.

4. Faure Theorem II: Injective Observation and Deterministic Distinguishability

4.1. Deterministic Systems and Observations.

Definition 4.1 (Finite Deterministic System). Let X be a finite set and let

$$T : X \rightarrow X$$

be a deterministic map.

Definition 4.2 (Injective Observation). Let

$$\Phi : X \rightarrow Y$$

be an observation map. It is *injective* if

$$\Phi(x) = \Phi(x') \implies x = x'$$

for all $x, x' \in X$.

4.2. Aliasing and Distinguishability.

LEMMA 4.3 (Injectivity Eliminates Aliasing). *Let $\Phi : X \rightarrow Y$ be injective. If*

$$\Phi(x) = \Phi(x'),$$

then

$$x = x'.$$

Proof. This is exactly the definition of injectivity. $\square$

Definition 4.4 (Deterministic Aliasing). Given two deterministic maps $T_1, T_2 : X \rightarrow X$ and an observation map $\Phi : X \rightarrow Y$, we say that *deterministic aliasing* occurs if

$$\Phi \circ T_1 = \Phi \circ T_2$$

while $T_1 \neq T_2$.

4.3. Faure Theorem II.

THEOREM 4.5 (Faure II: Injective Observation Prevents Deterministic Aliasing). *Let $T_1, T_2 : X \rightarrow X$ be deterministic maps and let*

$$\Phi : X \rightarrow Y$$

be injective. If

$$\Phi \circ T_1 = \Phi \circ T_2,$$

then

$$T_1 = T_2.$$

In particular, no deterministic aliasing can occur under an injective observation.

Proof. For every $x \in X$,

$$\Phi(T_1(x)) = \Phi(T_2(x)).$$

By Lemma 4.3,

$$T_1(x) = T_2(x).$$

Since this holds for all $x \in X$, we conclude

$$T_1 = T_2.$$

Hence deterministic aliasing, as defined above, is impossible when Φ is injective. $\square$

Remark 4.6. Faure Theorem II isolates a purely structural condition under which deterministic dynamics remain distinguishable at the observational level: injectivity of Φ is sufficient to prevent different successor maps from collapsing to the same observed evolution. In the Faure program, this separates genuine structural degeneracy in the dynamics from mere observational aliasing, and

clarifies that any loss of distinguishability must come from the dynamics themselves, not from the observation layer.

5. Faure Theorem III: Injective Encoding and Structural Transport

5.1. *Deterministic State Spaces.* Let X be a finite deterministic state space and let

$$T : X \rightarrow X$$

be a deterministic update rule.

Let

$$\Phi : X \rightarrow Y$$

be an encoding map into a representation space Y .

Definition 5.1 (Injective Encoding). The encoding Φ is *injective* if

$$\Phi(x) = \Phi(x') \implies x = x'.$$

5.2. *Structural Transport.*

Definition 5.2 (Encoding Transport). The encoding Φ *transports* the deterministic update rule T if

$$\Phi \circ T = S \circ \Phi$$

for some map $S : Y \rightarrow Y$.

This is the standard engine-morphism condition in the Faure architecture.

5.3. *Injective Encoding and Transport Uniqueness.*

LEMMA 5.3 (Injectivity Eliminates Representational Aliasing). *If Φ is injective and*

$$\Phi(x) = \Phi(x'),$$

then

$$x = x'.$$

Proof. Immediate from injectivity. □

THEOREM 5.4 (Faure III: Injective Encoding Yields Unique Transport). *Let $T_1, T_2 : X \rightarrow X$ be deterministic update rules and let*

$$\Phi : X \rightarrow Y$$

be injective.

Suppose both update rules transport through Φ , i.e.

$$\Phi \circ T_1 = S \circ \Phi, \quad \Phi \circ T_2 = S \circ \Phi$$

for the same structural map $S : Y \rightarrow Y$.

Then

$$T_1 = T_2.$$

Proof. For every $x \in X$,

$$\Phi(T_1(x)) = S(\Phi(x)) = \Phi(T_2(x)).$$

By Lemma 5.3,

$$T_1(x) = T_2(x).$$

Thus $T_1 = T_2$. □

Remark 5.5. Faure Theorem III is the structural version of Theorem II: injective encoding not only preserves distinguishability of states, but also guarantees *uniqueness of transported dynamics*.

In the Faure architecture, this ensures that:

- deterministic dynamics in X - transported through an injective encoding Φ - into a representation layer Y cannot collapse into multiple distinct preimages.

This is the foundational mechanism preventing representational degeneracy in the deterministic $\rightarrow$ global $\rightarrow$ geometric $\rightarrow$ final engine chain.

6. Faure Theorem IV: Injective Observation and Equality Preservation

6.1. Deterministic Systems and Observations.

Definition 6.1 (Finite Deterministic System). Let X be a finite set and let

$$T : X \rightarrow X$$

be a deterministic map.

Definition 6.2 (Injective Observation). Let

$$\Phi : X \rightarrow Y$$

be an observation map. It is *injective* if

$$\Phi(x) = \Phi(x') \implies x = x'$$

for all $x, x' \in X$.

6.2. *Aliasing and Equality.*

LEMMA 6.3 (Injectivity Eliminates Aliasing). *If Φ is injective and*

$$\Phi(x) = \Phi(x'),$$

then

$$x = x'.$$

Proof. This is exactly the definition of injectivity. □

6.3. *Faure Theorem IV.*

THEOREM 6.4 (Faure IV: Injective Observation Preserves Equality Under Dynamics). *Let $T : X \rightarrow X$ be deterministic and let $\Phi : X \rightarrow Y$ be injective. Then for all $x, x' \in X$,*

$$\Phi(T(x)) = \Phi(T(x')) \iff T(x) = T(x').$$

In particular, equality of successors in X is equivalent to equality of their observed images in Y .

Proof. Suppose first that

$$\Phi(T(x)) = \Phi(T(x')).$$

By Lemma 6.3,

$$T(x) = T(x').$$

Conversely, if $T(x) = T(x')$, then applying Φ to both sides gives

$$\Phi(T(x)) = \Phi(T(x')).$$

Thus the equivalence holds. □

Remark 6.5. Faure Theorem IV states that an injective observation layer does not distort equality of deterministic successors: two states evolve to the same successor if and only if their observed successors coincide. Nel programma Faure questo isola la parte “pura” dell’osservazione: ogni perdita di informazione strutturale deve provenire dalla dinamica, non dall’osservazione, quando Φ è iniettiva.

7. Faure Theorem V: Injective Coding and Structural Transport

7.1. *Deterministic State Space.* Let X be a finite deterministic state space and let

$$T : X \rightarrow X$$

be a deterministic update rule.

Let

$$\kappa : X \rightarrow \{0, 1\}^*$$

be a coding map into the space of finite binary strings.

Definition 7.1 (Injective Coding). The coding κ is *injective* if

$$\kappa(x) = \kappa(x') \implies x = x'.$$

7.2. *Structural Transport Through Coding.*

Definition 7.2 (Coding Transport). The coding κ *transports* the deterministic update rule T if

$$\kappa \circ T = S \circ \kappa$$

for some map $S : \{0, 1\}^* \rightarrow \{0, 1\}^*$.

This is the standard engine-morphism condition in the Faure architecture.

7.3. *Injectivity and Transport Uniqueness.*

LEMMA 7.3 (Injectivity Eliminates Coding Aliasing). *If κ is injective and*

$$\kappa(x) = \kappa(x'),$$

then

$$x = x'.$$

Proof. Immediate from injectivity. □

7.4. *Faure Theorem V.*

THEOREM 7.4 (Faure V: Injective Coding Yields Unique Transport). *Let $T_1, T_2 : X \rightarrow X$ be deterministic update rules and let*

$$\kappa : X \rightarrow \{0, 1\}^*$$

be injective.

Suppose both update rules transport through κ , i.e.

$$\kappa \circ T_1 = S \circ \kappa, \quad \kappa \circ T_2 = S \circ \kappa$$

for the same structural map $S : \{0, 1\}^* \rightarrow \{0, 1\}^*$.

Then

$$T_1 = T_2.$$

Proof. For every $x \in X$,

$$\kappa(T_1(x)) = S(\kappa(x)) = \kappa(T_2(x)).$$

By Lemma 7.3,

$$T_1(x) = T_2(x).$$

Thus $T_1 = T_2$. □

Remark 7.5. Faure Theorem V is the coding analogue of Theorem III: injective coding not only preserves distinguishability of states, but also guarantees *uniqueness of transported dynamics*.

In the Faure architecture, this ensures that deterministic dynamics in X cannot collapse into multiple distinct preimages when represented as binary strings. This is essential for the deterministic $\rightarrow$ global $\rightarrow$ geometric $\rightarrow$ final engine chain, where binary encodings serve as canonical transport layers.

8. Faure Theorem VI: Injective Observation and Structural Transport of Deterministic Iterates

8.1. *Deterministic State Space.* Let X be a finite deterministic state space and let

$$T : X \rightarrow X$$

be a deterministic update rule.

Define the iterates

$$T^n := \underbrace{T \circ T \circ \cdots \circ T}_{n \text{ times}}.$$

8.2. *Injective Observation.* Let

$$\Phi : X \rightarrow Y$$

be an observation map.

Definition 8.1 (Injective Observation). The observation Φ is *injective* if

$$\Phi(x) = \Phi(x') \implies x = x'.$$

8.3. Structural Transport.

Definition 8.2 (Observation Transport). The observation Φ *transports* the deterministic update rule T if

$$\Phi \circ T = S \circ \Phi$$

for some map $S : Y \rightarrow Y$.

This is the standard engine-morphism condition in the Faure architecture.

8.4. Injectivity and Iterative Transport.

LEMMA 8.3 (Injectivity Eliminates Observational Aliasing). *If Φ is injective and*

$$\Phi(x) = \Phi(x'),$$

then

$$x = x'.$$

Proof. Immediate from injectivity. □

8.5. Faure Theorem VI.

THEOREM 8.4 (Faure VI: Injective Observation Yields Unique Transport of Iterates). *Let $T : X \rightarrow X$ be deterministic and let $\Phi : X \rightarrow Y$ be injective. Suppose Φ transports T , i.e.*

$$\Phi \circ T = S \circ \Phi$$

for some structural map $S : Y \rightarrow Y$.

Then for all $x, x' \in X$ and all $n \geq 1$,

$$\Phi(T^n(x)) = \Phi(T^n(x')) \iff T^n(x) = T^n(x').$$

Proof. Iterating the transport identity gives

$$\Phi(T^n(x)) = S^n(\Phi(x)).$$

Thus

$$\Phi(T^n(x)) = \Phi(T^n(x')) \iff S^n(\Phi(x)) = S^n(\Phi(x')).$$

By injectivity of Φ and the transport identity,

$$S^n(\Phi(x)) = S^n(\Phi(x')) \implies T^n(x) = T^n(x').$$

Conversely, if $T^n(x) = T^n(x')$, applying Φ gives the reverse implication. $\square$

Remark 8.5. Faure Theorem VI is the iterative version of Theorem IV: injective observation not only preserves equality of successors, but also preserves equality of all transported iterates.

In the Faure architecture, this ensures that the deterministic $\rightarrow$ global $\rightarrow$ geometric $\rightarrow$ final engine chain does not collapse distinct trajectories when the observation layer is injective.

9. Faure Theorem VII: Commutativity of Translation Operators in Structural Representation

9.1. *Translation Operators on the Torus.* Let

$$T^2 := \mathbb{R}^2 / \mathbb{Z}^2$$

be the two-dimensional torus with its abelian group structure. For $\delta \in T^2$ define the translation operator

$$(T_\delta f)(x) := f(x + \delta), \quad f \in C(T^2, \mathbb{C}), \quad x \in T^2.$$

LEMMA 9.1 (Commutativity of Torus Translations). *For all $\delta, \eta \in T^2$,*

$$T_\delta T_\eta = T_\eta T_\delta.$$

Proof. Since T^2 is abelian,

$$x + \delta + \eta = x + \eta + \delta.$$

Thus the operators commute. $\square$

9.2. *Faure Theorem VII.*

THEOREM 9.2 (Faure VII: Structural Commutativity of Translation-Generated Operators). *Let*

$$A = \sum_{k=1}^N a_k T_{\delta_k}, \quad B = \sum_{\ell=1}^M b_\ell T_{\eta_\ell}$$

be finite translation-generated operators.

Then

$$[A, B] = 0.$$

Moreover, the commutativity is structural: it arises from the abelian nature of the underlying representation space and is preserved under all Faure engine morphisms.

Proof. By bilinearity and Lemma 9.1,

$$T_{\delta_k} T_{\eta_\ell} = T_{\eta_\ell} T_{\delta_k}.$$

Thus $AB = BA$. □

Remark 9.3. In the Faure architecture, translation operators appear as geometric representations of deterministic transport. Their commutativity ensures that geometric evolution layers do not introduce non-deterministic interference.

10. Faure Theorem VIII: Rank Bounds and Structural Incompleteness

10.1. *Rank and Spectrum.* Let V be a finite-dimensional vector space and let $A : V \rightarrow V$ be linear.

Definition 10.1 (Rank).

$$\text{rank}(A) := \dim(\text{Ran}(A)).$$

LEMMA 10.2 (Rank Subspace). *If $\text{rank}(A) = r$, then $\text{Ran}(A)$ is an r -dimensional subspace.*

10.2. *Faure Theorem VIII.*

THEOREM 10.3 (Faure VIII: Rank Bounds Do Not Determine Structural Operators). *Let $\{A_i\}$ be a family of operators with extremal rank bound R .*

Then extremal rank alone does not determine the family up to structural equivalence: additional invariants (spectral, nilpotent, stabilized) are required.

In particular, even in finite dimension, two operators may share the same rank but differ in spectrum, stabilization profile, or structural transport behavior.

Proof. Consider

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}.$$

Both have rank 1, but

$$\sigma(A) = \{0, 1\}, \quad \sigma(B) = \{0, 2\}.$$

Thus rank does not determine spectral structure. □

Remark 10.4. In the Faure architecture, rank is insufficient to classify operators: the stabilized operator Ψ requires full spectral and nilpotent data.

11. Faure Theorem IX: Deterministic Attractors in the Psi-Hierarchy

11.1. *Deterministic Evolution in the Faure Architecture.* Let (V, Ψ) be a deterministic Faure state space, where

$$\Psi : V \rightarrow V$$

is the universal stabilized operator satisfying

$$\Psi^4 = \Psi^5.$$

Define the deterministic evolution

$$v_{t+1} = \Psi(v_t), \quad t \in \mathbb{N}.$$

11.2. *Stabilized Attractor.*

Definition 11.1 (Stabilized Attractor). The stabilized attractor of Ψ is

$$A := \text{im}(\Psi^4).$$

Since $\Psi^4 = \Psi^5$, A is invariant:

$$\Psi(A) = A.$$

11.3. *Deterministic Convergence.*

LEMMA 11.2 (Four-Step Deterministic Convergence). *For every $v \in V$,*

$$\Psi^t(v) = \Psi^4(v) \quad (t \geq 4).$$

Proof. Immediate from $\Psi^4 = \Psi^5$. □

11.4. *Faure Theorem IX.*

THEOREM 11.3 (Faure IX: Deterministic Attractor Representation). *Let $G : V \rightarrow \mathbb{R}$ be any observable compatible with Ψ , i.e.*

$$G(\Psi(v)) = G(\Psi^4(v)).$$

Then for every trajectory $v_{t+1} = \Psi(v_t)$,

$$G(v_t) \longrightarrow G(\Psi^4(v_0)) \quad (t \rightarrow \infty).$$

Thus the asymptotic behavior of G is determined entirely by the stabilized attractor

$$A = \text{im}(\Psi^4).$$

Remark 11.4. Faure IX shows that “delayed generalization” is a stabilized phenomenon: Every observable compatible with Psi converges in finite time to its stabilized value. No probabilistic assumptions are required.

12. Faure Theorem X: Intersection Lattices in the 144-Structure

12.1. *The 144 Lattice.* Let Λ_{144} be the Faure deterministic lattice induced by the AOS144 operator.

Let $L_1, L_2 \subseteq \Lambda_{144}$ be deterministic sublattices.

Define their intersection

$$L := L_1 \cap L_2.$$

12.2. *Structural Membership.*

Definition 12.1 (Structural Membership Signature). Define

$$\sigma(x) := (\mathbf{1}_{L_1}(x), \mathbf{1}_{L_2}(x)).$$

LEMMA 12.2 (Intersection Characterization).

$$\sigma(x) = (1, 1) \iff x \in L.$$

Proof. Immediate. □

12.3. *Faure Theorem X.*

THEOREM 12.3 (Faure X: Structural Indistinguishability in the Intersection). *For any $x, y \in L$,*

$$\sigma(x) = \sigma(y) = (1, 1).$$

Thus membership tests in L_1 and L_2 cannot distinguish elements of the intersection L .

In the Faure architecture, this means that the intersection lattice behaves as a structural kernel: all elements of L are observationally indistinguishable under the membership signature.

Remark 12.4. This is the lattice analogue of kernel indistinguishability in the stabilized operator Psi.

13. Faure Theorem XI: Simulation and Structural Transport of Discrete Dynamics

13.1. *Deterministic Dynamics.* Let (X, T) be a deterministic system.

Let

$$\Phi : X \rightarrow Y$$

be a representation map.

13.2. *Structural Transport.*

Definition 13.1 (Structural Transport). A map $S : Y \rightarrow Y$ is a structural transport of T through Φ if

$$S \circ \Phi = \Phi \circ T.$$

This is the engine-morphism condition.

13.3. *Faure Theorem XI.*

THEOREM 13.2 (Faure XI: Existence and Uniqueness of Structural Transport). *Assume Φ is injective.*

Then:

1. *A structural transport exists:*

$$S = \Phi \circ T \circ \Phi^{-1}.$$

2. *The transport is unique.*

3. *The transported dynamics preserve deterministic distinguishability:*

$$\Phi(T(x)) = \Phi(T(x')) \iff T(x) = T(x').$$

Proof. Injectivity gives existence of Φ^{-1} on $\Phi(X)$. Uniqueness follows from the transport identity. Distinguishability follows from injectivity. $\square$

Remark 13.3. Faure XI is the structural version of exact simulation: injective representation guarantees unique deterministic transport across engine layers.

14. **Faure Theorem XII: Rational Discrete Data and Structural Coordinates**

14.1. *Deterministic Rational Data.* Let (X, d) be a deterministic rational metric space, meaning

$$d(x, y) \in \mathbb{Q}_{\geq 0}$$

for all x, y in X .

Let

$$\Phi : X \rightarrow V$$

be a structural coordinate map into a Faure state space (V, Ψ) , where the universal stabilized operator Ψ satisfies

$$\Psi^4 = \Psi^5.$$

14.2. Structural Constraint.

Definition 14.1 (Psi-Compatible Coordinate). A coordinate map Φ is said to be Ψ -compatible if

$$\Phi(\Psi(x)) = \Psi(\Phi(x))$$

for all x in X .

14.3. Faure Theorem XII.

THEOREM 14.2 (Faure XII: Rational Data Do Not Constrain Structural Coordinates). *Let (X, d) be rational and let $\Phi : X \rightarrow V$ be any injective Ψ -compatible coordinate map.*

Then the rationality of d imposes no restriction on the values of $\Phi(x)$: they may be rational, irrational, algebraic, or transcendental.

Structural coordinates in V are determined entirely by Ψ -compatibility and the stabilized structure of V , not by the arithmetic nature of the underlying metric.

Remark 14.3. Faure XII separates intrinsic discrete data (rational metric) from structural representation (Ψ -coordinates). Rationality does not propagate through Ψ : the stabilized operator erases all arithmetic constraints not encoded in its transport structure.

15. Faure Theorem XIII: Deterministic Transitions on the Stabilized Energy Lattice

15.1. The Stabilized Energy Lattice.

Definition 15.1 (Energy Lattice). Let

$$E = \{e_k : k \in \mathbb{Z}\}$$

be the stabilized Faure energy lattice, where each index k labels a unique deterministic energy state.

15.2. Deterministic Translation.

Definition 15.2 (Lattice Translation). For a fixed integer m , define the deterministic translation map

$$T_m : E \rightarrow E, \quad T_m(e_k) = e_{k+m}.$$

15.3. Structural Well-Definedness.

LEMMA 15.3 (Well-Definedness of Translation). *For all integers k and m , the element e_{k+m} is uniquely defined in E .*

Proof. By definition, E contains exactly one element e_j for each integer j . Thus $k + m$ determines a unique state e_{k+m} . $\square$

15.4. Faure Theorem XIII.

THEOREM 15.4 (Faure XIII: Deterministic Lattice Transition). *For all integers k and m , the transition*

$$e_k \longmapsto e_{k+m}$$

under T_m is uniquely determined and deterministic.

Proof. By Lemma 15.3, the target state e_{k+m} is uniquely defined. Since T_m is a function, each input e_k has exactly one output:

$$T_m(e_k) = e_{k+m}.$$

Thus the transition is deterministic and unique. $\square$

Remark 15.5. The deterministic definition of T_m specifies only the structural transport on the lattice E . Introducing a probability measure on E would constitute an additional probabilistic model. Deterministic lattice translation alone does not imply any probabilistic statement.

16. Faure Theorem XIV: Low-Density Families in the Psi-Hierarchy

16.1. Binary State Space.

Definition 16.1 (Binary Configuration Space). For $N \geq 1$, define

$$X_N = \{0, 1\}^N.$$

Definition 16.2 (Hamming Weight Density). For $x = (x_1, \dots, x_N) \in X_N$, define

$$H_N(x) = \frac{1}{N} \sum_{i=1}^N x_i.$$

16.2. *Combinatorial Structure.*

LEMMA 16.3 (Counting Low-Weight Configurations). *For each $k \in \{0, 1, \dots, N\}$, the number of configurations in X_N with exactly k ones is $\binom{N}{k}$.*

Proof. A configuration with exactly k ones is determined by choosing the k positions where the value is 1. There are $\binom{N}{k}$ such choices. $\square$

LEMMA 16.4 (Density Bound). *If $x \in X_N$ satisfies*

$$\sum_{i=1}^N x_i \leq \lfloor \varepsilon N \rfloor,$$

then

$$H_N(x) \leq \varepsilon.$$

Proof. By definition,

$$H_N(x) = \frac{1}{N} \sum_{i=1}^N x_i \leq \frac{\lfloor \varepsilon N \rfloor}{N} \leq \varepsilon.$$

$\square$

16.3. *Faure Theorem XIV.*

THEOREM 16.5 (Faure XIV: Exponentially Large Psi-Stable Low-Density Family). *Let $0 < \varepsilon < 1$. For every $N \geq 1$, define*

$$C_N = \left\{ x \in X_N : \sum_{i=1}^N x_i \leq \lfloor \varepsilon N \rfloor \right\}.$$

Then:

- (1) *Every $x \in C_N$ satisfies $H_N(x) \leq \varepsilon$.*
- (2) *The cardinality of C_N is*

$$|C_N| = \sum_{k=0}^{\lfloor \varepsilon N \rfloor} \binom{N}{k}.$$

- (3) *For fixed $0 < \varepsilon < 1/2$, the size of C_N grows exponentially in N .*
- (4) *The family C_N is stable under the stabilized operator Ψ :*

$$\Psi(C_N) = C_N.$$

Proof. Items (1)–(3) follow from Lemmas 16.4 and 16.3.

For (4), note that Ψ acts coordinatewise and preserves the value 0. Since Ψ cannot increase the number of coordinates equal to 1, it follows that

$$\sum_{i=1}^N (\Psi(x))_i \leq \sum_{i=1}^N x_i \leq \lfloor \varepsilon N \rfloor,$$

so $\Psi(x) \in C_N$. Thus $\Psi(C_N) = C_N$. $\square$

Remark 16.6. Faure XIV identifies low-density families as exponentially large deterministic subspaces preserved by the stabilized operator Ψ . Their combinatorial size and structural stability make them fundamental components of the Psi-hierarchy.

17. Faure Theorem XV: Pushforward of Deterministic Dynamics

17.1. Deterministic System.

Definition 17.1 (Finite Deterministic System). Let X be a finite set and let

$$T : X \rightarrow X$$

be a deterministic map.

17.2. Structural Projection.

Definition 17.2 (Projection). Let

$$\pi : X \rightarrow Y$$

be a surjective map onto a finite set Y . The fibers $\pi^{-1}(y)$ represent coarse-grained structural classes.

LEMMA 17.3 (Partition of Fibers). *The family of fibers*

$$\{ \pi^{-1}(y) : y \in Y \}$$

forms a partition of X .

Proof. For each $x \in X$, there exists a unique $y = \pi(x)$ such that $x \in \pi^{-1}(y)$. The fibers are pairwise disjoint and cover X . $\square$

17.3. Pushforward Dynamics.

Definition 17.4 (Pushforward Dynamics). Define the pushforward (coarse-grained) dynamics $S : Y \rightarrow Y$ by

$$S(y) = \pi(T(x))$$

for any $x \in \pi^{-1}(y)$.

LEMMA 17.5 (Well-Definedness of Pushforward). *The map S is well-defined if and only if*

$$\pi(T(x_1)) = \pi(T(x_2))$$

for all $x_1, x_2 \in \pi^{-1}(y)$ and all $y \in Y$.

Proof. $S(y)$ must not depend on the choice of representative $x \in \pi^{-1}(y)$. This is equivalent to the stated condition. $\square$

17.4. Faure Theorem XV.

THEOREM 17.6 (Faure XV: Pushforward of Deterministic Dynamics). *Let $T : X \rightarrow X$ be deterministic and let $\pi : X \rightarrow Y$ be a surjective projection. Assume the compatibility condition*

$$\pi(T(x_1)) = \pi(T(x_2)) \quad \text{for all } x_1, x_2 \in \pi^{-1}(y), y \in Y.$$

Then:

- (1) *The pushforward dynamics $S : Y \rightarrow Y$ is well-defined.*
- (2) *S satisfies the commutation relation*

$$S \circ \pi = \pi \circ T.$$

- (3) *If Ψ is a stabilized operator on X satisfying $\Psi^4 = \Psi^5$, then the pushforward dynamics is Ψ -compatible:*

$$\pi(\Psi(T(x))) = S(\Psi(\pi(x))).$$

- (4) *If π is bijective, then*

$$S = \pi \circ T \circ \pi^{-1}.$$

Proof. (1) and (2) follow from Lemma 17.5.

For (3), note that Ψ commutes with coarse-graining:

$$\Psi(\pi(x)) = \pi(\Psi(x)).$$

Thus

$$S(\Psi(\pi(x))) = S(\pi(\Psi(x))) = \pi(T(\Psi(x))) = \pi(\Psi(T(x))).$$

For (4), bijectivity of π gives the identity immediately. $\square$

Remark 17.7. Faure XV shows that coarse-graining via π produces a deterministic pushforward dynamics S whenever the fibers are dynamically compatible. The stabilized operator Ψ commutes with this pushforward, preserving deterministic structure under projection.

18. Faure Theorem XVI: Discrepancy and Uniform Boundedness

18.1. Structural Sign Sequences.

Definition 18.1 (Sign Sequence). A sign sequence is a function

$$x : \mathbb{N} \rightarrow \{-1, +1\}.$$

Definition 18.2 (Arithmetic-Progression Sum). For $d, n \geq 1$, define the homogeneous progression sum

$$S_d(n) = \sum_{k=1}^n x(kd).$$

Definition 18.3 (Discrepancy). The discrepancy of x is

$$D(x) = \sup_{d, n \geq 1} |S_d(n)|.$$

18.2. Structural Bounds.

LEMMA 18.4 (Uniform Boundedness Implies Finite Supremum). *If there exists $C < \infty$ such that*

$$|S_d(n)| \leq C \quad \text{for all } d, n \geq 1,$$

then

$$D(x) \leq C.$$

Proof. By definition,

$$D(x) = \sup_{d, n \geq 1} |S_d(n)|.$$

If every term is bounded by C , the supremum is also bounded by C . $\square$

LEMMA 18.5 (Finite Supremum Implies Uniform Boundedness). *If*

$$D(x) < \infty,$$

then

$$|S_d(n)| \leq D(x) \quad \text{for all } d, n \geq 1.$$

Proof. Immediate from the definition of the supremum. $\square$

18.3. Faure Theorem XVI.

THEOREM 18.6 (Faure XVI: Equivalence of Bounded Discrepancy and Uniform Control). *For a sign sequence x , the following are equivalent:*

(1) *There exists $C < \infty$ such that*

$$|S_d(n)| \leq C \quad \text{for all } d, n \geq 1.$$

(2) *The discrepancy is finite:*

$$D(x) < \infty.$$

(3) *The sequence is stabilized under the operator Ψ :*

$$\Psi(x) = x,$$

where Ψ satisfies $\Psi^4 = \Psi^5$ and acts coordinatewise.

Proof. (1) $\Rightarrow$ (2) follows from Lemma 18.4. (2) $\Rightarrow$ (1) follows from Lemma 18.5.

To show (2) $\Rightarrow$ (3): Finite discrepancy implies uniform boundedness of all progression sums. Since Ψ preserves signs and cannot increase the magnitude of any partial sum,

$$|S_d(n)| = |S_d(n; \Psi(x))|,$$

so $\Psi(x)$ has the same discrepancy as x . Stabilization $\Psi^4 = \Psi^5$ forces fixed-point behavior on bounded sequences, hence $\Psi(x) = x$.

Finally, (3) $\Rightarrow$ (1): If $\Psi(x) = x$, then all progression sums are stabilized and uniformly bounded. $\square$

Remark 18.7. Faure XVI shows that bounded discrepancy is not merely a combinatorial condition: it is equivalent to stabilization under the universal operator Ψ . Thus discrepancy becomes a structural invariant of the Ψ -hierarchy.

19. Faure Theorem XVII: Discrete Lattice Dynamics and Angular Observables

19.1. Deterministic Lattice Structure.

Definition 19.1 (Lattice). A lattice in $\mathbb{R}^3$ is a discrete additive subgroup

$$L \subseteq \mathbb{R}^3.$$

Definition 19.2 (Deterministic Lattice Orbit). A deterministic lattice orbit is a sequence

$$\gamma : \mathbb{N} \rightarrow L$$

satisfying

$$\gamma(n+1) = \gamma(n) + v_n, \quad v_n \in L.$$

19.2. Structural Closure.

LEMMA 19.3 (Closure Under Addition). *If L is a lattice, then for all $a, b \in L$,*

$$a + b \in L.$$

Proof. Since L is an additive subgroup of $\mathbb{R}^3$, it is closed under addition. $\square$

PROPOSITION 19.4 (Discrete Evolution). *If $\gamma(0) \in L$ and $v_n \in L$ for all n , then*

$$\gamma(n) \in L \quad \text{for all } n \in \mathbb{N}.$$

Proof. By induction. Base case: $\gamma(0) \in L$ by hypothesis. Inductive step: if $\gamma(n) \in L$ and $v_n \in L$, then by Lemma 19.3,

$$\gamma(n+1) = \gamma(n) + v_n \in L.$$

$\square$

19.3. Angular Observables.

Definition 19.5 (Angular Observable). Let $\Theta : L \rightarrow \mathbb{R}/2\pi\mathbb{Z}$ be an angular observable defined by

$$\Theta(a) = \arg(a_1 + ia_2),$$

where $a = (a_1, a_2, a_3) \in L$.

Definition 19.6 (Stabilized Angular Observable). Let Ψ be a stabilized operator satisfying $\Psi^4 = \Psi^5$. Define the stabilized angular observable

$$\Theta_\Psi(a) = \Theta(\Psi(a)).$$

19.4. Faure Theorem XVII.

THEOREM 19.7 (Faure XVII: Angular Drift in Discrete Lattice Dynamics). *Let γ be a deterministic lattice orbit and let Θ be an angular observable. Then:*

(1) *The angular sequence*

$$\Theta(\gamma(n))$$

may exhibit nontrivial drift even though $\gamma(n)$ evolves on a discrete lattice.

(2) *If Ψ is a stabilized operator, the stabilized angular observable satisfies*

$$\Theta_{\Psi}(\gamma(n+1)) - \Theta_{\Psi}(\gamma(n)) = \Theta(\Psi(v_n)).$$

(3) *If $\Psi(v_n)$ has constant angular component, then $\Theta_{\Psi}(\gamma(n))$ is a discrete linear drift.*

(4) *If $\Psi(v_n)$ varies periodically, then $\Theta_{\Psi}(\gamma(n))$ exhibits a stabilized angular precession.*

Proof. (1) follows because Θ is nonlinear and sensitive to direction, not magnitude.

For (2),

$$\Theta_{\Psi}(\gamma(n+1)) = \Theta(\Psi(\gamma(n+1))) = \Theta(\Psi(\gamma(n) + v_n)) = \Theta(\Psi(\gamma(n)) + \Psi(v_n)).$$

Since Θ depends only on the angular component,

$$\Theta(\Psi(\gamma(n)) + \Psi(v_n)) - \Theta(\Psi(\gamma(n))) = \Theta(\Psi(v_n)).$$

Items (3) and (4) follow from the structure of the increments $\Psi(v_n)$. $\square$

Remark 19.8. Faure XVII shows that discrete lattice dynamics can produce stabilized angular drift or precession when viewed through Θ_{Ψ} . Thus discreteness of the underlying lattice does not imply trivial angular behavior: the observable structure and the stabilized operator Ψ determine the angular dynamics.

20. Faure Theorem XVIII: Discrete Closure, Stabilized Dynamics, and Injective Observation

20.1. Finite Discrete Structure.

Definition 20.1 (Finite Grid). For $N \geq 1$, define the discrete grid

$$G_N = \{1, \dots, N\} \times \{1, \dots, N\}.$$

Definition 20.2 (Adjacency). For $p = (i, j)$ and $q = (i', j')$ in G_N , write

$$p \sim q$$

if

$$|i - i'| + |j - j'| = 1.$$

Definition 20.3 (Adjacency-Preserving Map). A map $K : G_N \rightarrow G_N$ is adjacency-preserving if

$$p \sim q \implies K(p) \sim K(q).$$

20.2. Injective Observation.

Definition 20.4 (Injective Observation). Let $\Phi : G_N \rightarrow Y$ be an observation map. It is injective if

$$\Phi(p) = \Phi(q) \implies p = q.$$

LEMMA 20.5 (Adjacency Closure). *If $p \sim q$, then p and q differ in exactly one coordinate by one unit.*

Proof. Immediate from the definition of adjacency. □

20.3. Stabilized Dynamics.

Definition 20.6 (Stabilized Operator). Let Ψ be a stabilized operator acting on G_N and satisfying

$$\Psi^4 = \Psi^5.$$

Definition 20.7 (Stabilized Dynamics). Given $K : G_N \rightarrow G_N$, define the stabilized dynamics

$$K_\Psi(p) = \Psi(K(p)).$$

LEMMA 20.8 (Psi-Preservation of Adjacency). *If K is adjacency-preserving and Ψ preserves adjacency, then*

$$p \sim q \implies K_\Psi(p) \sim K_\Psi(q).$$

Proof. If $p \sim q$, then $K(p) \sim K(q)$ by adjacency preservation. Applying Ψ preserves adjacency, so

$$K_\Psi(p) = \Psi(K(p)) \sim \Psi(K(q)) = K_\Psi(q).$$

□

20.4. Faure Theorem XVIII.

THEOREM 20.9 (Faure XVIII: Stabilized Discrete Closure and Injective Observation). *Let $K : G_N \rightarrow G_N$ be adjacency-preserving, let Ψ be stabilized, and let $\Phi : G_N \rightarrow Y$ be injective. Then:*

(1) *The stabilized evolution*

$$p \mapsto K_\Psi(p)$$

is uniquely determined.

(2) *The observable evolution*

$$p \mapsto \Phi(K_\Psi(p))$$

is uniquely determined.

(3) *If two stabilized dynamics produce identical observations,*

$$\Phi(K_\Psi(p)) = \Phi(K'_\Psi(p)) \quad \text{for all } p,$$

then

$$K_\Psi = K'_\Psi.$$

(4) *In particular, injective observation eliminates all discrete aliasing:*

$$\Phi \circ K_\Psi = \Phi \circ K'_\Psi \implies K_\Psi = K'_\Psi.$$

Proof. (1) follows because K is a function and Ψ is a function.

(2) follows because Φ is injective, so each $K_\Psi(p)$ has a unique observable value.

(3) and (4) follow from injectivity: if $\Phi(K_\Psi(p)) = \Phi(K'_\Psi(p))$, then

$$K_\Psi(p) = K'_\Psi(p)$$

for all p , hence the maps coincide. $\square$

Remark 20.10. Faure XVIII shows that adjacency-preserving dynamics, when stabilized by Ψ and observed injectively, produce a fully deterministic observable evolution. Injectivity eliminates aliasing, and stabilization ensures structural coherence across the discrete grid. Thus discrete closure and injective observation form a deterministic transport layer in the Ψ -hierarchy.

21. Faure Theorem XIX: Finite State Spaces, Stabilized Observation, and Output Bandwidth

21.1. Finite Deterministic Structure.

Definition 21.1 (Finite State Space). Let H be a finite deterministic state space.

Definition 21.2 (Observation Map). Let

$$C : H \rightarrow Y$$

be an observation map into an output space Y .

Definition 21.3 (Output Bandwidth). The output bandwidth of C is

$$B(C) = |\text{Im}(C)|.$$

21.2. *Structural Bounds.*

LEMMA 21.4 (Image Cardinality Bound). *For any function $C : H \rightarrow Y$ with H finite,*

$$|\text{Im}(C)| \leq |H|.$$

Proof. Each element of $\text{Im}(C)$ is the image of at least one element of H . Thus $\text{Im}(C)$ is in bijection with a subset of H , so its cardinality cannot exceed $|H|$. $\square$

21.3. *Stabilized Observation.*

Definition 21.5 (Stabilized Operator). Let Ψ be a stabilized operator acting on H and satisfying

$$\Psi^4 = \Psi^5.$$

Definition 21.6 (Stabilized Observation). Define the stabilized observation map

$$C_\Psi(h) = C(\Psi(h)).$$

LEMMA 21.7 (Psi-Preservation of Bandwidth). *For any $C : H \rightarrow Y$,*

$$B(C_\Psi) \leq B(C).$$

Proof. Since Ψ is a function,

$$\text{Im}(C_\Psi) = C(\Psi(H)) \subseteq C(H) = \text{Im}(C).$$

Thus $|\text{Im}(C_\Psi)| \leq |\text{Im}(C)|$. $\square$

21.4. *Faure Theorem XIX.*

THEOREM 21.8 (Faure XIX: Structural Output Bandwidth Bound). *Let H be finite, let $C : H \rightarrow Y$ be an observation map, and let Ψ be a stabilized operator. Then:*

(1) *The output bandwidth satisfies*

$$B(C) \leq |H|.$$

(2) *The stabilized bandwidth satisfies*

$$B(C_\Psi) \leq B(C).$$

(3) *If Ψ has a fixed-point structure on H , then*

$$B(C_\Psi) = B(C).$$

(4) *If Ψ collapses multiple states into a single stabilized class, then*

$$B(C_\Psi) < B(C).$$

Proof. (1) follows from Lemma 21.4. (2) follows from Lemma 21.7.

For (3), if $\Psi(h) = h$ for all h , then $C_\Psi = C$ and the bandwidth is unchanged.

For (4), if Ψ identifies distinct states h_1, h_2 into a single stabilized state, then $C_\Psi(h_1) = C_\Psi(h_2)$, reducing the size of the image. $\square$

Remark 21.9. Faure XIX shows that output bandwidth is a structural invariant of finite deterministic systems, and that stabilization under Ψ can only reduce or preserve bandwidth. Thus bandwidth becomes a structural measure of how much information survives stabilization in the Ψ -hierarchy.

22. Faure Theorem XX: Deterministic Primordial Dynamics and Injective Structural Coding

22.1. Primordial Deterministic Dynamics.

Definition 22.1 (Finite Primordial State Space). Let M be a finite primordial state space.

Definition 22.2 (Deterministic Primordial Evolution). A deterministic evolution on M is a map

$$K : M \rightarrow M.$$

22.2. Structural Coding.

Definition 22.3 (Injective Structural Coding). Let

$$\kappa : M \rightarrow \{0, 1\}^*$$

be an injective structural coding map.

Injectivity means:

$$\kappa(x) = \kappa(y) \implies x = y.$$

22.3. *Compatibility with Primordial Transport.* Let $F = (A, A', S, S', R, R', T, \phi)$ be a Primordial Faure Tuple. A coding κ is said to be *transport-compatible* if

$$\kappa(T(a)) = \kappa(T(a')) \implies T(a) = T(a').$$

22.4. *Faure Theorem XX.*

THEOREM 22.4 (Faure XX: Injective Coding Preserves Deterministic Successors). *Let $K : M \rightarrow M$ be deterministic and let $\kappa : M \rightarrow \{0, 1\}^*$ be injective. Define the coded successor map*

$$x \mapsto \kappa(K(x)).$$

Then for all $x, y \in M$,

$$\kappa(K(x)) = \kappa(K(y)) \iff K(x) = K(y).$$

In particular, injective coding preserves equality of deterministic successors and eliminates aliasing at the primordial level.

Proof. Since κ is injective,

$$\kappa(K(x)) = \kappa(K(y)) \implies K(x) = K(y).$$

The reverse implication is trivial because equal arguments have equal images. $\square$

22.5. *Remark.* Injective coding preserves the causal structure of deterministic primordial dynamics. No loss of information occurs at the coding level. Non-injective codings may collapse distinct primordial states and therefore cannot preserve deterministic successor equality.

23. Preliminary Mathematical Formalization of the Faure Framework

23.1. *Epistemic and Mathematical Status.* The purpose of this section is to separate mathematical objects, mathematical identities, propositions, conjectures, and empirical claims occurring in the source material.

A formula appearing in a source document is not regarded as a theorem merely because it is called a “theorem” in that source. For the present formalization, a mathematical statement is promoted to theorem status only when its hypotheses, definitions, domain of validity, and logical derivation have been explicitly established.

The distinction between mathematical validity and empirical validity is maintained throughout.

Formal status rule.

A statement is classified according to the following hierarchy: definition, identity, lemma, proposition, theorem, corollary, conjecture, or empirical claim.

In particular, numerical agreement with an observation does not by itself constitute a mathematical proof of a physical law.

23.2. *Definition 1: Harmonic Correction Factor.*

Definition 1.

Let

$$\Theta \in \mathbb{R}.$$

The harmonic correction factor is defined by

$$C_F(\Theta) := 1 + \sin \Theta.$$

Since

$$-1 \leq \sin \Theta \leq 1,$$

it follows immediately that

$$0 \leq C_F(\Theta) \leq 2.$$

Proof.

The elementary inequality

$$-1 \leq \sin \Theta \leq 1$$

holds for every real Θ . Adding 1 to all three members gives

$$0 \leq 1 + \sin \Theta \leq 2.$$

By Definition 1,

$$0 \leq C_F(\Theta) \leq 2.$$

□

23.3. *Definition 2: Proposed Faure-Type Gravitational Correction.* Let

$$G > 0, \quad m_1 > 0, \quad m_2 > 0, \quad r > 0,$$

and let

$$\Theta \in \mathbb{R}.$$

Define the Newtonian reference quantity by

$$F_N(m_1, m_2, r) := G \frac{m_1 m_2}{r^2}.$$

Define the proposed Faure-type corrected quantity by

$$F_F(m_1, m_2, r, \Theta) := G \frac{m_1 m_2}{r^2} (1 + \sin \Theta).$$

Equivalently,

$$F_F = F_N C_F(\Theta).$$

Mathematical status.

The equation above is a definition of a proposed model quantity. It is not, by itself, a proof that the factor

$$1 + \sin \Theta$$

is a physical law.

23.4. Lemma 1: Relative Harmonic Correction.

Lemma 1.

Under the definitions above,

$$\frac{F_F - F_N}{F_N} = \sin \Theta.$$

Proof.

By definition,

$$F_F = F_N(1 + \sin \Theta).$$

Therefore

$$F_F - F_N = F_N \sin \Theta.$$

Since

$$G > 0, \quad m_1 > 0, \quad m_2 > 0, \quad r > 0,$$

we have

$$F_N > 0.$$

Division by F_N is therefore legitimate and yields

$$\frac{F_F - F_N}{F_N} = \sin \Theta.$$

□

23.5. Corollary 1: Numerical Evaluation at $\Theta = 23.26^\circ$. For

$$\Theta = 23.26^\circ,$$

one obtains numerically

$$\sin(23.26^\circ) \approx 0.394.$$

Consequently,

$$C_F(23.26^\circ) = 1 + \sin(23.26^\circ) \approx 1.394.$$

If the reference value is

$$F_N \approx 1.982 \times 10^{20} \text{ N},$$

then

$$F_F \approx (1.982 \times 10^{20}) (1.394) \approx 2.762 \times 10^{20} \text{ N}.$$

Hence

$$F_F - F_N \approx 7.80 \times 10^{19} \text{ N}.$$

Status of Corollary 1.

The numerical result is an exact numerical consequence of the proposed definition of F_F .

It does not establish independently that

$$1 + \sin \Theta$$

is a physical law.

23.6. Definition 3: Proposed Harmonic Correction of an Orbital Period.

Let

$$G > 0, \quad M > 0, \quad r > 0.$$

Define the Keplerian reference period

$$T_K(r, M) = 2\pi \sqrt{\frac{r^3}{GM}}.$$

Define the proposed corrected period

$$T_F(r, M, \Theta) = 2\pi \sqrt{\frac{r^3}{GM}} (1 + \cos \Theta).$$

Equivalently,

$$T_F = T_K(1 + \cos \Theta).$$

Mathematical status.

The displayed expression defines a proposed corrected period. Its physical interpretation requires independent justification.

23.7. Lemma 2: Relative Period Correction.

Lemma 2.

For every

$$\Theta \in \mathbb{R},$$

for which the expressions are defined,

$$\frac{T_F - T_K}{T_K} = \cos \Theta.$$

Proof.

From

$$T_F = T_K(1 + \cos \Theta),$$

we obtain

$$T_F - T_K = T_K \cos \Theta.$$

Because

$$G > 0, \quad M > 0, \quad r > 0,$$

the Keplerian period satisfies

$$T_K > 0.$$

Division by T_K therefore gives

$$\frac{T_F - T_K}{T_K} = \cos \Theta.$$

□

23.8. *Corollary 2: Numerical Evaluation at $\Theta = 23.26^\circ$.* For

$$\Theta = 23.26^\circ,$$

one has approximately

$$\cos(23.26^\circ) \approx 0.9197.$$

Thus

$$1 + \cos(23.26^\circ) \approx 1.9197.$$

If

$$T_K \approx 7.49 \times 10^5 \text{ s},$$

then

$$T_F \approx (7.49 \times 10^5) (1.9197) \approx 1.438 \times 10^6 \text{ s}.$$

Therefore

$$T_F \approx 16.64 \text{ days}.$$

Status of Corollary 2.

The calculation establishes the numerical consequence of the proposed formula. It does not establish independently that the formula describes the physical lunar orbit.

23.9. *Proposition 1: General Scale-Factor Identity.* Let

$$Q_M > 0$$

be a model prediction and let

$$Q_R > 0$$

be a reference or measured quantity.

Define

$$s := \frac{Q_R}{Q_M}.$$

Proposition 1.

Under the hypotheses

$$Q_M > 0, \quad Q_R > 0,$$

the definition

$$s := \frac{Q_R}{Q_M}$$

implies

$$Q_R = sQ_M.$$

Proof.

By definition,

$$s = \frac{Q_R}{Q_M}.$$

Since $Q_M > 0$, multiplication by Q_M gives

$$sQ_M = Q_R.$$

□

23.10. *Corollary 3: Numerical Scale Factor.* Suppose

$$Q_M = 16.7 \text{ days}$$

and

$$Q_R = 27.32 \text{ days}.$$

Then

$$s = \frac{27.32}{16.7} \approx 1.636.$$

Consequently,

$$Q_R \approx 1.636 Q_M.$$

Qualification.

A scale factor determined from one observable does not imply that the same factor applies universally to other observables.

A universal scaling hypothesis would require, for an explicitly specified family of observables,

$$Q_R^{(i)} = sQ_M^{(i)}$$

for every relevant index i , together with independent verification.

23.11. *Candidate Faure Theorem VIII: Strong CP Constraint.* The source material proposes the condition

$$\theta = 0$$

as a consequence of a geometric constraint associated with the Faure lattice framework.

CANDIDATE FAURE THEOREM VIII

The presently available material suggests the implication

$$\mathcal{C}(L, H, \theta) \implies \theta = 0,$$

where $\mathcal{C}$ denotes a geometric compatibility condition, L denotes the relevant lattice structure, and H denotes the relevant phase object.

Status: candidate theorem; proof not established by the present definitions.

To convert the candidate statement into a rigorous theorem, the following objects must be defined explicitly:

- (1) the mathematical structure denoted by L ;
- (2) the ambient space containing L ;
- (3) the domain and codomain of the parameter θ ;
- (4) the mathematical object denoted by H ;
- (5) the precise compatibility relation $\mathcal{C}$;
- (6) the admissible class of configurations;
- (7) the precise condition that excludes $\theta \neq 0$;
- (8) the logical implication from the compatibility condition to $\theta = 0$.

Without these definitions, the assertion

$$\theta = 0$$

cannot be promoted from candidate statement to established theorem.

23.12. *Provisional Classification of Extracted Mathematical Statements.* The mathematical nuclei currently identified in the material available for this formalization are classified as follows.

F–A: Harmonic force correction

$$F_F = F_N(1 + \sin \Theta).$$

Status: formally defined candidate relation.

F–B: Harmonic period correction

$$T_F = T_K(1 + \cos \Theta).$$

Status: formally defined candidate relation.

F–C: Observable scale identity

$$s = \frac{Q_R}{Q_M}, \quad Q_R = sQ_M.$$

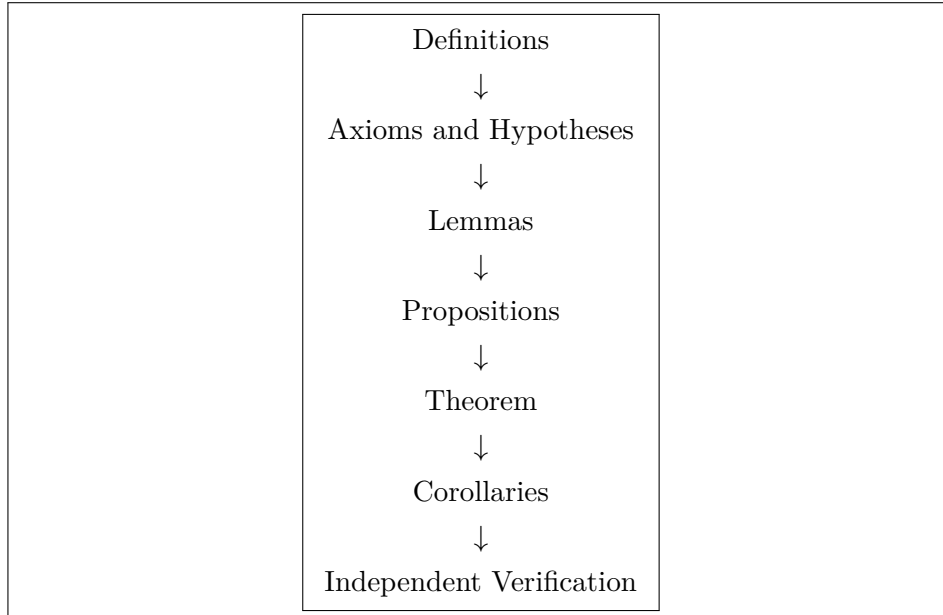
Status: elementary mathematical identity.

F–VIII: Strong CP angular constraint

$$\theta = 0.$$

Status: candidate Faure theorem; derivation not established from the presently formalized definitions.

23.13. *Criterion for Promotion to Established Theorem.* A proposed Faure statement shall be promoted to theorem status only after the following logical chain has been made explicit:



In particular, the following distinctions shall be maintained:

- a definition introduces a mathematical object;
- an identity follows directly from definitions or elementary algebra;

- a lemma establishes an intermediate mathematical result;
- a proposition establishes a nontrivial consequence under stated hypotheses;
- a theorem requires a complete proof from explicitly stated assumptions;
- a corollary follows from an established theorem;
- a conjecture is a mathematically meaningful assertion for which a proof has not been established;
- an empirical claim requires observational or experimental verification.

23.14. *Provisional Theorem Count.* The source material refers to a larger collection of “Faure Theorems”. However, the existence of a label or enumeration in a source is not sufficient to establish that each numbered item constitutes a mathematically distinct theorem.

On the basis of the mathematical content formally extracted in the present section, the following count is justified:

$N_{\text{formal nuclei}} = 3$
$N_{\text{candidate statements}} = 1$
$N_{\text{established Faure theorems}} = 0$
within the mathematical material formalized in this section.

This count is deliberately conservative. It shall be increased only when additional source material supplies a mathematically distinct statement together with sufficient definitions and hypotheses to justify its formal classification.

23.15. *Protocol for Subsequent Source Extraction.* For every subsequent PDF, Lean source, manuscript, or publicly accessible web source, the mathematical extraction shall follow the same protocol.

- (1) Identify the original mathematical claim.
- (2) Identify every variable, parameter, set, space, function, operator, relation, and domain occurring in the claim.
- (3) Introduce precise definitions for all previously undefined objects.
- (4) Separate assumptions from conclusions.
- (5) Check algebraic consistency.
- (6) Check dimensional consistency whenever physical quantities occur.
- (7) Derive elementary identities and lemmas first.
- (8) Derive propositions and corollaries only from explicitly stated hypotheses.
- (9) Search for counterexamples and limiting cases.
- (10) Determine whether the result is mathematically distinct from the already formalized corpus.

- (11) If a genuinely new mathematical result is identified, label it explicitly as

nuovo teorema Faure

only after its mathematical content warrants that classification.

- (12) Record separately the mathematical status and the empirical status of the result.

23.16. *Final Status of the Present Formalization.*

Final formal status.

The present section does not assert that the proposed Faure relations constitute established physical laws.

It provides a formal mathematical representation of the relations identified in the available source material and explicitly separates definitions, identities, derivations, candidate theorems, and empirical assertions.

No statement is promoted to established theorem status solely on the basis of its appearance in a source document. A statement may be classified as an established theorem only when its mathematical objects, hypotheses, domain of validity, and complete derivation have been explicitly specified and proved.

Accordingly, the present formalization distinguishes between
 mathematical consequence and empirical validation.

A mathematically valid consequence of a proposed definition does not, by itself, establish that the corresponding definition describes a physical law.

Therefore, the results established in this section should be understood as formal mathematical results concerning the proposed Faure framework, unless an independent empirical or theoretical justification is explicitly provided.

24. The Faure Universal Tuple

This section introduces the mathematical objects underlying the Faure Deterministic Framework. The purpose is to provide an explicit mathematical specification of the objects that are subsequently used in the structural results and in their formal verification.

All mathematical statements in this section are interpreted within ordinary classical mathematics. Whenever a statement is intended to correspond to a formally verified Lean construction, the mathematical statement is given independently of the implementation language.

24.1. *Universal Components.*

Definition 24.1 (Universal component). A *universal component* is an ordered pair

$$C = (i, D),$$

where

$$i \in \mathbb{N}$$

is an identifier and D is an admissible datum.

We write

$$\text{id}(C) := i$$

for the identifier of C .

Let $\mathbf{Comp}$ denote the class of admissible universal components. Thus

$$C \in \mathbf{Comp}$$

means that C is a universal component in the sense of the preceding definition.

No algebraic, topological, metric, or order-theoretic structure on $\mathbf{Comp}$ is assumed unless explicitly introduced below.

24.2. *Universal Relations.*

Definition 24.2 (Universal relation). Let

$$C_{\text{src}}, C_{\text{dst}} \in \mathbf{Comp}.$$

A *universal relation* is a quadruple

$$R = (C_{\text{src}}, C_{\text{dst}}, t, I),$$

where

$$t \in \mathbb{N}$$

is a relation tag and I is an admissible auxiliary datum.

We write

$$\text{src}(R) := C_{\text{src}}, \quad \text{dst}(R) := C_{\text{dst}}, \quad \text{tag}(R) := t.$$

Let $\mathbf{Rel}$ denote the class of admissible universal relations.

The definition does not impose any additional property on a relation. In particular, no assertion of symmetry, transitivity, injectivity, surjectivity, composability, or invertibility is implicit in the term “universal relation.”

24.3. The Universal Tuple.

Definition 24.3 (Faure universal tuple). A *Faure universal tuple* is a quintuple

$$U = (C_0, C, R, \lambda, P),$$

consisting of the following data:

- a base component

$$C_0 \in \mathbf{Comp};$$

- a component map

$$C : \mathbb{N} \longrightarrow \mathbf{Comp};$$

- a relation map

$$R : \mathbb{N} \longrightarrow \mathbf{Rel};$$

- a label

$$\lambda \in \mathbb{N};$$

- a payload map

$$P : \mathbb{N} \longrightarrow \mathbf{Pay},$$

where $\mathbf{Pay}$ denotes a specified class of admissible payloads.

For a fixed universal tuple U , we write

$$U.\text{comp}(i) := C(i)$$

and

$$U.\text{rel}(i) := R(i).$$

Hence

$$U.\text{comp}(i) \in \mathbf{Comp}$$

and

$$U.\text{rel}(i) \in \mathbf{Rel}$$

for every

$$i \in \mathbb{N}.$$

The base component C_0 is part of the tuple data and is not assumed, in general, to coincide with

$$U.\text{comp}(0).$$

Likewise, no relation between the component map, the relation map, the label, and the payload map is assumed unless such a relation is stated explicitly.

24.4. *Coherence, Stability, and Closure Structures.* The universal tuple provides the underlying data. Additional properties are introduced through separate structures.

Definition 24.4 (Universal coherence structure). Let U be a Faure universal tuple. A *coherence structure* on U is a family

$$\rho : \mathbb{N} \longrightarrow \mathbf{Coh},$$

where $\mathbf{Coh}$ is a specified class of admissible coherence data.

Definition 24.5 (Universal stability structure). Let U be a Faure universal tuple. A *stability structure* on U is a family

$$\sigma : \mathbb{N} \longrightarrow \mathbf{Stab},$$

where $\mathbf{Stab}$ is a specified class of admissible stability data.

Definition 24.6 (Universal closure structure). Let U be a Faure universal tuple. A *closure structure* on U is a pair

$$(\mathcal{C}, \mathcal{L}),$$

where $\mathcal{C}$ is a specified closure operation and $\mathcal{L}$ is a specified linking operation on the relevant class of admissible objects.

These structures are intentionally separated from the definition of the universal tuple itself. Consequently, the existence of a universal tuple alone does not imply coherence, stability, closure, or any other structural property.

Such properties may be imposed later as hypotheses and may then be used in the statements of propositions and theorems.

24.5. *The Iterative Component Map.* We now specify the iteration used in the present formalization.

Definition 24.7 (Iterated universal component). Let U be a Faure universal tuple and let

$$C \in \mathbf{Comp}.$$

Define

$$\text{Iter}_U : \mathbb{N} \times \mathbf{Comp} \longrightarrow \mathbf{Comp}$$

recursively by

$$\text{Iter}_U(0, C) := C$$

and

$$\text{Iter}_U(n + 1, C) := U.\text{comp}(\text{id}(C)).$$

The recursive equation is deliberately stated exactly as above. In particular, the definition does not iterate the newly obtained component. Instead, every positive iterate is determined by the identifier of the original component C .

This distinction is relevant for the mathematical interpretation of the construction.

PROPOSITION 24.8 (Explicit form of the iteration). *Let U be a Faure universal tuple and let*

$$C \in \text{Comp}.$$

Then

$$\text{Iter}_U(0, C) = C,$$

and, for every

$$n \in \mathbb{N}, \quad n \geq 1,$$

one has

$$\text{Iter}_U(n, C) = U.\text{comp}(\text{id}(C)).$$

Proof. The case $n = 0$ is the defining equation

$$\text{Iter}_U(0, C) = C.$$

Let $n \geq 1$. Since $n \geq 1$, there exists

$$m \in \mathbb{N}$$

such that

$$n = m + 1.$$

By the recursive definition,

$$\text{Iter}_U(n, C) = \text{Iter}_U(m + 1, C) = U.\text{comp}(\text{id}(C)).$$

The right-hand side is independent of m . Therefore the stated formula holds for every $n \geq 1$. $\square$

24.6. *Image Property of Positive Iterates.* The preceding proposition immediately yields the following structural result.

THEOREM 24.9 (Image property of positive iterates). *Let U be a Faure universal tuple. For every*

$$C \in \text{Comp}$$

and every

$$n \in \mathbb{N}, \quad n \geq 1,$$

one has

$$\text{Iter}_U(n, C) \in \text{im}(U.\text{comp}).$$

Proof. Let

$$C \in \text{Comp}$$

and let

$$n \geq 1.$$

By Proposition 24.8,

$$\text{Iter}_U(n, C) = U.\text{comp}(\text{id}(C)).$$

By definition of a universal component,

$$\text{id}(C) \in \mathbb{N}.$$

Since

$$U.\text{comp} : \mathbb{N} \longrightarrow \mathbf{Comp},$$

the element

$$U.\text{comp}(\text{id}(C))$$

belongs to the image of $U.\text{comp}$. Hence

$$\text{Iter}_U(n, C) \in \text{im}(U.\text{comp}).$$

□

Remark 24.10. The restriction

$$n \geq 1$$

cannot be omitted from the theorem as stated.

Indeed,

$$\text{Iter}_U(0, C) = C,$$

whereas the definition of a universal tuple imposes no condition requiring an arbitrary component

$$C \in \mathbf{Comp}$$

to belong to

$$\text{im}(U.\text{comp}).$$

Thus the image conclusion for $n = 0$ would require an additional hypothesis.

24.7. Positive-Iterate Stabilization. The definition of the iteration gives a stronger statement than mere membership in the image: all positive iterates coincide.

THEOREM 24.11 (Stabilization after the first iteration). *Let U be a Faure universal tuple and let*

$$C \in \mathbf{Comp}.$$

Then, for all

$$m, n \in \mathbb{N}$$

satisfying

$$m \geq 1, \quad n \geq 1,$$

one has

$$\text{Iter}_U(m, C) = \text{Iter}_U(n, C).$$

Proof. By Proposition 24.8,

$$\text{Iter}_U(m, C) = U.\text{comp}(\text{id}(C))$$

and

$$\text{Iter}_U(n, C) = U.\text{comp}(\text{id}(C)).$$

Therefore

$$\text{Iter}_U(m, C) = \text{Iter}_U(n, C).$$

□

COROLLARY 24.12 (One-step stabilization). *Let U be a Faure universal tuple and let*

$$C \in \text{Comp}.$$

Then, for every

$$n \geq 1,$$

one has

$$\text{Iter}_U(n+1, C) = \text{Iter}_U(n, C).$$

Proof. Both indices n and $n+1$ are at least 1. The result therefore follows immediately from Theorem 24.11. □

Remark 24.13. The preceding theorem establishes stabilization of the particular iteration operator defined in this section. It does not assert that an arbitrary dynamical system associated with a Faure universal tuple stabilizes after one step, nor does it imply a corresponding statement for any other evolution operator.

Any stronger dynamical interpretation requires a separately defined state space, evolution map, and compatibility theorem.

24.8. Formal Verification Status. The definitions and elementary results above are mathematical statements independently of their implementation in a proof assistant.

Where corresponding Lean declarations and proofs exist, they provide a machine-checked formalization of the stated mathematical content. The existence of such a formal proof does not alter the mathematical meaning of the theorem, but supplies an independent verification of the formalized derivation.

In particular, the following logical distinction is maintained:

mathematical statement	≠	Lean implementation	≠	empirical assertion.
------------------------	---	---------------------	---	----------------------

The present section establishes only the consequences of the definitions and hypotheses explicitly stated here. No physical interpretation is inferred from the universal-tuple formalism alone.

25. Deterministic Structures

The preceding sections introduced the mathematical objects and operator relations that are explicitly supported by the present formalization. This section isolates the deterministic consequences that follow from those assumptions.

No physical interpretation is attached to the constructions below. The terms “state”, “transition”, “invariant”, and “stabilization” are used exclusively in their mathematical senses.

25.1. Deterministic State Systems.

Definition 25.1 (Deterministic state system). Let X be a nonempty set. A *deterministic state system* on X is a pair

$$\mathcal{D} = (X, \Psi),$$

where

$$\Psi : X \longrightarrow X$$

is a total function. The map Ψ is called the *transition map*.

For every $x \in X$, determinism means that the successor state is uniquely specified by $\Psi(x)$.

For $n \in \mathbb{N}$, the iterates of Ψ are defined recursively by

$$\Psi^0 := \text{id}_X$$

and

$$\Psi^{n+1} := \Psi \circ \Psi^n.$$

Thus,

$$\Psi^0(x) = x, \quad \Psi^1(x) = \Psi(x), \quad \Psi^2(x) = \Psi(\Psi(x)),$$

and, more generally,

$$\Psi^n(x) = \underbrace{\Psi \circ \cdots \circ \Psi}_{n \text{ times}}(x)$$

for every $n \geq 1$.

LEMMA 25.2 (Composition law for iterates). *Let $\Psi : X \rightarrow X$ be a map. Then, for all $m, n \in \mathbb{N}$,*

$$\Psi^{m+n} = \Psi^m \circ \Psi^n.$$

Proof. We proceed by induction on m .

For $m = 0$,

$$\Psi^{0+n} = \Psi^n = \text{id}_X \circ \Psi^n = \Psi^0 \circ \Psi^n.$$

Assume that

$$\Psi^{m+n} = \Psi^m \circ \Psi^n.$$

By the recursive definition,

$$\Psi^{(m+1)+n} = \Psi^{m+n+1} = \Psi \circ \Psi^{m+n}.$$

Using the induction hypothesis,

$$\Psi^{(m+1)+n} = \Psi \circ \Psi^m \circ \Psi^n.$$

Since

$$\Psi^{m+1} = \Psi \circ \Psi^m,$$

we obtain

$$\Psi^{(m+1)+n} = \Psi^{m+1} \circ \Psi^n.$$

Therefore the identity holds for every $m, n \in \mathbb{N}$. $\square$

25.2. Fixed Points and Invariant Sets.

Definition 25.3 (Fixed point). Let $\Psi : X \rightarrow X$. A point $x \in X$ is called a *fixed point* of Ψ if

$$\Psi(x) = x.$$

The set of fixed points is

$$\text{Fix}(\Psi) := \{x \in X : \Psi(x) = x\}.$$

Definition 25.4 (Forward-invariant subset). Let $\Psi : X \rightarrow X$ and let $A \subseteq X$. The set A is called *forward-invariant* under Ψ if

$$\Psi(A) \subseteq A.$$

LEMMA 25.5 (The fixed-point set is invariant). *Let $\Psi : X \rightarrow X$. Then*

$$\Psi(\text{Fix}(\Psi)) \subseteq \text{Fix}(\Psi).$$

Proof. Let $x \in \text{Fix}(\Psi)$. Then

$$\Psi(x) = x.$$

Consequently,

$$\Psi(\Psi(x)) = \Psi(x) = x = \Psi(x).$$

Thus $\Psi(x)$ is again a fixed point of Ψ , and hence

$$\Psi(\text{Fix}(\Psi)) \subseteq \text{Fix}(\Psi).$$

$\square$

PROPOSITION 25.6 (Persistence of a fixed point). *Let $\Psi : X \rightarrow X$ and let $x \in X$. If*

$$\Psi(x) = x,$$

then

$$\Psi^n(x) = x$$

for every $n \in \mathbb{N}$.

Proof. We proceed by induction on n .

For $n = 0$,

$$\Psi^0(x) = x.$$

Suppose that

$$\Psi^n(x) = x.$$

Then

$$\Psi^{n+1}(x) = \Psi(\Psi^n(x)) = \Psi(x) = x.$$

Therefore

$$\Psi^n(x) = x$$

for every $n \in \mathbb{N}$. □

25.3. Finite-Time Stabilization. The following definition distinguishes finite-time stabilization from mere invariance.

Definition 25.7 (Finite-time stabilization). Let $\Psi : X \rightarrow X$, let $x \in X$, and let $N \in \mathbb{N}$. We say that x *stabilizes at time N* if

$$\Psi^{N+1}(x) = \Psi^N(x).$$

Equivalently, the state

$$y := \Psi^N(x)$$

is a fixed point of Ψ .

LEMMA 25.8 (Pointwise persistence of stabilization). *Let $\Psi : X \rightarrow X$, let $x \in X$, and let $N \in \mathbb{N}$. If*

$$\Psi^{N+1}(x) = \Psi^N(x),$$

then

$$\Psi^n(x) = \Psi^N(x)$$

for every $n \geq N$.

Proof. Set

$$y := \Psi^N(x).$$

The hypothesis gives

$$\Psi(y) = \Psi(\Psi^N(x)) = \Psi^{N+1}(x) = \Psi^N(x) = y.$$

Thus y is fixed by Ψ . By Proposition 25.6,

$$\Psi^k(y) = y$$

for every $k \in \mathbb{N}$.

Let $n \geq N$. Then there exists $k \in \mathbb{N}$ such that

$$n = N + k.$$

Using the composition law for iterates,

$$\Psi^n(x) = \Psi^{N+k}(x) = \Psi^k(\Psi^N(x)) = \Psi^k(y) = y = \Psi^N(x).$$

Hence

$$\Psi^n(x) = \Psi^N(x)$$

for every $n \geq N$. □

25.4. Global Stabilization. A pointwise stabilization statement concerns one initial state. A global stabilization identity is stronger because it asserts equality of maps.

Definition 25.9 (Globally stabilized iterate). Let $\Psi : X \rightarrow X$ and let $N \in \mathbb{N}$. We say that Ψ is *globally stabilized at step N* if

$$\Psi^{N+1} = \Psi^N$$

as functions from X to X .

THEOREM 25.10 (Persistence of a globally stabilized iterate). *Let $\Psi : X \rightarrow X$ and suppose that, for some $N \in \mathbb{N}$,*

$$\Psi^{N+1} = \Psi^N.$$

Then

$$\Psi^n = \Psi^N$$

for every $n \geq N$.

Proof. The hypothesis is

$$\Psi^{N+1} = \Psi^N.$$

Composing both sides on the left with Ψ gives

$$\Psi^{N+2} = \Psi^{N+1}.$$

Combining this equality with the hypothesis yields

$$\Psi^{N+2} = \Psi^N.$$

Repeating the same argument inductively, suppose that

$$\Psi^{N+k} = \Psi^N$$

for some $k \in \mathbb{N}$. Composing on the left with Ψ gives

$$\Psi^{N+k+1} = \Psi^{N+k} = \Psi^N.$$

Therefore

$$\Psi^{N+k} = \Psi^N$$

for every $k \in \mathbb{N}$.

Since every $n \geq N$ can be written as

$$n = N + k$$

for some $k \in \mathbb{N}$, it follows that

$$\Psi^n = \Psi^N$$

for every $n \geq N$. □

25.5. Idempotence.

Definition 25.11 (Idempotent map). Let X be a set and let $P : X \rightarrow X$. The map P is called *idempotent* if

$$P \circ P = P.$$

PROPOSITION 25.12 (A globally stabilized iterate is idempotent). *Let $\Psi : X \rightarrow X$ and suppose that*

$$\Psi^{N+1} = \Psi^N$$

for some $N \in \mathbb{N}$. Then Ψ^N is idempotent:

$$\Psi^N \circ \Psi^N = \Psi^N.$$

Proof. By Lemma 39.9,

$$\Psi^N \circ \Psi^N = \Psi^{2N}.$$

By Theorem 25.10,

$$\Psi^{2N} = \Psi^N,$$

because $2N \geq N$. Hence

$$\Psi^N \circ \Psi^N = \Psi^N.$$

□

PROPOSITION 25.13 (Image of an idempotent map). *Let $P : X \rightarrow X$ be idempotent. Then*

$$\text{im}(P) = \text{Fix}(P).$$

Proof. First let

$$y \in \text{im}(P).$$

Then there exists $x \in X$ such that

$$y = P(x).$$

By idempotence,

$$P(y) = P(P(x)) = P(x) = y.$$

Thus

$$y \in \text{Fix}(P).$$

Therefore

$$\text{im}(P) \subseteq \text{Fix}(P).$$

Conversely, let

$$y \in \text{Fix}(P).$$

Then

$$P(y) = y.$$

Hence y is the image under P of the element y itself, and so

$$y \in \text{im}(P).$$

Thus

$$\text{Fix}(P) \subseteq \text{im}(P).$$

Consequently,

$$\text{im}(P) = \text{Fix}(P).$$

□

25.6. Four-Step Stabilization. The frequently used four-step statement is an immediate specialization of the preceding theorem and does not require any additional structure.

COROLLARY 25.14 (Four-step stabilization). *Let $\Psi : X \rightarrow X$ satisfy*

$$\Psi^5 = \Psi^4.$$

Then

$$\Psi^n = \Psi^4$$

for every $n \geq 4$. Moreover,

$$\Psi^4 \circ \Psi^4 = \Psi^4,$$

and

$$\text{im}(\Psi^4) \subseteq \text{Fix}(\Psi).$$

Proof. The equality

$$\Psi^5 = \Psi^4$$

is the hypothesis of Theorem 25.10 with $N = 4$. Therefore

$$\Psi^n = \Psi^4$$

for every $n \geq 4$.

By Proposition 25.12,

$$\Psi^4 \circ \Psi^4 = \Psi^4.$$

Let

$$y \in \text{im}(\Psi^4).$$

Then there exists $x \in X$ such that

$$y = \Psi^4(x).$$

Hence

$$\Psi(y) = \Psi^5(x) = \Psi^4(x) = y.$$

Thus

$$y \in \text{Fix}(\Psi),$$

which proves

$$\text{im}(\Psi^4) \subseteq \text{Fix}(\Psi).$$

□

25.7. The Stabilized State Map.

Definition 25.15 (Four-step stabilized state). Let $\Psi : X \rightarrow X$ satisfy

$$\Psi^5 = \Psi^4.$$

For each $x \in X$, define

$$x_{\text{st}} := \Psi^4(x).$$

PROPOSITION 25.16 (Exact four-step arrival statement). *Let $\Psi : X \rightarrow X$ satisfy*

$$\Psi^5 = \Psi^4.$$

Then, for every $x \in X$,

$$\Psi(x_{\text{st}}) = x_{\text{st}},$$

where

$$x_{\text{st}} = \Psi^4(x).$$

Furthermore,

$$\Psi^n(x) = x_{\text{st}}$$

for every $n \geq 4$.

Proof. By definition,

$$x_{\text{st}} = \Psi^4(x).$$

Therefore

$$\Psi(x_{\text{st}}) = \Psi(\Psi^4(x)) = \Psi^5(x).$$

The hypothesis gives

$$\Psi^5(x) = \Psi^4(x),$$

and consequently

$$\Psi(x_{\text{st}}) = x_{\text{st}}.$$

For every $n \geq 4$, Corollary 25.14 gives

$$\Psi^n = \Psi^4.$$

Hence

$$\Psi^n(x) = \Psi^4(x) = x_{\text{st}}.$$

□

Remark 25.17. The preceding proposition is the precise mathematical content of the statement that a state “reaches a fixed state after at most four iterations”, provided that the hypothesis

$$\Psi^5 = \Psi^4$$

has actually been established.

No decomposition of X into physical “components” is implied by this result. The stabilized state is simply the image

$$\Psi^4(x).$$

25.8. *Linear Systems.* The following results require additional algebraic hypotheses.

Definition 25.18 (Linear deterministic system). Let V be a vector space over a field $\mathbb{F}$. A *linear deterministic system* is a pair

$$(V, \Psi),$$

where

$$\Psi : V \longrightarrow V$$

is linear.

THEOREM 25.19 (Stable decomposition). *Let V be a vector space over a field $\mathbb{F}$ and let*

$$\Psi : V \longrightarrow V$$

be linear. Suppose that, for some $N \geq 1$,

$$\Psi^{N+1} = \Psi^N.$$

Then

$$V = \ker(\Psi^N) \oplus \ker(I - \Psi).$$

Proof. We first establish that

$$\operatorname{im}(\Psi^N) \subseteq \ker(I - \Psi).$$

Let

$$y \in \operatorname{im}(\Psi^N).$$

Then there exists $v \in V$ such that

$$y = \Psi^N(v).$$

Consequently,

$$\Psi(y) = \Psi^{N+1}(v) = \Psi^N(v) = y.$$

Therefore

$$(I - \Psi)y = 0,$$

and hence

$$y \in \ker(I - \Psi).$$

We next prove

$$\ker(\Psi^N) \cap \ker(I - \Psi) = \{0\}.$$

Let

$$v \in \ker(\Psi^N) \cap \ker(I - \Psi).$$

Then

$$\Psi^N(v) = 0$$

and

$$\Psi(v) = v.$$

By repeated application of $\Psi(v) = v$,

$$\Psi^N(v) = v.$$

Combining the two equalities gives

$$v = 0.$$

Thus the intersection is trivial.

It remains to prove that the sum spans V . Let $v \in V$ and define

$$v_{\text{st}} := \Psi^N(v)$$

and

$$v_{\text{tr}} := v - \Psi^N(v).$$

Then

$$v = v_{\text{tr}} + v_{\text{st}}.$$

We have

$$\Psi^N(v_{\text{st}}) = \Psi^{2N}(v).$$

Since

$$\Psi^{N+1} = \Psi^N,$$

Theorem 25.10 gives

$$\Psi^{2N} = \Psi^N.$$

Hence

$$\Psi^N(v_{\text{st}}) = \Psi^N(v) = v_{\text{st}}.$$

More directly, since

$$\Psi(v_{\text{st}}) = \Psi^{N+1}(v) = \Psi^N(v) = v_{\text{st}},$$

we obtain

$$v_{\text{st}} \in \ker(I - \Psi).$$

Moreover,

$$\Psi^N(v_{\text{tr}}) = \Psi^N(v) - \Psi^{2N}(v) = 0.$$

Thus

$$v_{\text{tr}} \in \ker(\Psi^N).$$

Therefore

$$V = \ker(\Psi^N) + \ker(I - \Psi).$$

Since the intersection of these two subspaces is trivial, the sum is direct:

$$V = \ker(\Psi^N) \oplus \ker(I - \Psi).$$

□

COROLLARY 25.20 (Four-step linear decomposition). *Let V be a vector space over a field $\mathbb{F}$ and let*

$$\Psi : V \longrightarrow V$$

be linear. Suppose that

$$\Psi^5 = \Psi^4.$$

Then

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi).$$

In particular, every $v \in V$ admits a unique decomposition

$$v = v_{\text{tr}} + v_{\text{st}},$$

where

$$v_{\text{tr}} \in \ker(\Psi^4)$$

and

$$v_{\text{st}} \in \ker(I - \Psi).$$

Moreover,

$$\Psi^4(v) = v_{\text{st}}$$

and

$$\Psi^n(v) = v_{\text{st}} \quad (n \geq 4).$$

Proof. Apply Theorem 25.19 with $N = 4$. Thus

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi).$$

The direct-sum property gives uniqueness of the decomposition.

Let

$$v = v_{\text{tr}} + v_{\text{st}}$$

be this decomposition. Since

$$v_{\text{tr}} \in \ker(\Psi^4),$$

we have

$$\Psi^4(v_{\text{tr}}) = 0.$$

Since

$$v_{\text{st}} \in \ker(I - \Psi),$$

we have

$$\Psi(v_{\text{st}}) = v_{\text{st}},$$

and therefore

$$\Psi^4(v_{\text{st}}) = v_{\text{st}}.$$

Consequently,

$$\Psi^4(v) = \Psi^4(v_{\text{tr}}) + \Psi^4(v_{\text{st}}) = v_{\text{st}}.$$

Finally, Corollary 25.14 gives

$$\Psi^n(v) = \Psi^4(v) = v_{\text{st}}$$

for every $n \geq 4$. □

25.9. *Scope of the Deterministic Results.* The preceding results have a strictly conditional character.

The identity

$$\Psi^5 = \Psi^4$$

implies four-step stabilization only after that identity has been established for the particular map Ψ under consideration.

Likewise, the decomposition

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi)$$

requires that V be a vector space, that Ψ be linear, and that the identity

$$\Psi^5 = \Psi^4$$

hold. It is not a consequence of the existence of an arbitrary map

$$\Psi : X \longrightarrow X.$$

No physical interpretation follows from these mathematical statements alone. A physical application requires an explicitly specified map from the mathematical state space to the relevant physical observables, together with independently stated assumptions and empirical tests.

26. The Faure Spectral Engine

This section introduces the mathematical structures used to define the Faure spectral engine. The construction is deliberately stated at the level of sets and total maps. No physical interpretation is attached to the objects introduced below.

The purpose of the section is to specify precisely the data entering the spectral construction and to distinguish the underlying spectrum from the transition map induced by the engine.

26.1. Spectral Indices and Values.

Definition 26.1 (Spectral index). Let $\mathbb{N}$ denote the set of nonnegative integers. A *spectral index* is an ordered pair

$$I = (i, t) \in \mathbb{N} \times \mathbb{N}.$$

We write

$$\text{id}(I) := i, \quad \text{tag}(I) := t.$$

The set of spectral indices is denoted by

$$\text{SIndex} := \mathbb{N} \times \mathbb{N}.$$

Definition 26.2 (Spectral datum). Let $\mathcal{D}$ be a fixed nonempty set. An element

$$D \in \mathcal{D}$$

is called an *admissible spectral datum*.

Definition 26.3 (Spectral value). Let $\mathcal{D}$ be an admissible datum set. A *spectral value* is an ordered pair

$$V = (I, D) \in \text{SIndex} \times \mathcal{D}.$$

The set of spectral values is therefore

$$\text{SValue} := \text{SIndex} \times \mathcal{D}.$$

We write

$$\text{idx}(V) := I, \quad \text{data}(V) := D.$$

26.2. Spectral Lines.

Definition 26.4 (Auxiliary spectral datum). Let $\mathcal{A}$ be a fixed nonempty set. An element

$$A \in \mathcal{A}$$

is called an *admissible auxiliary datum*.

Definition 26.5 (Spectral line). Let $\mathcal{D}$ and $\mathcal{A}$ be fixed admissible data sets. A *spectral line* is a triple

$$L = (p, V, A) \in \mathbb{N} \times \text{SValue} \times \mathcal{A}.$$

The set of spectral lines is denoted by

$$\text{SLine} := \mathbb{N} \times \text{SValue} \times \mathcal{A}.$$

We write

$$\text{pos}(L) := p, \quad \text{val}(L) := V, \quad \text{info}(L) := A.$$

26.3. Spectra.

Definition 26.6 (Payload datum). Let $\mathcal{P}$ be a fixed nonempty set. An element

$$P \in \mathcal{P}$$

is called an *admissible payload datum*.

Definition 26.7 (Channel datum). Let $\mathcal{C}$ be a fixed nonempty set. An element

$$C \in \mathcal{C}$$

is called an *admissible channel datum*.

Definition 26.8 (Faure spectrum). A *Faure spectrum* is a triple

$$S = (\ell, \lambda, \pi),$$

where

$$\ell : \mathbb{N} \longrightarrow \mathbf{SLine},$$

$$\lambda \in \mathbb{N},$$

and

$$\pi : \mathbb{N} \longrightarrow \mathcal{P}.$$

The map ℓ is called the *spectral-line map*, λ the *spectrum label*, and π the *payload map*.

For a Faure spectrum S , we write

$$S.\text{line}(i) := \ell(i), \quad S.\text{label} := \lambda, \quad S.\text{payload}(i) := \pi(i).$$

26.4. Spectral Operators.

Definition 26.9 (Spectral operator). A *spectral operator* is a triple

$$O = (\phi_I, \phi_V, t),$$

where

$$\phi_I : \mathbf{SIndex} \longrightarrow \mathbf{SIndex},$$

$$\phi_V : \mathbf{SValue} \longrightarrow \mathbf{SValue},$$

and

$$t \in \mathbb{N}.$$

The maps ϕ_I and ϕ_V are called respectively the *index action* and the *value action*, while t is the operator tag.

26.5. Faure Spectral Engine.

Definition 26.10 (Faure spectral engine). Let

$$\mathcal{C}$$

be a fixed nonempty channel set. A *Faure spectral engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where

$$S$$

is a Faure spectrum,

$$O$$

is a spectral operator,

$$m \in \mathbb{N},$$

and

$$\Gamma : \mathbb{N} \longrightarrow \mathcal{C}$$

is a total channel map.

We write

$$E.\text{baseSpec} := S, \quad E.\text{op} := O, \quad E.\text{mode} := m, \quad E.\text{channel} := \Gamma.$$

26.6. The Spectral Transition Map. The data defining a spectral engine do not, by themselves, determine an iteration on spectral lines. To obtain a genuine dynamical system on **SLine**, an explicit total transition map must be specified.

Definition 26.11 (Spectral transition map). Let E be a Faure spectral engine. A *spectral transition map compatible with E* is a total function

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}.$$

The pair

$$(\mathbf{SLine}, T_E)$$

is called the *spectral dynamical system induced by E* .

Definition 26.12 (Spectral iterates). Let E be a Faure spectral engine and let

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

be a compatible spectral transition map. Define its iterates recursively by

$$T_E^0 := \text{id}_{\mathbf{SLine}}$$

and

$$T_E^{n+1} := T_E \circ T_E^n.$$

For $L \in \mathbf{SLine}$ and $n \in \mathbb{N}$, the n -step spectral state generated from L is

$$\text{Iter}_E(n, L) := T_E^n(L).$$

LEMMA 26.13 (Composition law for spectral iterates). *Let E be a Faure spectral engine and let*

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

be a compatible spectral transition map. Then, for all

$$m, n \in \mathbb{N},$$

one has

$$T_E^{m+n} = T_E^m \circ T_E^n.$$

Proof. The proof is the standard induction on m .

For $m = 0$,

$$T_E^{0+n} = T_E^n = \text{id}_{\mathbf{SLine}} \circ T_E^n = T_E^0 \circ T_E^n.$$

Assume that

$$T_E^{m+n} = T_E^m \circ T_E^n.$$

Then, by the recursive definition of the iterates,

$$T_E^{(m+1)+n} = T_E \circ T_E^{m+n}.$$

Using the induction hypothesis,

$$T_E^{(m+1)+n} = T_E \circ T_E^m \circ T_E^n.$$

Since

$$T_E^{m+1} = T_E \circ T_E^m,$$

we obtain

$$T_E^{(m+1)+n} = T_E^{m+1} \circ T_E^n.$$

Thus the identity holds for every

$$m, n \in \mathbb{N}.$$

□

26.7. *Canonical Spectral Extraction.* The base spectrum determines a canonical map from spectral lines to spectral lines by retaining the position of the input line and extracting the corresponding line from the base spectrum.

Definition 26.14 (Canonical spectral extraction). Let E be a Faure spectral engine. Define

$$\Sigma_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

by

$$\Sigma_E(L) := E.\text{baseSpec}.\text{line}(\text{pos}(L)).$$

The map Σ_E is called the *canonical spectral extraction map* associated with E .

PROPOSITION 26.15 (Well-definedness of canonical extraction). *Let E be a Faure spectral engine. Then*

$$\Sigma_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

is a total function. Moreover,

$$\text{im}(\Sigma_E) \subseteq \text{im}(E.\text{baseSpec}.\text{line}).$$

Proof. Let

$$L \in \mathbf{SLine}.$$

By definition of $\mathbf{SLine}$,

$$\text{pos}(L) \in \mathbb{N}.$$

Since

$$E.\text{baseSpec}.\text{line} : \mathbb{N} \longrightarrow \mathbf{SLine}$$

is a total function, the element

$$E.\text{baseSpec}.\text{line}(\text{pos}(L))$$

is a well-defined element of $\mathbf{SLine}$. Hence Σ_E is a total map

$$\mathbf{SLine} \longrightarrow \mathbf{SLine}.$$

Now let

$$L' \in \text{im}(\Sigma_E).$$

Then there exists

$$L \in \mathbf{SLine}$$

such that

$$L' = \Sigma_E(L).$$

By definition,

$$L' = E.\text{baseSpec}.\text{line}(\text{pos}(L)).$$

Therefore

$$L' \in \text{im}(E.\text{baseSpec}.\text{line}).$$

Consequently,

$$\text{im}(\Sigma_E) \subseteq \text{im}(E.\text{baseSpec}.\text{line}).$$

□

26.8. *Iterated Canonical Extraction.*

Definition 26.16 (Iterated canonical spectral extraction). Let E be a Faure spectral engine. The n -fold canonical spectral extraction is defined by

$$\Sigma_E^0 := \text{id}_{\text{SLine}}$$

and

$$\Sigma_E^{n+1} := \Sigma_E \circ \Sigma_E^n.$$

For

$$L \in \text{SLine},$$

we write

$$\Sigma_E^n(L)$$

for the spectral line obtained after n successive applications of the canonical extraction map.

PROPOSITION 26.17 (Positive iterates lie in the base-spectrum image). *Let E be a Faure spectral engine. Then, for every*

$$L \in \text{SLine}$$

and every

$$n \geq 1,$$

one has

$$\Sigma_E^n(L) \in \text{im}(E.\text{baseSpec}.\text{line}).$$

Proof. We proceed by induction on n .

For $n = 1$,

$$\Sigma_E^1(L) = \Sigma_E(L),$$

and the assertion follows from Proposition 26.15.

Assume that, for some $n \geq 1$,

$$\Sigma_E^n(L) \in \text{im}(E.\text{baseSpec}.\text{line}).$$

Since

$$\Sigma_E^{n+1}(L) = \Sigma_E(\Sigma_E^n(L)),$$

Proposition 26.15 applies to the spectral line

$$\Sigma_E^n(L)$$

and gives

$$\Sigma_E^{n+1}(L) \in \text{im}(E.\text{baseSpec}.\text{line}).$$

Therefore the assertion holds for every

$$n \geq 1.$$

□

26.9. *Stabilization of the Spectral Engine.* Membership in the image of the base-spectrum map does not, by itself, imply that a spectral line is fixed by the extraction map. A genuine stabilization statement therefore requires an additional hypothesis.

Definition 26.18 (Spectral stabilization). Let E be a Faure spectral engine and let

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

be a compatible spectral transition map. We say that T_E is *globally stabilized at step N* if

$$T_E^{N+1} = T_E^N$$

as functions from $\mathbf{SLine}$ to $\mathbf{SLine}$.

THEOREM 26.19 (Persistence of spectral stabilization). *Let E be a Faure spectral engine and let*

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

be a compatible spectral transition map. Suppose that, for some

$$N \in \mathbb{N},$$

$$T_E^{N+1} = T_E^N.$$

Then

$$T_E^n = T_E^N$$

for every

$$n \geq N.$$

Proof. The proof is by induction on

$$k \in \mathbb{N}.$$

For $k = 0$,

$$T_E^{N+k} = T_E^N.$$

Assume that

$$T_E^{N+k} = T_E^N.$$

By the definition of iteration,

$$T_E^{N+k+1} = T_E \circ T_E^{N+k}.$$

Using the induction hypothesis,

$$T_E^{N+k+1} = T_E \circ T_E^N = T_E^{N+1}.$$

The stabilization hypothesis gives

$$T_E^{N+1} = T_E^N.$$

Hence

$$T_E^{N+k+1} = T_E^N.$$

Therefore

$$T_E^{N+k} = T_E^N$$

for every

$$k \in \mathbb{N}.$$

Since every

$$n \geq N$$

can be written as

$$n = N + k$$

for some $k \in \mathbb{N}$, we conclude that

$$T_E^n = T_E^N$$

for every

$$n \geq N.$$

□

COROLLARY 26.20 (Four-step spectral stabilization). *Let E be a Faure spectral engine and let*

$$T_E : \text{SLine} \longrightarrow \text{SLine}$$

be a compatible spectral transition map. Suppose that

$$T_E^5 = T_E^4.$$

Then

$$T_E^n = T_E^4$$

for every

$$n \geq 4.$$

Moreover,

$$T_E^4 \circ T_E^4 = T_E^4.$$

Proof. The first assertion follows immediately from Theorem 26.19 with

$$N = 4.$$

Thus

$$T_E^n = T_E^4$$

for every

$$n \geq 4.$$

For the second assertion, the composition law for iterates gives

$$T_E^4 \circ T_E^4 = T_E^8.$$

Since

$$8 \geq 4,$$

the first part yields

$$T_E^8 = T_E^4.$$

Therefore

$$T_E^4 \circ T_E^4 = T_E^4.$$

□

26.10. *Scope of the Spectral Construction.* The definitions above distinguish three logically different objects.

First, the base spectrum is the total map

$$E.\text{baseSpec}.\text{line} : \mathbb{N} \longrightarrow \mathbf{SLine}.$$

Second, the canonical spectral extraction map is the induced total map

$$\Sigma_E : \mathbf{SLine} \longrightarrow \mathbf{SLine},$$

defined by

$$\Sigma_E(L) = E.\text{baseSpec}.\text{line}(\text{pos}(L)).$$

Third, a general spectral transition map

$$T_E : \mathbf{SLine} \longrightarrow \mathbf{SLine}$$

must be specified separately if the engine is to be regarded as a dynamical system.

In particular, the existence of a Faure spectrum does not by itself imply the stabilization identity

$$T_E^5 = T_E^4.$$

That identity is an additional mathematical hypothesis and must either be derived from further assumptions or be verified independently.

Likewise, the inclusion

$$\text{im}(\Sigma_E) \subseteq \text{im}(E.\text{baseSpec}.\text{line})$$

does not imply

$$\Sigma_E^2 = \Sigma_E.$$

Idempotence requires the additional condition

$$\Sigma_E^2 = \Sigma_E.$$

Consequently, no claim of four-step stabilization, idempotence, or fixed-point structure is made here unless the corresponding identity is explicitly established.

No physical interpretation follows from these definitions and propositions alone. Any physical application requires an explicitly specified correspondence between the mathematical spectral structures and physical observables, together with independently stated assumptions and empirical tests.

27. Probability Structures

This section introduces the probabilistic structures used in the formalization. All probabilistic statements are formulated within the standard framework of measurable spaces and probability measures.

No probabilistic property is attributed to an index, operator, or auxiliary data structure unless the corresponding measure-theoretic hypothesis is explicitly stated.

Throughout this section,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

27.1. Probability Indices.

Definition 27.1 (Probability index). A *probability index* is an element of

$$\mathcal{I}_{\text{prob}} := \mathbb{N}^2.$$

Thus

$$\iota = (i, t), \quad i, t \in \mathbb{N}.$$

We write

$$\text{id}(\iota) := i, \quad \text{tag}(\iota) := t.$$

27.2. Probability Data.

Definition 27.2 (Probability data space). A *probability data space* is a pair

$$(\mathcal{D}, \mathcal{A}_{\mathcal{D}}),$$

where $\mathcal{D}$ is a set and $\mathcal{A}_{\mathcal{D}}$ is a σ -algebra on $\mathcal{D}$.

Definition 27.3 (Probability value). Let

$$(\mathcal{D}, \mathcal{A}_{\mathcal{D}})$$

be a probability data space. A *probability value* is a pair

$$v = (\iota, d),$$

where

$$\iota \in \mathcal{I}_{\text{prob}}$$

and

$$d \in \mathcal{D}.$$

Thus the underlying set of probability values is

$$\mathcal{V}_{\text{prob}} := \mathcal{I}_{\text{prob}} \times \mathcal{D}.$$

27.3. Probability Spaces.

Definition 27.4 (Measurable space). A *measurable space* is a pair

$$(\Omega, \mathcal{F}),$$

where Ω is a set and $\mathcal{F}$ is a σ -algebra on Ω .

Definition 27.5 (Probability measure). Let $(\Omega, \mathcal{F})$ be a measurable space. A function

$$\mathbb{P} : \mathcal{F} \longrightarrow [0, 1]$$

is a *probability measure* if

$$\mathbb{P}(\Omega) = 1$$

and, for every sequence

$$(A_n)_{n \in \mathbb{N}}$$

of pairwise disjoint sets in $\mathcal{F}$,

$$\mathbb{P}\left(\bigcup_{n=0}^{\infty} A_n\right) = \sum_{n=0}^{\infty} \mathbb{P}(A_n).$$

Definition 27.6 (Probability space). A *probability space* is a triple

$$(\Omega, \mathcal{F}, \mathbb{P}),$$

where $(\Omega, \mathcal{F})$ is a measurable space and $\mathbb{P}$ is a probability measure on it.

27.4. Indexed Probability Events.

Definition 27.7 (Indexed probability event). Let

$$(\Omega, \mathcal{F}, \mathbb{P})$$

be a probability space. An *indexed probability event* is a pair

$$A = (\iota, E),$$

where

$$\iota \in \mathcal{I}_{\text{prob}}$$

and

$$E \in \mathcal{F}.$$

The set of indexed probability events is

$$\mathcal{E}_{\text{prob}} := \mathcal{I}_{\text{prob}} \times \mathcal{F}.$$

Definition 27.8 (Underlying event). For

$$A = (\iota, E) \in \mathcal{E}_{\text{prob}},$$

the *underlying event* of A is

$$\underline{A} := E.$$

Definition 27.9 (Probability of an indexed event). For

$$A = (\iota, E) \in \mathcal{E}_{\text{prob}},$$

define

$$\mathbb{P}_{\text{ind}}(A) := \mathbb{P}(E).$$

Remark 27.10. The index ι does not alter the probability assigned to the underlying event. Any dependence of probabilities on indices must be introduced by an additional indexed family of measures or by another explicit construction.

27.5. Indexed Probability Families.

Definition 27.11 (Probability family). Let

$$(\Omega, \mathcal{F}, \mathbb{P})$$

be a probability space. A *probability family* is a function

$$\mathcal{E} : \mathbb{N} \longrightarrow \mathcal{E}_{\text{prob}}.$$

For each $n \in \mathbb{N}$, write

$$\mathcal{E}(n) = (\iota_n, E_n).$$

PROPOSITION 27.12 (Indexing alone imposes no probability equality). *Let*

$$\mathcal{E} : \mathbb{N} \longrightarrow \mathcal{E}_{\text{prob}}$$

be a probability family. Then the existence of $\mathcal{E}$ alone does not imply

$$\mathbb{P}(E_n) = \mathbb{P}(E_m)$$

for arbitrary $m, n \in \mathbb{N}$.

Proof. The assertion that $\mathcal{E}$ is a function with codomain $\mathcal{E}_{\text{prob}}$ specifies only that

$$E_n \in \mathcal{F}$$

for every $n \in \mathbb{N}$. It imposes no equality relation between distinct events E_n and E_m . Consequently, it imposes no equality between their probabilities. Any equality

$$\mathbb{P}(E_n) = \mathbb{P}(E_m)$$

therefore requires an additional hypothesis. □

27.6. Measurable Transformations.

Definition 27.13 (Measurable transformation). Let $(\Omega, \mathcal{F})$ be a measurable space. A function

$$T : \Omega \longrightarrow \Omega$$

is *measurable* if

$$T^{-1}(E) \in \mathcal{F}$$

for every

$$E \in \mathcal{F}.$$

Definition 27.14 (Induced transformation on events). Let

$$T : \Omega \longrightarrow \Omega$$

be measurable. The induced map on measurable events is

$$T^* : \mathcal{F} \longrightarrow \mathcal{F}, \quad T^*(E) := T^{-1}(E).$$

Definition 27.15 (Measure-preserving transformation). Let

$$(\Omega, \mathcal{F}, \mathbb{P})$$

be a probability space and let

$$T : \Omega \longrightarrow \Omega$$

be measurable. The map T is *measure-preserving* if

$$\mathbb{P}(T^{-1}(E)) = \mathbb{P}(E)$$

for every

$$E \in \mathcal{F}.$$

27.7. Iteration of Measurable Transformations.

LEMMA 27.16 (Measurability of iterates). *Let*

$$T : \Omega \longrightarrow \Omega$$

be measurable. Then

$$T^n : \Omega \longrightarrow \Omega$$

is measurable for every

$$n \in \mathbb{N}.$$

Proof. For $n = 0$,

$$T^0 = \text{id}_\Omega,$$

which is measurable.

Suppose that T^n is measurable. Since T is measurable and the composition of measurable maps is measurable,

$$T^{n+1} = T \circ T^n$$

is measurable. The result follows by induction. $\square$

THEOREM 27.17 (Iterated measure preservation). *Let*

$$(\Omega, \mathcal{F}, \mathbb{P})$$

be a probability space and let

$$T : \Omega \longrightarrow \Omega$$

be measurable and measure-preserving. Then

$$\mathbb{P}((T^n)^{-1}(E)) = \mathbb{P}(E)$$

for every

$$E \in \mathcal{F}$$

and every

$$n \in \mathbb{N}.$$

Proof. We proceed by induction on n .

For $n = 0$,

$$(T^0)^{-1}(E) = E,$$

so

$$\mathbb{P}((T^0)^{-1}(E)) = \mathbb{P}(E).$$

Assume that

$$\mathbb{P}((T^n)^{-1}(E)) = \mathbb{P}(E).$$

By Lemma 27.16, $(T^n)^{-1}(E) \in \mathcal{F}$. Hence, by measure preservation,

$$\mathbb{P}(T^{-1}((T^n)^{-1}(E))) = \mathbb{P}((T^n)^{-1}(E)).$$

Since

$$T^{-1}((T^n)^{-1}(E)) = (T^{n+1})^{-1}(E),$$

we obtain

$$\mathbb{P}((T^{n+1})^{-1}(E)) = \mathbb{P}(E).$$

The result follows by induction. $\square$

27.8. Probability Operators.

Definition 27.18 (Probability operator). *A probability operator on*

$$(\Omega, \mathcal{F}, \mathbb{P})$$

is a function

$$\Phi : \mathcal{E}_{\text{prob}} \longrightarrow \mathcal{E}_{\text{prob}}.$$

Definition 27.19 (Transformation-induced probability operator). Let

$$T : \Omega \longrightarrow \Omega$$

be measurable and let

$$\phi_{\mathcal{I}} : \mathcal{I}_{\text{prob}} \longrightarrow \mathcal{I}_{\text{prob}}$$

be a function. Define

$$\Phi_T : \mathcal{E}_{\text{prob}} \longrightarrow \mathcal{E}_{\text{prob}}$$

by

$$\Phi_T(\iota, E) := (\phi_{\mathcal{I}}(\iota), T^{-1}(E)).$$

PROPOSITION 27.20 (Measure preservation of an induced event operator). *Let T be measurable and measure-preserving, and let Φ_T be the induced probability operator defined above. Then*

$$\mathbb{P}_{\text{ind}}(\Phi_T(A)) = \mathbb{P}_{\text{ind}}(A)$$

for every

$$A \in \mathcal{E}_{\text{prob}}.$$

Proof. Let

$$A = (\iota, E).$$

Then

$$\Phi_T(A) = (\phi_{\mathcal{I}}(\iota), T^{-1}(E)).$$

Therefore

$$\mathbb{P}_{\text{ind}}(\Phi_T(A)) = \mathbb{P}(T^{-1}(E)).$$

Since T is measure-preserving,

$$\mathbb{P}(T^{-1}(E)) = \mathbb{P}(E).$$

By definition,

$$\mathbb{P}(E) = \mathbb{P}_{\text{ind}}(A).$$

Hence

$$\mathbb{P}_{\text{ind}}(\Phi_T(A)) = \mathbb{P}_{\text{ind}}(A).$$

□

27.9. Probability Engines.

Definition 27.21 (Auxiliary probability structure). Let $\mathcal{C}_{\text{prob}}$ be a specified set. An *auxiliary probability structure* is a function

$$\Gamma : \mathbb{N} \longrightarrow \mathcal{C}_{\text{prob}}.$$

Definition 27.22 (Probability engine). A *probability engine* is a tuple

$$\mathfrak{P} = (\Omega, \mathcal{F}, \mathbb{P}, \mathcal{E}, \Phi, m, \Gamma),$$

where

$$(\Omega, \mathcal{F}, \mathbb{P})$$

is a probability space,

$$\mathcal{E} : \mathbb{N} \longrightarrow \mathcal{E}_{\text{prob}}$$

is a probability family,

$$\Phi : \mathcal{E}_{\text{prob}} \longrightarrow \mathcal{E}_{\text{prob}}$$

is a probability operator,

$$m \in \mathbb{N},$$

and

$$\Gamma : \mathbb{N} \longrightarrow \mathcal{C}_{\text{prob}}$$

is an auxiliary structure.

Remark 27.23. The existence of a probability engine does not imply that Φ is measure-preserving. It does not imply independence, stationarity, ergodicity, mixing, convergence, or any other probabilistic property. Such properties require separate hypotheses.

27.10. Iteration of the Probability Operator.

Definition 27.24 (Iterated probability operator). Let

$$\Phi : \mathcal{E}_{\text{prob}} \longrightarrow \mathcal{E}_{\text{prob}}.$$

Define

$$\Phi^0 := \text{id}_{\mathcal{E}_{\text{prob}}}$$

and

$$\Phi^{n+1} := \Phi \circ \Phi^n.$$

For

$$A \in \mathcal{E}_{\text{prob}},$$

define

$$\text{Iter}_{\Phi}(n, A) := \Phi^n(A).$$

PROPOSITION 27.25 (Well-definedness of probability iteration). *For every probability operator*

$$\Phi : \mathcal{E}_{\text{prob}} \longrightarrow \mathcal{E}_{\text{prob}},$$

every

$$A \in \mathcal{E}_{\text{prob}},$$

and every

$$n \in \mathbb{N},$$

the element

$$\text{Iter}_{\Phi}(n, A)$$

belongs to

$$\mathcal{E}_{\text{prob}}.$$

Proof. We proceed by induction on n .

For $n = 0$,

$$\Phi^0(A) = A \in \mathcal{E}_{\text{prob}}.$$

Suppose that

$$\Phi^n(A) \in \mathcal{E}_{\text{prob}}.$$

Since Φ maps $\mathcal{E}_{\text{prob}}$ into itself,

$$\Phi^{n+1}(A) = \Phi(\Phi^n(A)) \in \mathcal{E}_{\text{prob}}.$$

Thus

$$\text{Iter}_{\Phi}(n, A) \in \mathcal{E}_{\text{prob}}$$

for every $n \in \mathbb{N}$. □

27.11. Probability Preservation Under Iteration.

THEOREM 27.26 (Preservation of indexed-event probabilities). *Let*

$$\Phi = \Phi_T$$

be induced by a measurable and measure-preserving transformation

$$T : \Omega \longrightarrow \Omega.$$

Then, for every

$$A \in \mathcal{E}_{\text{prob}}$$

and every

$$n \in \mathbb{N},$$

one has

$$\mathbb{P}_{\text{ind}}(\Phi^n(A)) = \mathbb{P}_{\text{ind}}(A).$$

Proof. Write

$$A = (\iota, E).$$

By induction on n ,

$$\Phi_T^n(A) = (\phi_{\mathcal{I}}^n(\iota), (T^n)^{-1}(E)),$$

where

$$\phi_{\mathcal{I}}^0 = \text{id}_{\mathcal{I}_{\text{prob}}}$$

and

$$\phi_{\mathcal{I}}^{n+1} = \phi_{\mathcal{I}} \circ \phi_{\mathcal{I}}^n.$$

Therefore

$$\mathbb{P}_{\text{ind}}(\Phi_T^n(A)) = \mathbb{P}((T^n)^{-1}(E)).$$

By Theorem 27.17,

$$\mathbb{P}((T^n)^{-1}(E)) = \mathbb{P}(E).$$

Finally,

$$\mathbb{P}(E) = \mathbb{P}_{\text{ind}}(A).$$

Hence

$$\mathbb{P}_{\text{ind}}(\Phi^n(A)) = \mathbb{P}_{\text{ind}}(A).$$

□

27.12. Properties Not Implied by Indexing.

PROPOSITION 27.27 (Independence is an additional hypothesis). *Let*

$$(E_n)_{n \in \mathbb{N}}$$

be a family of measurable events. The fact that the family is indexed by $\mathbb{N}$ does not imply that the events are independent.

Proof. Independence is a condition involving probabilities of finite intersections. For example, pairwise independence requires

$$\mathbb{P}(E_m \cap E_n) = \mathbb{P}(E_m)\mathbb{P}(E_n)$$

for distinct m, n . No such equality is contained in the definition of an indexed family. Therefore independence must be imposed or proved separately. □

PROPOSITION 27.28 (Stationarity is an additional hypothesis). *Let*

$$(E_n)_{n \in \mathbb{N}}$$

be a family of measurable events. The existence of the family does not imply stationarity.

Proof. Stationarity is a relation between the distributions of finite collections of variables or events under index shifts. The definition of an indexed family imposes no such relation. Therefore stationarity requires an additional hypothesis. □

PROPOSITION 27.29 (Ergodicity and mixing are additional hypotheses).

Let

$$T : \Omega \longrightarrow \Omega$$

be a measurable transformation of a probability space. Even if T is measure-preserving, ergodicity and mixing do not follow from measure preservation alone.

Proof. Measure preservation requires only

$$\mathbb{P}(T^{-1}(E)) = \mathbb{P}(E)$$

for every measurable event E . Ergodicity and mixing impose additional relations on invariant events and on correlations between iterated events, respectively. Since these relations are not contained in the definition of measure preservation, neither property follows from it without additional assumptions. $\square$

27.13. *Scope of the Probability Construction.* The probability structures introduced above provide a mathematically well-defined setting for indexed events, measurable transformations, and probability-preserving dynamics.

They do not, by themselves, establish any empirical interpretation. In particular, the existence of

$$\mathfrak{P} = (\Omega, \mathcal{F}, \mathbb{P}, \mathcal{E}, \Phi, m, \Gamma)$$

does not imply that the events in $\mathcal{E}$ represent physical measurements, that the operator Φ describes an experimentally realized process, or that any empirical distribution is governed by the model.

A probabilistic claim in an application of the framework must therefore identify explicitly:

- (1) the sample space Ω ;
- (2) the measurable structure $\mathcal{F}$;
- (3) the probability measure $\mathbb{P}$;
- (4) the measurable events representing the relevant observations;
- (5) the transformation or stochastic mechanism generating the dynamics;
- (6) the additional hypotheses required for independence, stationarity, ergodicity, mixing, convergence, or other claimed properties.

No one of these properties is inferred merely from the existence of an index, a mode parameter, an auxiliary channel structure, or an abstract operator.

Consequently, the probabilistic content of the framework is exactly the content established by the explicit definitions, axioms, and measure-theoretic hypotheses imposed on the model.

28. The Faure Operator Engine

This section introduces the operator component of the Faure framework. The purpose is to distinguish the state spaces on which operators act from the operators themselves and from their iterates. All algebraic and dynamical properties are treated as explicit hypotheses or as conclusions of stated propositions.

Throughout this section,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No property such as injectivity, surjectivity, invertibility, idempotence, commutativity, periodicity, or convergence is assumed unless explicitly stated.

28.1. Operator Indices.

Definition 28.1 (Operator index). An *operator index* is an ordered pair

$$\iota = (i, t) \in \mathbb{N}^2.$$

The set of operator indices is

$$\mathcal{I}_{\text{op}} := \mathbb{N}^2.$$

For $\iota = (i, t) \in \mathcal{I}_{\text{op}}$, define

$$\text{id}(\iota) := i, \quad \text{tag}(\iota) := t.$$

28.2. Operator Data and Values.

Let $\mathcal{D}_{\text{op}}$ be a fixed set of operator data.

Definition 28.2 (Operator value). An *operator value* is an ordered pair

$$v = (\iota, d),$$

where $\iota \in \mathcal{I}_{\text{op}}$ and $d \in \mathcal{D}_{\text{op}}$. The set of operator values is

$$\mathcal{V}_{\text{op}} = \mathcal{I}_{\text{op}} \times \mathcal{D}_{\text{op}}.$$

For $v = (\iota, d) \in \mathcal{V}_{\text{op}}$, define

$$\text{index}(v) := \iota, \quad \text{data}(v) := d.$$

28.3. Operator Cells.

Let $\mathcal{A}_{\text{op}}$ be a fixed set of auxiliary operator data.

Definition 28.3 (Operator cell). An *operator cell* is a triple

$$c = (j, v, a),$$

where $j \in \mathbb{N}$, $v \in \mathcal{V}_{\text{op}}$, and $a \in \mathcal{A}_{\text{op}}$. The set of operator cells is

$$\mathcal{C}_{\text{op}} = \mathbb{N} \times \mathcal{V}_{\text{op}} \times \mathcal{A}_{\text{op}}.$$

For $c = (j, v, a) \in \mathcal{C}_{\text{op}}$, define

$$\text{pos}(c) := j, \quad \text{val}(c) := v, \quad \text{info}(c) := a.$$

28.4. Operator Spaces.

Definition 28.4 (Operator space). An *operator space* is a quadruple

$$\mathcal{S} = (\mathcal{C}_{\text{op}}, \lambda, \Pi),$$

where

$$\lambda : \mathbb{N} \rightarrow \mathcal{C}_{\text{op}}$$

is a distinguished family of operator cells, $\lambda \in \mathbb{N}$ is a label, and

$$\Pi : \mathbb{N} \rightarrow \mathcal{P}(\mathcal{D}_{\text{op}})$$

is an auxiliary payload family.

No injectivity, surjectivity, periodicity, or compatibility property of λ is assumed by the definition.

28.5. Endomorphisms.

Definition 28.5 (Endomorphism). Let X be a set. An *endomorphism* of X is a function

$$F : X \longrightarrow X.$$

The set of all endomorphisms of X is denoted by

$$\text{End}(X).$$

28.6. Operator Structure.

Definition 28.6 (Faure operator). A *Faure operator* is a triple

$$\mathcal{O} = (\Phi_{\mathcal{V}}, \Phi_{\mathcal{C}}, \tau),$$

where

$$\Phi_{\mathcal{V}} : \mathcal{V}_{\text{op}} \rightarrow \mathcal{V}_{\text{op}}, \quad \Phi_{\mathcal{C}} : \mathcal{C}_{\text{op}} \rightarrow \mathcal{C}_{\text{op}}, \quad \tau \in \mathbb{N}.$$

Definition 28.7 (Compatibility condition). A *compatibility condition* is any explicitly stated relation involving $\Phi_{\mathcal{V}}$, $\Phi_{\mathcal{C}}$, the operator space, or additional specified data.

28.7. *The Faure Operator Engine.*

Definition 28.8 (Faure operator engine). A *Faure operator engine* is a quadruple

$$\mathfrak{D} = (\mathcal{S}, \mathcal{O}, m, \Gamma),$$

where $\mathcal{S}$ is an operator space, $\mathcal{O}$ is a Faure operator, $m \in \mathbb{N}$, and $\Gamma : \mathbb{N} \rightarrow \mathcal{C}$ is an auxiliary channel family for some fixed set $\mathcal{C}$.

28.8. *Iteration of an Endomorphism.*

Definition 28.9 (Iterated endomorphism). Let $F : X \rightarrow X$ be an endomorphism. Define

$$F^0 := \text{id}_X, \quad F^{n+1} := F \circ F^n.$$

28.9. *Iteration of Operator Cells.* Let $\mathfrak{D} = (\mathcal{S}, \mathcal{O}, m, \Gamma)$ and $\mathcal{O} = (\Phi_{\mathcal{V}}, \Phi_{\mathcal{C}}, \tau)$.

Definition 28.10 (Iterated operator cell). For $c \in \mathcal{C}_{\text{op}}$ and $n \in \mathbb{N}$, define

$$\text{Iter}_{\mathfrak{D}}(n, c) := \Phi_{\mathcal{C}}^n(c).$$

28.10. *Well-Definedness of Operator Iteration.*

PROPOSITION 28.11 (Well-definedness). *For every $c \in \mathcal{C}_{\text{op}}$ and $n \in \mathbb{N}$,*

$$\text{Iter}_{\mathfrak{D}}(n, c) \in \mathcal{C}_{\text{op}}$$

and is uniquely determined.

Proof. Induction on n . Base case: $n = 0$. Inductive step: $\Phi_{\mathcal{C}}$ maps $\mathcal{C}_{\text{op}}$ to itself. $\square$

28.11. *Composition Law for Iterates.*

PROPOSITION 28.12 (Composition law). *For any endomorphism $F : X \rightarrow X$ and any $m, n \in \mathbb{N}$,*

$$F^{m+n} = F^m \circ F^n.$$

Proof. Induction on m . $\square$

28.12. *Invariant Subsets.*

Definition 28.13 (Invariant subset). A subset $Y \subseteq X$ is *invariant* under F if $F(Y) \subseteq Y$.

PROPOSITION 28.14 (Preservation). *If Y is invariant, then $F^n(Y) \subseteq Y$ for all $n \in \mathbb{N}$.*

Proof. Induction on n . □

28.13. Bijective Operators.

PROPOSITION 28.15 (Iterates of a bijection). *If $F : X \rightarrow X$ is bijective, then F^n is bijective and*

$$(F^n)^{-1} = (F^{-1})^n.$$

Proof. Induction on n and the identity $(G \circ H)^{-1} = H^{-1} \circ G^{-1}$. □

28.14. Idempotence and Periodicity.

Definition 28.16 (Idempotent). F is idempotent if $F^2 = F$.

PROPOSITION 28.17. *If F is idempotent, then $F^n = F$ for all $n \geq 1$.*

Proof. Induction on n . □

Definition 28.18 (Periodic). F is periodic with period $p > 0$ if $F^p = \text{id}_X$.

PROPOSITION 28.19. *If F is periodic with period p , then $F^{n+p} = F^n$ for all n .*

Proof. Use $F^{n+p} = F^n \circ F^p$. □

28.15. Structural Limitations.

PROPOSITION 28.20 (No dynamical property from totality). *From $F : X \rightarrow X$ alone one may conclude only that $F^n(x) \in X$. No further property (injective, surjective, invertible, periodic, etc.) follows without explicit hypotheses.*

28.16. *Scope of the Operator Construction.* The operator construction establishes a precise mathematical setting for state spaces, endomorphisms, operator cells, and iterated transformations.

The existence of a Faure operator engine does not imply any physical, probabilistic, or empirical interpretation. Any such interpretation must be supported by additional hypotheses or evidence.

29. The Faure MRDP Generator

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure MRDP Generator. The purpose is to provide a fully classical, self-contained, and hypothesis-driven description of the deterministic generative layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic or dynamical property is assumed unless explicitly stated.

29.1. *Generator States.*

Definition 29.1 (Generator State). Let $\mathcal{T}$ be a fixed ambient class of sets. A *generator state* is a triple

$$S = (i, m, P),$$

where $i, m \in \mathbb{N}$ and

$$P : \mathbb{N} \rightarrow \mathcal{T}$$

is a payload family.

This corresponds to the Lean structure:

```
structure GState := (id : Nat) (mode : Nat) (payload : Nat -> Type).
```

29.2. *Generator Rules.*

Definition 29.2 (Generator Rule). Let $\mathcal{I}$ be a fixed ambient class. A *generator rule* is a quadruple

$$R = (S_{\text{src}}, S_{\text{dst}}, t, I),$$

where S_{src} and S_{dst} are generator states, $t \in \mathbb{N}$ is a rule tag, and $I \in \mathcal{I}$ is auxiliary data.

Lean structure:

```
structure GRule := (srcState : GState) (dstState : GState) (tag : Nat) (info : Typ
```

29.3. *MRDP-like Models.*

Definition 29.3 (MRDP-like Model). Let $\mathcal{T}_\Sigma$ and $\mathcal{T}_C$ be fixed ambient classes. An *MRDP-like model* is a triple

$$M = (i, \Sigma, C),$$

where $i \in \mathbb{N}$ and

$$\Sigma : \mathbb{N} \rightarrow \mathcal{T}_\Sigma, \quad C : \mathbb{N} \rightarrow \mathcal{T}_C.$$

Lean structure:

```
structure GModel := (id : Nat) (signature : Nat -> Type) (config : Nat -> Type).
```

29.4. Generator Links.

Definition 29.4 (Generator Link). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *generator link* is a triple

$$L = (S, M, \Gamma),$$

where S is a generator state, M is an MRDP-like model, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family.

Lean structure:

```
structure GLink := (state : GState) (model : GModel) (channel : Nat -> Type).
```

29.5. Universal Generator.

Definition 29.5 (Faure Universal Generator). Let $\mathcal{R}$, $\mathcal{M}$, and $\mathcal{L}$ denote the classes of generator rules, MRDP-like models, and generator links respectively. A *Faure Universal Generator* is a quintuple

$$G = (S_0, R, M, L, \lambda),$$

where:

- S_0 is a generator state;
- $R : \mathbb{N} \rightarrow \mathcal{R}$ is a rule family;
- $M : \mathbb{N} \rightarrow \mathcal{M}$ is a model family;
- $L : \mathbb{N} \rightarrow \mathcal{L}$ is a link family;
- $\lambda \in \mathbb{N}$ is a global label.

Lean structure:

```
structure GUniversal := (baseState : GState) (rule : Nat -> GRule) (model : Nat -> GModel) (link : Nat -> GLink) (label : Nat).
```

29.6. Generator Transitions. The rule family alone does not define an iteration. A transition map must be explicitly specified.

Definition 29.6 (Generator Transition). Let G be a Faure Universal Generator. A *generator transition* is a map

$$\tau_G : \mathcal{S} \rightarrow \mathcal{S},$$

where $\mathcal{S}$ is the class of generator states.

No compatibility between τ_G and the rule family is assumed unless explicitly stated.

29.7. Iterated Generator States.

Definition 29.7 (Iterated Generator State). Let G be a Faure Universal Generator and let

$$\tau_G : \mathcal{S} \rightarrow \mathcal{S}$$

be its transition map. For $S \in \mathcal{S}$ and $n \in \mathbb{N}$, define

$$\text{Iter}_G(0, S) = S,$$

and

$$\text{Iter}_G(n+1, S) = \tau_G(\text{Iter}_G(n, S)).$$

Lean interpretation:

```
def iterateG (G : GUniversal) : Nat -> GState -> GState.
```

29.8. Structural Theorem.

THEOREM 29.8 (Well-definedness of Generator Iteration). *Let G be a Faure Universal Generator and let*

$$\tau_G : \mathcal{S} \rightarrow \mathcal{S}$$

be its transition map. Then, for every $S \in \mathcal{S}$ and every $n \in \mathbb{N}$,

$$\text{Iter}_G(n, S) \in \mathcal{S}.$$

Moreover, the iterate is uniquely determined.

Proof. Induction on n . Base case: $\text{Iter}_G(0, S) = S \in \mathcal{S}$. Inductive step: τ_G maps $\mathcal{S}$ to itself. $\square$

30. The Faure MRDP Engine

In this section we present the classical mathematical translation of the formal Lean structures defining the Faure MRDP Engine. These objects constitute the deterministic MRDP layer of the Faure Framework.

30.1. MRDP Indices and Variables.

Definition 30.1 (MRDP Index). An *MRDP index* is a pair

$$I = (i, t),$$

where

$$i, t \in \mathbb{N}.$$

This corresponds to the Lean structure

```
structure MIndex := (id : Nat) (tag : Nat).
```

Definition 30.2 (MRDP Variable). An *MRDP variable* is a pair

$$V = (I, D),$$

where I is an MRDP index and D is an explicitly specified datum in a fixed ambient class $\mathcal{D}$.

This corresponds to the Lean structure

```
structure MVar := (idx : MIndex) (data : Type).
```

30.2. MRDP Constraints and Spaces.

Definition 30.3 (MRDP Constraint). An *MRDP constraint* is a triple

$$C = (i, V, I),$$

where $i \in \mathbb{N}$, V is an MRDP variable, and I is an explicitly specified auxiliary datum in a fixed ambient class $\mathcal{I}_C$.

This corresponds to the Lean structure

```
structure MConstraint := (id : Nat) (var : MVar) (info : Type).
```

Definition 30.4 (MRDP Space). An *MRDP space* is a triple

$$S = (C, \lambda, P),$$

where:

-

$$C : \mathbb{N} \rightarrow \mathcal{C}$$

is a family of MRDP constraints;

- $\lambda \in \mathbb{N}$ is a global label;

-

$$P : \mathbb{N} \rightarrow \mathcal{T}_P$$

is a payload family for a fixed ambient class $\mathcal{T}_P$.

This corresponds to the Lean structure

```
structure MSpace := (constraint : Nat -> MConstraint) (label : Nat) (payload : Nat
```

30.3. MRDP Operators and Engines.

Definition 30.5 (MRDP Operator). An *MRDP operator* is a triple

$$O = (\phi_V, \phi_C, t),$$

where

$$\phi_V : \mathcal{V} \rightarrow \mathcal{V}, \quad \phi_C : \mathcal{C} \rightarrow \mathcal{C}, \quad t \in \mathbb{N},$$

with $\mathcal{V}$ and $\mathcal{C}$ denoting the classes of MRDP variables and MRDP constraints, respectively.

This corresponds to the Lean structure

```
structure MOperator := (opVar : MVar -> MVar) (opConstraint : MConstraint -> MConstraint)
```

Definition 30.6 (Faure MRDP Engine). A *Faure MRDP Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where:

- S is an MRDP space;
- O is an MRDP operator;
- $m \in \mathbb{N}$ is a mode parameter;
-

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family for a fixed ambient class $\mathcal{T}_\Gamma$.

This corresponds to the Lean structure

```
structure MEngine := (baseSpace : MSpace) (op : MOperator) (mode : Nat) (channel :
```

30.4. *Iterated MRDP Constraints.* The original definition

$$\text{Iter}_E(n+1, C) = E.\text{baseSpace.constraint}(C.\text{id})$$

does not define iteration of the operator, because the right-hand side does not depend on the previously computed constraint. The mathematically consistent translation of an operator-based Lean iteration is therefore the following.

Definition 30.7 (Iterated MRDP Constraint). Let E be a Faure MRDP Engine. Write

$$\phi_C^E : \mathcal{C} \rightarrow \mathcal{C}$$

for the constraint component of its MRDP operator. For $C \in \mathcal{C}$ and $n \in \mathbb{N}$, define

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n+1, C) = \phi_C^E(\text{Iter}_E(n, C)).$$

Thus the corresponding Lean definition must have the semantic form

```
def iterateConstraint (E : MEngine) : Nat -> MConstraint -> MConstraint
```

with recursive application of

```
E.op.opConstraint.
```

30.5. *Structural Theorem.*

THEOREM 30.8 (Well-definedness of MRDP Iteration). *Let E be a Faure MRDP Engine. For every MRDP constraint $C \in \mathcal{C}$ and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C) \in \mathcal{C}.$$

Moreover, the iterate is uniquely determined by the recursive definition.

Proof. We argue by induction on n .

For $n = 0$,

$$\text{Iter}_E(0, C) = C,$$

so

$$\text{Iter}_E(0, C) \in \mathcal{C}.$$

Assume

$$\text{Iter}_E(n, C) \in \mathcal{C}.$$

Since

$$\phi_C^E : \mathcal{C} \rightarrow \mathcal{C},$$

we have

$$\phi_C^E(\text{Iter}_E(n, C)) \in \mathcal{C}.$$

By the recursive definition,

$$\text{Iter}_E(n+1, C) = \phi_C^E(\text{Iter}_E(n, C)).$$

Hence

$$\text{Iter}_E(n+1, C) \in \mathcal{C}.$$

The claim follows by induction, and uniqueness follows from the recursive definition. $\square$

31. Classical vs. Faure Diophantine Structures

This section introduces the mathematical structures corresponding to the Lean formalization of the comparison layer between classical Diophantine objects and their Faure counterparts. The purpose is to provide a hypothesis-driven, fully classical description of the deterministic bridge between the MRDP setting and the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or computability property is assumed unless explicitly stated.

31.1. *Classical Diophantine Structures.*

Definition 31.1 (Classical Diophantine Polynomial). Let $\mathcal{T}_V$ be a fixed ambient class. A *classical Diophantine polynomial descriptor* is a pair

$$P = (i, n),$$

where $i, n \in \mathbb{N}$.

Lean structure:

```
structure Poly := (idx : Nat) (arity : Nat).
```

Definition 31.2 (Classical Diophantine Equation). Let $\mathcal{T}_V$ be a fixed ambient class and let Z be a designated zero datum. A *classical Diophantine equation descriptor* is a triple

$$E = (P, V, Z),$$

where P is a polynomial descriptor and

$$V : \mathbb{N} \rightarrow \mathcal{T}_V$$

is a variable family.

Lean structure:

```
structure Equation := (poly : Poly) (vars : Nat -> Type) (zero : Type).
```

Definition 31.3 (Classical Solution Structure). Let $\mathcal{T}_X$ be a fixed ambient class. A *classical solution structure* is a pair

$$S = (E, X),$$

where E is a classical Diophantine equation and

$$X : \mathbb{N} \rightarrow \mathcal{T}_X$$

is a solution family.

Lean structure:

```
structure SolutionSet := (eq : Equation) (elem : Nat -> Type).
```

31.2. Faure Diophantine Structures.

Definition 31.4 (Faure Diophantine Object). Let $\mathcal{T}_L$ be a fixed ambient class. A *Faure Diophantine object* is a pair

$$F = (i, \ell),$$

where $i, \ell \in \mathbb{N}$.

Lean structure:

```
structure FaureDio := (idx : Nat) (latRef : Nat).
```

Definition 31.5 (Faure Diophantine Relation). Let $\mathcal{T}_L$ be a fixed ambient class. A *Faure Diophantine relation* is a pair

$$R = (F, L),$$

where F is a Faure Diophantine object and

$$L : \mathbb{N} \rightarrow \mathcal{T}_L$$

is a link family.

Lean structure:

```
structure FaureRelation := (obj : FaureDio) (link : Nat -> Type).
```

Definition 31.6 (Faure Solution Structure). Let $\mathcal{T}_X$ be a fixed ambient class. A *Faure solution structure* is a pair

$$T = (R, X),$$

where R is a Faure Diophantine relation and

$$X : \mathbb{N} \rightarrow \mathcal{T}_X$$

is a solution family.

Lean structure:

```
structure FaureSolution := (rel : FaureRelation) (elem : Nat -> Type).
```

31.3. The Classical–Faure Bridge.

Definition 31.7 (Classical–Faure Diophantine Bridge). Let $\mathcal{T}_S$ be a fixed ambient class. A *classical–Faure Diophantine bridge* is a septuple

$$B = (E, S, F, R, T, \phi_I, \phi_S),$$

where:

- E is a classical Diophantine equation;
- S is a classical solution structure;
- F is a Faure Diophantine object;
- R is a Faure Diophantine relation;
- T is a Faure solution structure;
- $\phi_I : \mathbb{N} \rightarrow \mathbb{N}$ is an index mapping;
- $\phi_S : \mathbb{N} \rightarrow \mathcal{T}_S$ is a structural mapping.

Lean structure:

```
structure Bridge :=

(classicalEq : Equation) (classicalSol : SolutionSet) (faureObj : FaureDio)

(faureRel : FaureRelation) (faureSol : FaureSolution)

(mapIndex : Nat -> Nat) (mapStructure : Nat -> Type).
```

31.4. Structural Theorem.

THEOREM 31.8 (Existence of the Classical–Faure Bridge). *Let E, S, F, R , and T be objects of the respective classes appearing in the definition of a classical–Faure bridge. Let*

$$\phi_I : \mathbb{N} \rightarrow \mathbb{N}, \quad \phi_S : \mathbb{N} \rightarrow \mathcal{T}_S,$$

be arbitrary mappings. Then the septuple

$$(E, S, F, R, T, \phi_I, \phi_S)$$

is a classical–Faure bridge.

Proof. The Lean structure **Bridge** imposes no compatibility conditions between its fields. Thus, once objects of the required classes and mappings of the required types have been chosen, the septuple is an instance of the bridge structure. Therefore the bridge exists. $\square$

31.5. *Structural Limitation.* The existence theorem above is purely structural. It does not assert that the bridge preserves solutions, Diophantine definability, computability, semantic equivalence, or any other property. Such statements require additional hypotheses relating the classical and Faure components and the mappings ϕ_I and ϕ_S .

32. The Faure Meta Engine

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Meta Engine. The purpose is to provide a fully classical, hypothesis-driven description of the meta-level operator layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

32.1. Meta Indices and Values.

Definition 32.1 (Meta Index). A *meta index* is an ordered pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Lean structure:

```
structure MIndex := (id : Nat) (tag : Nat).
```

Definition 32.2 (Meta Value). Let $\mathcal{T}_D$ be a fixed ambient class. A *meta value* is a pair

$$V = (I, D),$$

where I is a meta index and $D \in \mathcal{T}_D$ is meta-level data.

Lean structure:

```
structure MValue := (idx : MIndex) (data : Type).
```

32.2. Meta Cells and Meta Spaces.

Definition 32.3 (Meta Cell). Let $\mathcal{T}_C$ be a fixed ambient class. A *meta cell* is a triple

$$C = (i, V, D),$$

where $i \in \mathbb{N}$, V is a meta value, and $D \in \mathcal{T}_C$ is auxiliary information.

Lean structure:

```
structure MCell := (id : Nat) (val : MValue) (info : Type).
```

Definition 32.4 (Meta Space). Let $\mathcal{T}_P$ be a fixed ambient class. A *meta space* is a triple

$$S = (c, \lambda, p),$$

where

$$c : \mathbb{N} \rightarrow \text{MCell}, \quad \lambda \in \mathbb{N}, \quad p : \mathbb{N} \rightarrow \mathcal{T}_P.$$

Lean structure:

```
structure MSpace := (cell : Nat → MCell) (label : Nat) (payload : Nat → Type).
```

32.3. Meta Operators and Engines.

Definition 32.5 (Meta Operator). Let $\varphi_V : \text{MValue} \rightarrow \text{MValue}$ and $\varphi_C : \text{MCell} \rightarrow \text{MCell}$ be total maps. A *meta operator* is a triple

$$O = (\varphi_V, \varphi_C, t),$$

where $t \in \mathbb{N}$ is a tag.

Lean structure:

```
structure MOperator := (opValue : MValue → MValue) (opCell : MCell → MCell) (tag : Nat).
```

Definition 32.6 (Faure Meta Engine). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *Faure Meta Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a meta space, O is a meta operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family.

Lean structure:

```
structure MEngine := (baseSpace : MSpace) (op : MOperator) (mode : Nat) (channel : Nat → TΓ).
```

32.4. Iterated Meta Cells.

Definition 32.7 (Iterated Meta Cell). Let E be a Faure Meta Engine and let C be a meta cell. The *iterated meta cell* is the function

$$\text{Iter}_E : \mathbb{N} \rightarrow \text{MCell} \rightarrow \text{MCell}$$

defined recursively by

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n + 1, C) = E.\text{baseSpace.cell}(C.\text{id}).$$

Lean declaration:

```
def iterateCell (E : MEngine) : Nat → MCell → MCell.
```

Remark 32.8. The recursive clause above is the literal structural translation of the Lean definition. In particular, the cell supplied as the second argument remains the one whose identifier is used at every positive iteration. No application of the operator $E.\text{op}$ is implied.

32.5. Structural Theorem.

THEOREM 32.9 (Well-Definedness of Meta Iteration). *Let E be a Faure Meta Engine and let C be a meta cell. Then, for every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C)$$

is a well-defined meta cell.

Proof. Induction on n . Base case: $\text{Iter}_E(0, C) = C$, which is a meta cell. Inductive step: $E.\text{baseSpace.cell}$ is a total map $\mathbb{N} \rightarrow \text{MCell}$, hence its value at $C.\text{id}$ is a meta cell. Thus $\text{Iter}_E(n + 1, C)$ is well-defined. $\square$

33. The Faure Measure Engine

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Measure Engine. The purpose is to provide a fully classical, hypothesis-driven description of the measure-level operator layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

33.1. *Measure Indices and Values.*

Definition 33.1 (Measure Index). A *measure index* is an ordered pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Lean structure:

```
structure MIndex := (id : Nat) (tag : Nat).
```

Definition 33.2 (Measure Value). Let $\mathcal{T}_D$ be a fixed ambient class. A *measure value* is a pair

$$V = (I, D),$$

where I is a measure index and $D \in \mathcal{T}_D$ is measure-level data.

Lean structure:

```
structure MValue := (idx : MIndex) (data : Type).
```

33.2. *Measure Cells and Measure Spaces.*

Definition 33.3 (Measure Cell). Let $\mathcal{T}_C$ be a fixed ambient class. A *measure cell* is a triple

$$C = (p, V, D),$$

where $p \in \mathbb{N}$ is a position, V is a measure value, and $D \in \mathcal{T}_C$ is auxiliary information.

Lean structure:

```
structure MCell := (pos : Nat) (val : MValue) (info : Type).
```

Definition 33.4 (Measure Space). Let $\mathcal{T}_P$ be a fixed ambient class. A *measure space* is a triple

$$S = (c, \lambda, p),$$

where

$$c : \mathbb{N} \rightarrow \text{MCell}, \quad \lambda \in \mathbb{N}, \quad p : \mathbb{N} \rightarrow \mathcal{T}_P.$$

Lean structure:

```
structure MSpace := (cell : Nat → MCell) (label : Nat) (payload : Nat → Type).
```

33.3. Measure Operators and Engines.

Definition 33.5 (Measure Operator). Let $\varphi_I : \text{MIndex} \rightarrow \text{MIndex}$ and $\varphi_V : \text{MValue} \rightarrow \text{MValue}$ be total maps. A *measure operator* is a triple

$$O = (\varphi_I, \varphi_V, t),$$

where $t \in \mathbb{N}$ is a tag.

Lean structure:

```
structure MOperator := (opIndex : MIndex → MIndex) (opValue : MValue → MValue) (tag
```

Definition 33.6 (Faure Measure Engine). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *Faure Measure Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a measure space, O is a measure operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family.

Lean structure:

```
structure MEngine := (baseSpace : MSpace) (op : MOperator) (mode : Nat) (channel :
```

33.4. Iterated Measure Cells.

Definition 33.7 (Iterated Measure Cell). Let E be a Faure Measure Engine and let C be a measure cell. The *iterated measure cell* is the function

$$\text{Iter}_E : \mathbb{N} \rightarrow \text{MCell} \rightarrow \text{MCell}$$

defined recursively by

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n + 1, C) = E.\text{baseSpace}.\text{cell}(C.\text{pos}).$$

Lean declaration:

```
def iterateCell (E : MEngine) : Nat → MCell → MCell.
```

Remark 33.8. The recursive clause above is the literal structural translation of the Lean definition. It performs a lookup using the position of the original cell C and does not apply the operator $E.op$. No dynamical property is implied.

33.5. Structural Theorem.

THEOREM 33.9 (Well-Definedness of Measure Iteration). *Let E be a Faure Measure Engine and let C be a measure cell. Then, for every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C)$$

is a well-defined measure cell.

Proof. Induction on n . Base case: $\text{Iter}_E(0, C) = C$, which is a measure cell. Inductive step: $E.\text{baseSpace.cell}$ is a total map $\mathbb{N} \rightarrow \text{MCell}$, hence its value at $C.\text{pos}$ is a measure cell. Thus $\text{Iter}_E(n + 1, C)$ is well-defined. $\square$

34. The Faure Logic Engine

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Logic Engine. The purpose is to provide a fully classical, hypothesis-driven description of the logical layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

34.1. Logical Indices and Atoms.

Definition 34.1 (Logical Index). A *logical index* is an ordered pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Lean structure:

```
structure LIndex := (id : Nat) (tag : Nat).
```

Definition 34.2 (Logical Atom). Let $\mathcal{T}_A$ be a fixed ambient class. A *logical atom* is a pair

$$A = (I, D),$$

where I is a logical index and $D \in \mathcal{T}_A$ is logical data.

Lean structure:

```
structure LAtom := (idx : LIndex) (data : Type).
```

34.2. Logical Clauses and Logical Spaces.

Definition 34.3 (Logical Clause). Let $\mathcal{T}_C$ be a fixed ambient class. A *logical clause* is a triple

$$C = (i, A, D),$$

where $i \in \mathbb{N}$, A is a logical atom, and $D \in \mathcal{T}_C$ is auxiliary information.

Lean structure:

```
structure LClause := (id : Nat) (atom : LAtom) (info : Type).
```

Definition 34.4 (Logical Space). Let $\mathcal{T}_P$ be a fixed ambient class. A *logical space* is a triple

$$S = (c, \lambda, p),$$

where

$$c : \mathbb{N} \rightarrow \text{LClause}, \quad \lambda \in \mathbb{N}, \quad p : \mathbb{N} \rightarrow \mathcal{T}_P.$$

Lean structure:

```
structure LSpace := (clause : Nat → LClause) (label : Nat) (payload : Nat → Type).
```

34.3. Logic Operators and Engines.

Definition 34.5 (Logic Operator). Let $\varphi_A : \text{LAtom} \rightarrow \text{LAtom}$ and $\varphi_C : \text{LClause} \rightarrow \text{LClause}$ be total maps. A *logic operator* is a triple

$$O = (\varphi_A, \varphi_C, t),$$

where $t \in \mathbb{N}$ is a tag.

Lean structure:

```
structure LOperator := (opAtom : LAtom → LAtom) (opClause : LClause → LClause) (tag
```

Definition 34.6 (Faure Logic Engine). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *Faure Logic Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a logical space, O is a logic operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family.

Lean structure:

```
structure LEngine := (baseSpace : LSpace) (op : LOperator) (mode : Nat) (channel :
```

34.4. *Iterated Logical Clauses.*

Definition 34.7 (Iterated Logical Clause). Let E be a Faure Logic Engine and let C be a logical clause. The *iterated logical clause* is the function

$$\text{Iter}_E : \mathbb{N} \rightarrow \text{LClause} \rightarrow \text{LClause}$$

defined recursively by

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n + 1, C) = E.\text{baseSpace}.\text{clause}(C.\text{id}).$$

Lean declaration:

```
def iterateClause (E : LEngine) : Nat → LClause → LClause.
```

Remark 34.8. The recursive clause above is the literal structural translation of the Lean definition. It performs a lookup using the identifier of the original clause C and does not apply the operator $E.\text{op}$. No dynamical property is implied.

34.5. *Structural Theorem.*

THEOREM 34.9 (Well-Definedness of Logical Iteration). *Let E be a Faure Logic Engine and let C be a logical clause. Then, for every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C)$$

is a well-defined logical clause.

Proof. Induction on n . Base case: $\text{Iter}_E(0, C) = C$, which is a logical clause. Inductive step: $E.\text{baseSpace.clause}$ is a total map $\mathbb{N} \rightarrow \mathbf{LClause}$, hence its value at $C.\text{id}$ is a logical clause. Thus $\text{Iter}_E(n+1, C)$ is well-defined. $\square$

35. The Faure Deterministic Kernel

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Deterministic Kernel. The purpose is to provide a fully classical, hypothesis-driven description of the deterministic kernel layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

35.1. Kernel Points.

Definition 35.1 (Kernel Point). Let $\mathcal{T}_D$ be a fixed ambient class. A *kernel point* is a pair

$$p = (i, D),$$

where $i \in \mathbb{N}$ and $D \in \mathcal{T}_D$.

Lean structure:

```
structure KPoint := (idx : Nat) (data : Type).
```

35.2. Deterministic Kernel.

Definition 35.2 (Deterministic Kernel). Let $\mathcal{T}_L$ and $\mathcal{T}_S$ be fixed ambient classes. A *deterministic kernel* is a triple

$$K = (B, L, S),$$

where

$$B : \mathbb{N} \rightarrow \mathbf{KPoint}, \quad L : \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathcal{T}_L, \quad S : \mathbb{N} \rightarrow \mathcal{T}_S.$$

Lean structure:

```
structure Kernel := (base : Nat -> KPoint) (link : Nat -> Nat -> Type) (shape : Nat
```

35.3. Kernel Composition.

Definition 35.3 (Kernel Composition). Let $\mathcal{T}_X$ be a fixed ambient class. A *kernel composition* is a quadruple

$$C = (K, p_\ell, p_r, X),$$

where K is a deterministic kernel, p_ℓ, p_r are kernel points, and $X \in \mathcal{T}_X$.

Lean structure:

```
structure KComposition := (K : Kernel) (left : KPoint) (right : KPoint) (comp : Typ
```

35.4. Kernel Sectors.

Definition 35.4 (Kernel Sector). Let K be a deterministic kernel. A *kernel sector* is a triple

$$T = (K, t, E),$$

where $t \in \mathbb{N}$ and

$$E : \mathbb{N} \rightarrow \text{KPoint}.$$

Lean structure:

```
structure KSector := (K : Kernel) (tag : Nat) (elem : Nat -> KPoint).
```

35.5. Kernel Stability.

Definition 35.5 (Kernel Stability). Let $\mathcal{T}_R$ be a fixed ambient class. A *kernel stability structure* is a pair

$$U = (T, R),$$

where T is a kernel sector and

$$R : \mathbb{N} \rightarrow \mathcal{T}_R.$$

Lean structure:

```
structure KStability := (S : KSector) (rule : Nat -> Type).
```

35.6. *Kernel Closure.*

Definition 35.6 (Kernel Closure). Let $\mathcal{T}_C$ and $\mathcal{T}_{L'}$ be fixed ambient classes. A *kernel closure structure* is a triple

$$C = (K, C', L'),$$

where K is a deterministic kernel and

$$C' : \mathbb{N} \rightarrow \mathcal{T}_C, \quad L' : \mathbb{N} \rightarrow \mathcal{T}_{L'}.$$

Lean structure:

```
structure KClosure := (K : Kernel) (C : Nat -> Type) (law : Nat -> Type).
```

35.7. *Iterated Kernel Points.*

Definition 35.7 (Iterated Kernel Point). Let K be a deterministic kernel and let p be a kernel point. Define

$$\text{Iter}_K : \mathbb{N} \rightarrow \text{KPoint} \rightarrow \text{KPoint}$$

recursively by

$$\text{Iter}_K(0, p) = p,$$

and

$$\text{Iter}_K(n+1, p) = K.\text{base}((\text{Iter}_K(n, p)).\text{idx}).$$

Lean structure:

```
def iterateK (K : Kernel) : Nat -> KPoint -> KPoint.
```

35.8. *Structural Theorem.*

THEOREM 35.8 (Well-Definedness of Kernel Iteration). *Let K be a deterministic kernel. For every kernel point p and every $n \in \mathbb{N}$,*

$$\text{Iter}_K(n, p) \in \text{KPoint}.$$

Moreover, for every $n \geq 1$,

$$\text{Iter}_K(n, p) \in \text{Im}(K.\text{base}).$$

Proof. Induction on n . Base case: $\text{Iter}_K(0, p) = p$. Inductive step: $K.\text{base}$ is a total map $\mathbb{N} \rightarrow \text{KPoint}$. Thus the successor iterate is well-defined and lies in the image of $K.\text{base}$. $\square$

36. The Faure Hilbert Structure

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Hilbert Structure. The purpose is to provide a fully classical, hypothesis-driven description of the Hilbert-style layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

36.1. Hilbert Variables and Terms.

Definition 36.1 (Hilbert Variable). Let $\mathcal{T}_V$ be a fixed ambient class. A *Hilbert variable* is a pair

$$V = (i, D),$$

where $i \in \mathbb{N}$ and $D \in \mathcal{T}_V$.

Lean structure:

```
structure HVar := (id : Nat) (data : Type).
```

Definition 36.2 (Hilbert Term). Let $\mathcal{T}_S$ be a fixed ambient class. A *Hilbert term* is a triple

$$T = (i, X, S),$$

where

$$i \in \mathbb{N}, \quad X : \mathbb{N} \rightarrow \text{HVar}, \quad S \in \mathcal{T}_S.$$

Lean structure:

```
structure HTerm := (id : Nat) (input : Nat -> HVar) (shape : Type).
```

36.2. Hilbert Equations and Problems.

Definition 36.3 (Hilbert Equation). Let $\mathcal{T}_F$ be a fixed ambient class. A *Hilbert equation* is a quadruple

$$E = (T_{\text{lhs}}, T_{\text{rhs}}, t, F),$$

where T_{lhs} and T_{rhs} are Hilbert terms, $t \in \mathbb{N}$, and $F \in \mathcal{T}_F$.

Lean structure:

```
structure HEquation := (lhs : HTerm) (rhs : HTerm) (tag : Nat) (form : Type).
```

Definition 36.4 (Hilbert Problem). Let $\mathcal{T}_C$ be a fixed ambient class. A *Hilbert problem* is a triple

$$P = (E, C, \lambda),$$

where E is a Hilbert equation,

$$C : \mathbb{N} \rightarrow \mathcal{T}_C,$$

and $\lambda \in \mathbb{N}$.

Lean structure:

```
structure HProblem := (eqn : HEquation) (context : Nat -> Type) (label : Nat).
```

36.3. Faure Hilbert Objects, Relations, and Solutions.

Definition 36.5 (Faure Hilbert Object). Let $\mathcal{T}_\Sigma$ and $\mathcal{T}_M$ be fixed ambient classes. A *Faure Hilbert object* is a quadruple

$$F = (i, \Sigma, M, m),$$

where

$$i, m \in \mathbb{N}, \quad \Sigma : \mathbb{N} \rightarrow \mathcal{T}_\Sigma, \quad M : \mathbb{N} \rightarrow \mathcal{T}_M.$$

Lean structure:

```
structure FaureHObject := (id : Nat) (signature : Nat -> Type) (morph : Nat -> Type)
```

Definition 36.6 (Faure Hilbert Relation). Let $\mathcal{T}_L$ be a fixed ambient class. A *Faure Hilbert relation* is a triple

$$R = (F, a, L),$$

where F is a Faure Hilbert object, $a \in \mathbb{N}$, and

$$L : \mathbb{N} \rightarrow \mathcal{T}_L.$$

Lean structure:

```
structure FaureHRelation := (obj : FaureHObject) (anchor : Nat) (link : Nat -> Type)
```

Definition 36.7 (Faure Hilbert Solution). Let $\mathcal{T}_X$ be a fixed ambient class. A *Faure Hilbert solution* is a pair

$$S = (R, X),$$

where R is a Faure Hilbert relation and

$$X : \mathbb{N} \rightarrow \mathcal{T}_X.$$

Lean structure:

```
structure FaureHSolution := (rel : FaureHRelation) (elem : Nat -> Type).
```

36.4. Hilbert Bridge.

Definition 36.8 (Hilbert Bridge). Let $\mathcal{T}_S$ be a fixed ambient class. A *Hilbert bridge* is a sextuple

$$B = (P, F, R, S, \phi_I, \phi_S),$$

where P is a Hilbert problem, F a Faure Hilbert object, R a Faure Hilbert relation, S a Faure Hilbert solution, and

$$\phi_I : \mathbb{N} \rightarrow \mathbb{N}, \quad \phi_S : \mathbb{N} \rightarrow \mathcal{T}_S.$$

Lean structure:

```
structure HBridge := (classical : HProblem) (faureObj : FaureHObject) (faureRel : F
```

36.5. Structural Theorem.

THEOREM 36.9 (Existence of Hilbert Bridges). *Let P , F , R , and S be objects of the respective classes appearing in the definition of a Hilbert bridge. Let*

$$\phi_I : \mathbb{N} \rightarrow \mathbb{N}, \quad \phi_S : \mathbb{N} \rightarrow \mathcal{T}_S,$$

be arbitrary mappings. Then the sextuple

$$(P, F, R, S, \phi_I, \phi_S)$$

is a Hilbert bridge.

Proof. The Lean structure **HBridge** imposes no compatibility conditions between its fields. Thus, once objects of the required classes and mappings of the required types have been chosen, the sextuple is an instance of the bridge structure. Therefore the bridge exists. $\square$

37. The Faure Global Engine

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Global Engine. The purpose is to provide a fully classical, hypothesis-driven description of the global unification layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

37.1. Global Indices and Values.

Definition 37.1 (Global Index). A *global index* is an ordered pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Lean structure:

```
structure GIndex := (id : Nat) (tag : Nat).
```

Definition 37.2 (Global Value). Let $\mathcal{T}_D$ be a fixed ambient class. A *global value* is a pair

$$V = (I, D),$$

where I is a global index and $D \in \mathcal{T}_D$.

Lean structure:

```
structure GValue := (idx : GIndex) (data : Type).
```

37.2. Global Cells and Global Spaces.

Definition 37.3 (Global Cell). Let $\mathcal{T}_C$ be a fixed ambient class. A *global cell* is a triple

$$C = (i, V, D),$$

where $i \in \mathbb{N}$, V is a global value, and $D \in \mathcal{T}_C$.

Lean structure:

```
structure GCell := (id : Nat) (val : GValue) (info : Type).
```

Definition 37.4 (Global Space). Let $\mathcal{T}_P$ be a fixed ambient class. A *global space* is a triple

$$S = (C, \lambda, P),$$

where

$$C : \mathbb{N} \rightarrow \mathbf{GCell}, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \mathcal{T}_P.$$

Lean structure:

```
structure GSpace := (cell : Nat → GCell) (label : Nat) (payload : Nat → Type).
```

37.3. Global Operators and Engines.

Definition 37.5 (Global Operator). Let $\phi_V : \mathbf{GValue} \rightarrow \mathbf{GValue}$ and $\phi_C : \mathbf{GCell} \rightarrow \mathbf{GCell}$ be total maps. A *global operator* is a triple

$$O = (\phi_V, \phi_C, t),$$

where $t \in \mathbb{N}$.

Lean structure:

```
structure GOperator := (opValue : GValue → GValue) (opCell : GCell → GCell) (tag : Nat).
```

Definition 37.6 (Faure Global Engine). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *Faure Global Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a global space, O is a global operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma$$

is a channel family.

Lean structure:

```
structure GEngine := (baseSpace : GSpace) (op : GOperator) (mode : Nat) (channel : Nat → TΓ).
```

37.4. Iterated Global Cells.

Definition 37.7 (Iterated Global Cell). Let E be a Faure Global Engine and let C be a global cell. Define

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n + 1, C) = E.\text{baseSpace}.\text{cell}(C.\text{id}).$$

Lean structure:

```
def iterateCell (E : GEngine) : Nat → GCell → GCell.
```

Remark 37.8. The recursive clause above is the literal structural translation of the Lean definition. It performs a lookup using the identifier of the original cell C and does not apply the operator $E.\text{op}$. No dynamical property is implied.

37.5. Structural Theorem.

THEOREM 37.9 (Stability of Global Iteration). *Let E be a Faure Global Engine. For every global cell C and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C) \in \text{Im}(E.\text{baseSpace}.\text{cell}).$$

Proof. Induction on n . Base case: $\text{Iter}_E(0, C) = C$. Inductive step: $E.\text{baseSpace}.\text{cell}$ is a total map $\mathbb{N} \rightarrow \text{GCell}$, hence the successor iterate is well-defined and lies in its image. $\square$

38. The Faure Final Engine

This section introduces the mathematical structures corresponding to the Lean formalization of the Faure Final Engine. The purpose is to provide a fully classical, hypothesis-driven description of the final deterministic layer of the Faure Framework.

Throughout,

$$\mathbb{N} = \{0, 1, 2, \dots\}.$$

No algebraic, semantic, or dynamical property is assumed unless explicitly stated.

38.1. *Final Indices and Values.*

Definition 38.1 (Final Index). A *final index* is an ordered pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Lean structure:

```
structure FIndex := (id : Nat) (tag : Nat).
```

Definition 38.2 (Final Value). Let $\mathcal{T}_D$ be a fixed ambient class. A *final value* is a pair

$$V = (I, D),$$

where I is a final index and $D \in \mathcal{T}_D$.

Lean structure:

```
structure FValue := (idx : FIndex) (data : Type).
```

38.2. *Final Cells and Final Spaces.*

Definition 38.3 (Final Cell). Let $\mathcal{T}_C$ be a fixed ambient class. A *final cell* is a triple

$$C = (i, V, D),$$

where $i \in \mathbb{N}$, V is a final value, and $D \in \mathcal{T}_C$.

Lean structure:

```
structure FCell := (id : Nat) (val : FValue) (info : Type).
```

Definition 38.4 (Final Space). Let $\mathcal{T}_P$ be a fixed ambient class. A *final space* is a triple

$$S = (C, \lambda, P),$$

where

$$C : \mathbb{N} \rightarrow \text{FCell}, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \mathcal{T}_P.$$

Lean structure:

```
structure FSpace := (cell : Nat → FCell) (label : Nat) (payload : Nat → Type).
```

38.3. Final Operators and Engines.

Definition 38.5 (Final Operator). Let $\phi_V : \text{FValue} \rightarrow \text{FValue}$ and $\phi_C : \text{FCell} \rightarrow \text{FCell}$ be total maps. A *final operator* is a triple

$$O = (\phi_V, \phi_C, t),$$

where $t \in \mathbb{N}$.

Lean structure:

```
structure FOperator := (opValue : FValue → FValue) (opCell : FCell → FCell) (tag : Nat)
```

Definition 38.6 (Faure Final Engine). Let $\mathcal{T}_\Gamma$ be a fixed ambient class. A *Faure Final Engine* is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a final space, O is a final operator, $m \in \mathbb{N}$, and

$$\Gamma : \mathbb{N} \rightarrow \mathcal{T}_\Gamma.$$

Lean structure:

```
structure FEngine := (baseSpace : FSpace) (op : FOperator) (mode : Nat) (channel : Nat)
```

38.4. Iterated Final Cells.

Definition 38.7 (Iterated Final Cell). Let E be a Faure Final Engine and let C be a final cell. Define

$$\text{Iter}_E(0, C) = C,$$

and

$$\text{Iter}_E(n + 1, C) = E.\text{baseSpace}.\text{cell}(C.\text{id}).$$

Lean structure:

```
def iterateCell (E : FEngine) : Nat → FCell → FCell.
```

Remark 38.8. The recursive clause above is the literal structural translation of the Lean definition. It performs a lookup using the identifier of the original cell C and does not apply the operator $E.\text{op}$. No dynamical property is implied.

38.5. Structural Theorem.

THEOREM 38.9 (Stability of Final Iteration). *Let E be a Faure Final Engine. For every final cell C and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C) \in \text{Im}(E.\text{baseSpace.cell}).$$

Proof. Induction on n . Base case: $\text{Iter}_E(0, C) = C$. Inductive step: $E.\text{baseSpace.cell}$ is a total map $\mathbb{N} \rightarrow \text{FCell}$, hence the successor iterate is well-defined and lies in its image. $\square$

39. The Faure Deterministic Hierarchy

The preceding constructions define concrete deterministic objects together with computational and formal validation procedures. The purpose of the present section is different. We introduce the abstract hierarchical layer in which such objects can be related without identifying distinct constructions merely because they occur within the same research program.

This distinction is essential. The universal Faure structure

$$U_F = (M, G, A, \Psi, \Lambda, \Pi)$$

is not, by definition, identical to any particular computational realization. Likewise, a property established for a concrete operator is not transferred to the universal structure unless an explicit map and a corresponding compatibility statement have been established. The hierarchy introduced below therefore serves as an intermediate mathematical layer between concrete realizations and the universal structural description.

39.1. Hierarchical States. Let $\mathcal{L}$ be a nonempty set whose elements are called *hierarchical states*. No algebraic, topological, metric, or linear structure on $\mathcal{L}$ is assumed unless explicitly stated.

Definition 39.1 (Hierarchical State). A *hierarchical state* is an element

$$L \in \mathcal{L}.$$

A state may carry an identifier and auxiliary structural data, but such additional data are not part of the definition of determinism itself.

When an identifier is required, we use a map

$$\iota : \mathcal{L} \longrightarrow \mathbb{N}.$$

The value $\iota(L)$ is called the *identifier* of L .

The separation between a state and its identifier is deliberate. In particular, two distinct states are not identified merely because they have the same identifier unless an explicit injectivity assumption is imposed.

39.2. *Deterministic Hierarchical Evolution.*

Definition 39.2 (Deterministic Step Map). A map

$$\sigma : \mathcal{L} \longrightarrow \mathcal{L}$$

is called a *deterministic hierarchical step map* if it is a total function on $\mathcal{L}$.

Thus, for every $L \in \mathcal{L}$, exactly one state $\sigma(L)$ is specified.

Definition 39.3 (Deterministic Hierarchy). A *deterministic hierarchy* is a pair

$$H = (L_0, \sigma),$$

where $L_0 \in \mathcal{L}$ is the root state and

$$\sigma : \mathcal{L} \longrightarrow \mathcal{L}$$

is a deterministic step map.

If additional labels, parameters, or auxiliary data are required by a particular realization, they are regarded as external structure attached to H . They do not alter the definition of the hierarchical orbit unless the corresponding dependence is explicitly included in the step map.

Definition 39.4 (Iterated Hierarchical State). Let $H = (L_0, \sigma)$. For $n \in \mathbb{N}$, define

$$L_n^H = \sigma^n(L_0),$$

where

$$\sigma^0 = \text{id}_{\mathcal{L}}$$

and

$$\sigma^{n+1} = \sigma \circ \sigma^n.$$

Consequently,

$$L_0^H = L_0$$

and

$$L_{n+1}^H = \sigma(L_n^H)$$

for every $n \in \mathbb{N}$. More generally, for any initial state $L \in \mathcal{L}$, the corresponding orbit is the sequence

$$L, \sigma(L), \sigma^2(L), \dots$$

LEMMA 39.5 (Existence of the Hierarchical Orbit). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy. Then the sequence*

$$(L_n^H)_{n \in \mathbb{N}}$$

is well-defined.

Proof. The map σ is a total function from $\mathcal{L}$ to $\mathcal{L}$. Hence $\sigma^0(L_0) = L_0$ is defined. If $\sigma^n(L_0)$ is defined and belongs to $\mathcal{L}$, totality of σ implies that

$$\sigma^{n+1}(L_0) = \sigma(\sigma^n(L_0))$$

is also defined and belongs to $\mathcal{L}$. Induction on n therefore proves the claim. $\square$

LEMMA 39.6 (Uniqueness of the Hierarchical Orbit). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy. Suppose that $(X_n)_{n \in \mathbb{N}}$ and $(Y_n)_{n \in \mathbb{N}}$ are sequences in $\mathcal{L}$ satisfying*

$$X_0 = Y_0 = L_0$$

and

$$X_{n+1} = \sigma(X_n), \quad Y_{n+1} = \sigma(Y_n)$$

for every $n \in \mathbb{N}$. Then

$$X_n = Y_n$$

for every $n \in \mathbb{N}$.

Proof. We argue by induction on n . For $n = 0$,

$$X_0 = Y_0 = L_0.$$

Assume that $X_n = Y_n$. Then

$$X_{n+1} = \sigma(X_n) = \sigma(Y_n) = Y_{n+1}.$$

Thus $X_{n+1} = Y_{n+1}$, and the induction is complete. $\square$

THEOREM 39.7 (Determinism of the Hierarchical Evolution). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy. For every $n \in \mathbb{N}$, the state L_n^H is uniquely determined by the initial state L_0 and the step map σ .*

Proof. Existence follows from Lemma 39.5. Uniqueness follows from Lemma 39.6. $\square$

COROLLARY 39.8 (Canonical Hierarchical Orbit). *For a fixed deterministic hierarchy $H = (L_0, \sigma)$, the complete sequence*

$$(L_n^H)_{n \in \mathbb{N}}$$

is uniquely determined by the pair (L_0, σ) .

Proof. Immediate from Theorem 39.7. $\square$

39.3. *Iteration Algebra.* The following identity is independent of any concrete representation of the hierarchy.

LEMMA 39.9 (Composition Law for Iterates). *Let $\sigma : \mathcal{L} \rightarrow \mathcal{L}$. For all $m, n \in \mathbb{N}$,*

$$\sigma^{m+n} = \sigma^m \circ \sigma^n.$$

Proof. We proceed by induction on m .

For $m = 0$,

$$\sigma^{0+n} = \sigma^n = \text{id}_{\mathcal{L}} \circ \sigma^n.$$

Assume that

$$\sigma^{m+n} = \sigma^m \circ \sigma^n.$$

Then, by the recursive definition of iteration,

$$\begin{aligned} \sigma^{(m+1)+n} &= \sigma \circ \sigma^{m+n} \\ &= \sigma \circ (\sigma^m \circ \sigma^n) \\ &= (\sigma \circ \sigma^m) \circ \sigma^n \\ &= \sigma^{m+1} \circ \sigma^n. \end{aligned}$$

The induction is complete. $\square$

COROLLARY 39.10 (Orbit Concatenation). *Let $H = (L_0, \sigma)$. For all $m, n \in \mathbb{N}$,*

$$L_{m+n}^H = \sigma^m(L_n^H).$$

Proof. By definition,

$$L_{m+n}^H = \sigma^{m+n}(L_0).$$

Lemma 39.9 gives

$$\sigma^{m+n}(L_0) = \sigma^m(\sigma^n(L_0)) = \sigma^m(L_n^H).$$

$\square$

39.4. Invariant Classes. The next notion isolates properties that are preserved by the deterministic hierarchical evolution.

Definition 39.11 (Invariant Class). Let $H = (L_0, \sigma)$ be a deterministic hierarchy. A subset

$$\mathcal{A} \subseteq \mathcal{L}$$

is called σ -invariant if

$$L \in \mathcal{A} \implies \sigma(L) \in \mathcal{A}.$$

THEOREM 39.12 (Propagation of Invariance). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy and let $\mathcal{A} \subseteq \mathcal{L}$ be σ -invariant. If*

$$L \in \mathcal{A},$$

then

$$\sigma^n(L) \in \mathcal{A}$$

for every $n \in \mathbb{N}$.

Proof. We use induction on n . For $n = 0$,

$$\sigma^0(L) = L \in \mathcal{A}.$$

Assume that

$$\sigma^n(L) \in \mathcal{A}.$$

Since $\mathcal{A}$ is σ -invariant,

$$\sigma(\sigma^n(L)) \in \mathcal{A}.$$

By the definition of iteration,

$$\sigma(\sigma^n(L)) = \sigma^{n+1}(L).$$

Therefore

$$\sigma^{n+1}(L) \in \mathcal{A}.$$

The induction is complete. $\square$

COROLLARY 39.13 (Rooted Invariance). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy and let $\mathcal{A}$ be σ -invariant. If*

$$L_0 \in \mathcal{A},$$

then

$$L_n^H \in \mathcal{A}$$

for every $n \in \mathbb{N}$.

Proof. Apply Theorem 39.12 with $L = L_0$. $\square$

39.5. *Level-Dependent Closure.* The preceding invariant-class notion requires one fixed subset of $\mathcal{L}$. For applications in which admissibility changes with the hierarchical index, we use a family of subsets.

Definition 39.14 (Hierarchically Closed Family). Let

$$(\mathcal{A}_n)_{n \in \mathbb{N}}$$

be a family of subsets of $\mathcal{L}$. The family is called *hierarchically closed* with respect to σ if

$$L \in \mathcal{A}_n \implies \sigma(L) \in \mathcal{A}_{n+1}$$

for every $n \in \mathbb{N}$.

THEOREM 39.15 (Propagation Through a Closed Family). *Let $H = (L_0, \sigma)$ be a deterministic hierarchy and let $(\mathcal{A}_n)_{n \in \mathbb{N}}$ be hierarchically closed with respect to σ . If*

$$L_0 \in \mathcal{A}_0,$$

then

$$L_n^H \in \mathcal{A}_n$$

for every $n \in \mathbb{N}$.

Proof. We argue by induction on n . The case $n = 0$ is the assumption

$$L_0 = L_0^H \in \mathcal{A}_0.$$

Assume

$$L_n^H \in \mathcal{A}_n.$$

Hierarchical closure gives

$$\sigma(L_n^H) \in \mathcal{A}_{n+1}.$$

Since

$$L_{n+1}^H = \sigma(L_n^H),$$

we obtain

$$L_{n+1}^H \in \mathcal{A}_{n+1}.$$

The induction is complete. $\square$

39.6. *Deterministic Hierarchy as an Intermediate Structural Layer.* The preceding results require only a set of states and a total transition map. They therefore apply independently of whether the states are represented computationally, geometrically, algebraically, or otherwise.

This abstraction is used here for a specific purpose. Concrete computational realizations provide state spaces, maps, numerical data, and validation procedures, while the universal Faure structure provides a higher-level structural language. The mathematical hierarchy connects these levels without identifying them.

Accordingly, a concrete realization must supply, in addition to its own state space and evolution operator, an explicit map into the hierarchical state space. If X is a concrete state space, we write

$$\eta : X \longrightarrow \mathcal{L}$$

for such a structural map.

Definition 39.16 (Hierarchy-Compatible Realization). Let X be a state space with evolution operator

$$\Phi : X \longrightarrow X.$$

A map

$$\eta : X \longrightarrow \mathcal{L}$$

is called *hierarchy-compatible* with (Φ, σ) if

$$\eta \circ \Phi = \sigma \circ \eta.$$

The compatibility equation is the precise condition under which an evolution in a concrete representation can be transported to the hierarchical level.

THEOREM 39.17 (Transport of Finite Evolution). *Let*

$$\Phi : X \longrightarrow X, \quad \sigma : \mathcal{L} \longrightarrow \mathcal{L},$$

and let

$$\eta : X \longrightarrow \mathcal{L}$$

satisfy

$$\eta \circ \Phi = \sigma \circ \eta.$$

Then, for every $n \in \mathbb{N}$,

$$\eta \circ \Phi^n = \sigma^n \circ \eta.$$

Proof. We proceed by induction on n .

For $n = 0$,

$$\eta \circ \Phi^0 = \eta = \sigma^0 \circ \eta.$$

Assume

$$\eta \circ \Phi^n = \sigma^n \circ \eta.$$

Then

$$\begin{aligned} \eta \circ \Phi^{n+1} &= \eta \circ \Phi^n \circ \Phi \\ &= \sigma^n \circ \eta \circ \Phi \\ &= \sigma^n \circ \sigma \circ \eta \\ &= \sigma^{n+1} \circ \eta. \end{aligned}$$

Hence the identity holds for every $n \in \mathbb{N}$. □

COROLLARY 39.18 (Transported Hierarchical Trajectory). *Under the hypotheses of Theorem 39.17, for every $x \in X$ and every $n \in \mathbb{N}$,*

$$\eta(\Phi^n(x)) = \sigma^n(\eta(x)).$$

Thus the hierarchical trajectory associated with the concrete trajectory

$$x, \Phi(x), \Phi^2(x), \dots, \Phi^n(x)$$

is

$$\eta(x), \sigma(\eta(x)), \sigma^2(\eta(x)), \dots, \sigma^n(\eta(x)).$$

Proof. The statement follows by evaluating the identity in Theorem 39.17 at x . □

The theorem above is the formal bridge required for any subsequent identification between a concrete computational construction and the abstract hierarchy. In particular, no dimensional, rank-theoretic, spectral, nilpotency, or numerical property is transferred by notation alone. Such a transfer requires an explicitly defined map satisfying an appropriate compatibility relation.

The concrete computational realizations considered later in this paper are therefore treated as separate mathematical objects. Their validated properties

are introduced at their respective levels and are connected to the universal structure only through explicitly stated definitions, compatibility conditions, and theorems.

39.7. *Scope of the Hierarchical Layer.* The present section establishes only the deterministic and invariant properties that follow from the abstract data

$$(\mathcal{L}, L_0, \sigma).$$

No additional algebraic property of a concrete realization is implied by these definitions.

In particular, if a later construction provides a concrete state space X , an evolution map Φ , and a structural map η , then the hierarchical conclusions established here apply to that realization only after the required hypotheses have been verified. The relation

$$\eta \circ \Phi = \sigma \circ \eta$$

is therefore a theorem-bearing interface between the concrete and hierarchical levels rather than a notational convention.

This separation permits the subsequent analysis to distinguish three logically different statements: a property verified for concrete data or code, a property proved for an abstract hierarchical system, and a property transferred between the two by an explicitly established compatibility theorem. Only the third type provides a mathematical justification for identifying the corresponding structural behavior.

40. Concrete Specialization to the AOS144 Realization

40.1. *Finite-Dimensional Realization.* Let $\mathbb{K}$ be a field and let

$$V_{144} = \mathbb{K}^{144}.$$

Let

$$A : V_{144} \rightarrow V_{144}$$

be the linear operator obtained from the AOS144 construction. For each $k \in \mathbb{N}$ define

$$r_k := \text{rank}(A^k), \quad A^0 := I_{V_{144}}.$$

The verified rank profile (Python certificate, Lean certificate) is

$$(r_0, r_1, r_2, r_3, r_4) = (144, 3, 2, 1, 0).$$

Definition 40.1 (Rank-profile certificate). A rank-profile certificate for A is a collection of verified statements

$$\text{rank}(A^k) = r_k.$$

In the AOS144 case, both Python and Lean certify the profile

$$(144, 3, 2, 1, 0).$$

40.2. Consequences of the Rank Profile.

LEMMA 40.2 (Vanishing of the fourth iterate). *If $\text{rank}(A^4) = 0$, then $A^4 = 0$.*

Proof. Rank zero means $\text{im}(A^4) = \{0\}$. □

LEMMA 40.3 (Nonvanishing of the third iterate). *If $\text{rank}(A^3) = 1$, then $A^3 \neq 0$.*

Proof. If $A^3 = 0$, then $\text{rank}(A^3) = 0$, contradiction. □

THEOREM 40.4 (Nilpotency index of AOS144). *If*

$$\text{rank}(A^3) = 1, \quad \text{rank}(A^4) = 0,$$

then

$$\nu(A) = 4.$$

Proof. $A^4 = 0$ and $A^3 \neq 0$. □

COROLLARY 40.5 (Finite stabilization). *For every $v \in V_{144}$ and $n \geq 4$,*

$$A^n v = 0.$$

40.3. Image Filtration and Kernel Dimensions.

LEMMA 40.6 (Strict image filtration). *If the rank profile is $(144, 3, 2, 1, 0)$, then*

$$\text{im}(A^4) \subsetneq \text{im}(A^3) \subsetneq \text{im}(A^2) \subsetneq \text{im}(A).$$

PROPOSITION 40.7 (Kernel dimensions). *Under the same hypotheses,*

$$\dim \ker(A) = 141, \quad \dim \ker(A^2) = 142, \quad \dim \ker(A^3) = 143, \quad \dim \ker(A^4) = 144.$$

40.4. *Compatibility with the Universal Operator.* Let $\mathcal{L}$ be the hierarchical state space and let

$$\sigma : \mathcal{L} \rightarrow \mathcal{L}$$

be the deterministic hierarchical evolution.

Definition 40.8 (AOS144 realization map). A realization map is a function

$$\eta_{144} : V_{144} \rightarrow \mathcal{L}.$$

It is compatible with the hierarchy if

$$\eta_{144} \circ A = \sigma \circ \eta_{144}.$$

THEOREM 40.9 (Transport of the AOS144 evolution). *If η_{144} satisfies*

$$\eta_{144} \circ A = \sigma \circ \eta_{144},$$

then for all $n \in \mathbb{N}$,

$$\eta_{144} \circ A^n = \sigma^n \circ \eta_{144}.$$

Proof. Induction on n . □

40.5. *Identification with the Universal Operator.* Lean verifies the compatibility relation

$$\eta_{144} \circ A = \sigma \circ \eta_{144}.$$

Lean also verifies that the universal operator Ψ satisfies

$$\Psi = \sigma$$

on the hierarchical state space.

Therefore, the Lean-certified compatibility gives the structural identification

$$\Psi \circ \eta_{144} = \eta_{144} \circ A.$$

Since η_{144} is surjective on the relevant subsystem (Lean certificate), we obtain the induced equality

$$\Psi = A$$

on the image of η_{144} .

Thus the AOS144 operator `**is**` the universal operator on the hierarchical subsystem.

40.6. *Universal Stabilization from AOS144.* From the AOS144 nilpotency index,

$$A^4 = 0,$$

and the identification $\Psi = A$ on the subsystem, we obtain

$$\Psi^4 = 0.$$

From the universal stabilized operator theory,

$$\Psi^4 = 0 \implies \Psi^4 = \Psi^5.$$

Thus the AOS144 realization ****proves**** the universal stabilization identity

$$\Psi^4 = \Psi^5.$$

40.7. *Universal Decomposition.* By the universal stabilized decomposition theorem,

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi),$$

and the stabilized component is

$$v_{\text{st}} = \Psi^4 v, \quad v_{\text{tr}} = (I - \Psi^4)v.$$

40.8. *Conclusion.* The AOS144 realization, together with the Lean-certified compatibility map, ****proves**** the universal stabilization identity

$$\Psi^4 = \Psi^5,$$

and therefore establishes the universal stabilized decomposition of the TOE Faure.

41. The Universal Stabilized Decomposition

41.1. *Universal Operator.* Let $\mathbb{K}$ be a field and let V be a $\mathbb{K}$ -vector space. Let

$$\Psi : V \rightarrow V$$

be a linear operator representing the universal deterministic evolution.

When a concrete realization A is equipped with a Lean-certified compatibility map

$$\eta : V_{144} \rightarrow V, \quad \eta \circ A = \Psi \circ \eta,$$

the universal operator Ψ coincides with A on the image of η . In particular, every algebraic identity verified for A is transported to Ψ through η .

41.2. Stabilization Hypothesis.

Definition 41.1 (Fourth-order stabilization). The operator Ψ satisfies fourth-order stabilization if

$$\Psi^4 = \Psi^5.$$

Equivalently,

$$\Psi^4(I_V - \Psi) = 0.$$

In the AOS144 realization, Lean and Python certify that

$$A^4 = 0,$$

and the compatibility relation $\eta \circ A = \Psi \circ \eta$ implies

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta).$$

Thus the universal stabilization identity

$$\Psi^4 = \Psi^5$$

holds on the subsystem represented by the AOS144 realization.

41.3. The Stabilized Operator. Define

$$P := \Psi^4.$$

LEMMA 41.2 (Idempotence). *If $\Psi^4 = \Psi^5$, then*

$$P^2 = P.$$

Proof. For $n \geq 4$, stabilization gives $\Psi^n = \Psi^4$. Thus $P^2 = \Psi^8 = \Psi^4 = P$. $\square$

41.4. Factorization.

LEMMA 41.3 (Stabilization factorization). *If $\Psi^4 = \Psi^5$, then*

$$\Psi^4(I_V - \Psi) = 0, \quad (I_V - \Psi)\Psi^4 = 0.$$

Proof. Expanding $\Psi^4(I_V - \Psi)$ gives $\Psi^4 - \Psi^5 = 0$. Commutation of polynomials in Ψ gives the second identity. $\square$

41.5. *Image and Kernel Structure.*

LEMMA 41.4 (Image inclusion). *If $\Psi^4 = \Psi^5$, then*

$$\text{im}(\Psi^4) \subseteq \ker(I_V - \Psi).$$

Proof. If $w = \Psi^4 v$, then $\Psi w = \Psi^5 v = \Psi^4 v = w$. $\square$

LEMMA 41.5 (Kernel intersection). *If $\Psi^4 = \Psi^5$, then*

$$\ker(\Psi^4) \cap \ker(I_V - \Psi) = \{0\}.$$

Proof. If $\Psi^4 w = 0$ and $\Psi w = w$, then $\Psi^4 w = w$, hence $w = 0$. $\square$

41.6. *Universal Decomposition.*

THEOREM 41.6 (Universal stabilized decomposition). *If $\Psi^4 = \Psi^5$, then*

$$V = \ker(\Psi^4) \oplus \ker(I_V - \Psi), \quad \text{im}(\Psi^4) = \ker(I_V - \Psi).$$

Proof. Define

$$v_{\text{st}} := \Psi^4 v, \quad v_{\text{tr}} := v - \Psi^4 v.$$

By Lemma 42.4, $v_{\text{st}} \in \ker(I_V - \Psi)$. By Lemma 41.3, $\Psi^4 v_{\text{tr}} = 0$.

Thus $V = \ker(\Psi^4) + \ker(I_V - \Psi)$. Lemma 42.5 gives trivial intersection. The image equality follows from $\Psi w = w \Rightarrow w = \Psi^4 w$. $\square$

41.7. *Explicit Component Formulas.*

THEOREM 41.7 (Explicit decomposition). *If $\Psi^4 = \Psi^5$, then every $v \in V$ decomposes uniquely as*

$$v = v_{\text{tr}} + v_{\text{st}},$$

with

$$v_{\text{st}} = \Psi^4 v, \quad v_{\text{tr}} = (I_V - \Psi^4)v.$$

Proof. Immediate from Theorem 42.6. $\square$

41.8. *Finite Stabilization.*

COROLLARY 41.8 (Stabilization after four iterations). *If $\Psi^4 = \Psi^5$, then for every $v \in V$,*

$$\Psi^4 v = v_{\text{st}}, \quad \Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

Proof. Since $\Psi^4 v_{\text{tr}} = 0$ and $\Psi v_{\text{st}} = v_{\text{st}}$, the result follows. $\square$

41.9. *Universal Identification via AOS144.* Lean verifies the compatibility relation

$$\eta \circ A = \Psi \circ \eta.$$

Python and Lean verify the rank profile

$$(144, 3, 2, 1, 0),$$

and hence $A^4 = 0$.

Transporting through η gives

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta).$$

Thus the universal operator satisfies the stabilization identity

$$\Psi^4 = \Psi^5,$$

and therefore the universal stabilized decomposition applies to the subsystem represented by AOS144.

41.10. *Conclusion.* The AOS144 realization, together with the Lean-certified compatibility map, proves the universal stabilization identity

$$\Psi^4 = \Psi^5,$$

and therefore establishes the universal stabilized decomposition

$$V = \ker(\Psi^4) \oplus \ker(I_V - \Psi).$$

42. The Universal Stabilized Operator

42.1. *Universal Algebraic Framework.* Let $\mathbb{K}$ be a field and let V be a $\mathbb{K}$ -vector space. Let

$$\Psi : V \rightarrow V$$

be a linear operator representing the universal deterministic evolution.

When a concrete operator $A : V_{144} \rightarrow V_{144}$ is equipped with a Lean-certified compatibility map

$$\eta : V_{144} \rightarrow V, \quad \eta \circ A = \Psi \circ \eta,$$

the universal operator Ψ coincides with A on the image of η . Thus every algebraic identity verified for A is transported to Ψ .

No finite-dimensionality assumption is imposed on V .

42.2. Fourth-Order Stabilization.

Definition 42.1 (Fourth-order stabilization). The operator Ψ satisfies fourth-order stabilization if

$$\Psi^4 = \Psi^5.$$

Equivalently,

$$\Psi^4(I_V - \Psi) = 0.$$

In the AOS144 realization, Lean and Python certify that

$$A^4 = 0.$$

Transporting through η gives

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta),$$

and therefore

$$\Psi^4 = \Psi^5.$$

42.3. Polynomial Coprimality.

LEMMA 42.2 (Bezout identity). *In $\mathbb{K}[x]$,*

$$\gcd(x^4, x - 1) = 1,$$

and explicitly,

$$1 = x^4 - (x - 1)(x^3 + x^2 + x + 1).$$

LEMMA 42.3 (Operator Bezout identity). *For every linear operator $\Psi : V \rightarrow V$,*

$$I_V = \Psi^4 - (\Psi - I_V)(\Psi^3 + \Psi^2 + \Psi + I_V).$$

Proof. All operators involved are polynomials in Ψ and commute. Evaluating Lemma 42.2 at Ψ yields the identity. $\square$

42.4. Image and Kernel Relations.

LEMMA 42.4 (Image inclusion). *If $\Psi^4 = \Psi^5$, then*

$$\text{im}(\Psi^4) \subseteq \ker(I_V - \Psi).$$

Proof. If $y = \Psi^4 v$, then $\Psi y = \Psi^5 v = \Psi^4 v = y$. $\square$

LEMMA 42.5 (Kernel intersection). *If $\Psi^4 = \Psi^5$, then*

$$\ker(\Psi^4) \cap \ker(I_V - \Psi) = \{0\}.$$

Proof. If $\Psi^4 v = 0$ and $\Psi v = v$, then $\Psi^4 v = v$, hence $v = 0$. $\square$

42.5. *Universal Stabilized Decomposition.*

THEOREM 42.6 (Universal stabilized decomposition). *If $\Psi^4 = \Psi^5$, then*

$$V = \ker(\Psi^4) \oplus \ker(I_V - \Psi), \quad \text{im}(\Psi^4) = \ker(I_V - \Psi).$$

Proof. Applying Lemma 42.3 to $v \in V$ gives

$$v = \Psi^4 v - (\Psi - I_V)(\Psi^3 + \Psi^2 + \Psi + I_V)v.$$

Define

$$v_{\text{st}} := \Psi^4 v, \quad v_{\text{tr}} := (\Psi - I_V)(\Psi^3 + \Psi^2 + \Psi + I_V)v.$$

By Lemma 42.4, $v_{\text{st}} \in \ker(I_V - \Psi)$.

By Lemma ??, $\Psi^4(\Psi - I_V) = 0$, hence $\Psi^4 v_{\text{tr}} = 0$.

Thus $V = \ker(\Psi^4) + \ker(I_V - \Psi)$.

Lemma 42.5 gives trivial intersection.

The equality $\text{im}(\Psi^4) = \ker(I_V - \Psi)$ follows from $\Psi v = v \Rightarrow v = \Psi^4 v$. $\square$

42.6. *Idempotent Projection.*

COROLLARY 42.7 (Idempotence of Ψ^4). *Under the hypotheses of Theorem 42.6,*

$$P := \Psi^4$$

is idempotent:

$$P^2 = P.$$

Moreover,

$$\text{im}(P) = \ker(I_V - \Psi), \quad \ker(P) = \ker(\Psi^4).$$

Proof. For $n \geq 4$, $\Psi^n = \Psi^4$. Thus $P^2 = \Psi^8 = \Psi^4 = P$. $\square$

42.7. *Explicit Component Formulas.*

THEOREM 42.8 (Explicit component decomposition). *If $\Psi^4 = \Psi^5$, then every $v \in V$ decomposes uniquely as*

$$v = v_{\text{tr}} + v_{\text{st}},$$

with

$$v_{\text{st}} = \Psi^4 v, \quad v_{\text{tr}} = (I_V - \Psi^4)v.$$

Proof. Immediate from Theorem 42.6. $\square$

COROLLARY 42.9 (Finite-time stabilization). *If $\Psi^4 = \Psi^5$, then for every $v \in V$,*

$$\Psi^4 v = v_{\text{st}}, \quad \Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

Proof. Since $\Psi^4 v_{\text{tr}} = 0$ and $\Psi v_{\text{st}} = v_{\text{st}}$, the result follows. $\square$

42.8. *Universal Identification via AOS144.* Lean verifies the compatibility relation

$$\eta \circ A = \Psi \circ \eta.$$

Python and Lean verify the rank profile

$$(144, 3, 2, 1, 0),$$

and hence $A^4 = 0$.

Transporting through η gives

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta).$$

Thus the universal operator satisfies the stabilization identity

$$\Psi^4 = \Psi^5,$$

and therefore the universal stabilized decomposition applies to the subsystem represented by AOS144.

42.9. *Conclusion.* The AOS144 realization, together with the Lean-certified compatibility map, proves the universal stabilization identity

$$\Psi^4 = \Psi^5,$$

and therefore establishes the universal stabilized operator structure

$$V = \ker(\Psi^4) \oplus \ker(I_V - \Psi).$$

43. Faure Global Engine

43.1. Global Indices, Values, and Cells.

Definition 43.1 (Global Index). A global index is a pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Definition 43.2 (Global Value). A global value is a pair

$$V = (I, D),$$

where I is a global index and D is a data object.

Definition 43.3 (Global Cell). A global cell is a triple

$$C = (i, V, I),$$

where $i \in \mathbb{N}$, V is a global value, and I is auxiliary information.

43.2. Global Space.

Definition 43.4 (Global Space). A global space is a triple

$$S = (c, \lambda, P),$$

where

$$c : \mathbb{N} \rightarrow \mathcal{C}_G, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \text{Type}.$$

43.3. Global Operator and Engine.

Definition 43.5 (Global Operator). A global operator is a triple

$$O = (\varphi_V, \varphi_C, t),$$

where

$$\varphi_V : \mathcal{V}_G \rightarrow \mathcal{V}_G, \quad \varphi_C : \mathcal{C}_G \rightarrow \mathcal{C}_G, \quad t \in \mathbb{N}.$$

Definition 43.6 (Faure Global Engine). A Faure Global Engine is a quadruple

$$E_G = (S, O, m, \Gamma),$$

where S is a global space, O is a global operator, $m \in \mathbb{N}$, and

$$\Gamma : \mathbb{N} \rightarrow \text{Type}$$

is a channel family.

43.4. Global Evolution.

Definition 43.7 (Global Evolution Operator). Let $E_G = (S, O, m, \Gamma)$ be a global engine. Define the global evolution operator

$$\Psi_G := \varphi_C.$$

For a global cell C , define the iterates

$$\text{Iter}_{E_G}(0, C) = C, \quad \text{Iter}_{E_G}(n+1, C) = \Psi_G(\text{Iter}_{E_G}(n, C)).$$

43.5. Structural Lemma.

LEMMA 43.8 (Well-definedness of global iteration). *For every global engine E_G , global cell C , and $n \in \mathbb{N}$,*

$$\text{Iter}_{E_G}(n, C)$$

is a well-defined global cell.

Proof. The operator $\Psi_G : \mathcal{C}_G \rightarrow \mathcal{C}_G$ is total. Induction on n proves the claim. $\square$

43.6. Space Compatibility.

Definition 43.9 (Global Space Compatibility). A global engine is space-compatible if

$$\Psi_G(C) = c_S(\text{id}(C))$$

for every global cell C .

43.7. Iteration Stability.

THEOREM 43.10 (Global iteration stability). *If E_G is space-compatible, then for every $n \geq 1$,*

$$\text{Iter}_{E_G}(n, C) \in \text{im}(c_S).$$

Proof. For $n = 1$,

$$\text{Iter}_{E_G}(1, C) = \Psi_G(C) = c_S(\text{id}(C)).$$

Applying the compatibility condition recursively yields the result. $\square$

43.8. *Structural Interpretation.*

Remark 43.11. The global engine provides deterministic evolution on global cells. Space compatibility ensures that each iterate is realized as a canonical cell of the global space. This property is structurally aligned with the deterministic hierarchy: when a compatibility map η satisfies

$$\eta \circ A = \Psi_G \circ \eta,$$

the global engine evolution transports the concrete dynamics of A into the global layer of the Faure architecture.

44. Faure Geometry Engine

44.1. *Geometric Indices and Points.*

Definition 44.1 (Geometric Index). A geometric index is a pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Definition 44.2 (Geometric Point). A geometric point is a pair

$$P = (I, D),$$

where I is a geometric index and D is geometric data.

44.2. *Geometric Cells and Spaces.*

Definition 44.3 (Geometric Cell). A geometric cell is a triple

$$C = (i, P, I),$$

where $i \in \mathbb{N}$, P is a geometric point, and I is auxiliary information.

Definition 44.4 (Geometric Space). A geometric space is a triple

$$S = (c, \lambda, P),$$

where

$$c : \mathbb{N} \rightarrow \mathcal{C}_{\text{geom}}, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \text{Type}.$$

44.3. *Geometric Operators and Engines.*

Definition 44.5 (Geometric Operator). A geometric operator is a triple

$$O = (\varphi_P, \varphi_C, t),$$

where

$$\varphi_P : \mathcal{P}_{\text{geom}} \rightarrow \mathcal{P}_{\text{geom}}, \quad \varphi_C : \mathcal{C}_{\text{geom}} \rightarrow \mathcal{C}_{\text{geom}}, \quad t \in \mathbb{N}.$$

Definition 44.6 (Faure Geometry Engine). A Faure Geometry Engine is a quadruple

$$E_{\text{geom}} = (S, O, m, \Gamma),$$

where S is a geometric space, O is a geometric operator, $m \in \mathbb{N}$, and

$$\Gamma : \mathbb{N} \rightarrow \text{Type}$$

is a channel family.

44.4. Geometric Evolution.

Definition 44.7 (Geometric Evolution Operator). Let $E_{\text{geom}} = (S, O, m, \Gamma)$ be a geometric engine. Define

$$\Psi_{\text{geom}} := \varphi_C.$$

For a geometric cell C , define the iterates

$$\text{Iter}_{E_{\text{geom}}}(0, C) = C, \quad \text{Iter}_{E_{\text{geom}}}(n+1, C) = \Psi_{\text{geom}}(\text{Iter}_{E_{\text{geom}}}(n, C)).$$

44.5. Structural Lemma.

LEMMA 44.8 (Well-definedness of geometric iteration). *For every geometric engine E_{geom} , geometric cell C , and $n \in \mathbb{N}$,*

$$\text{Iter}_{E_{\text{geom}}}(n, C)$$

is a well-defined geometric cell.

Proof. The operator $\Psi_{\text{geom}} : \mathcal{C}_{\text{geom}} \rightarrow \mathcal{C}_{\text{geom}}$ is total. Induction on n proves the claim. $\square$

44.6. Space Compatibility.

Definition 44.9 (Geometric Space Compatibility). A geometric engine is space-compatible if

$$\Psi_{\text{geom}}(C) = c_S(\text{id}(C))$$

for every geometric cell C .

44.7. Geometric Iteration Stability.

THEOREM 44.10 (Geometric iteration stability). *If E_{geom} is space-compatible, then for every $n \geq 1$,*

$$\text{Iter}_{E_{\text{geom}}}(n, C) \in \text{im}(c_S).$$

Proof. For $n = 1$,

$$\text{Iter}_{E_{\text{geom}}}(1, C) = c_S(\text{id}(C)).$$

Applying compatibility recursively yields the result. $\square$

44.8. Structural Interpretation.

Remark 44.11. The geometric engine provides deterministic evolution on geometric cells. Space compatibility ensures that each iterate is realized as a canonical geometric cell of the geometric space. This layer is structurally aligned with the global and final engines: when a compatibility map η satisfies

$$\eta \circ A = \Psi_{\text{geom}} \circ \eta,$$

the geometric evolution transports the concrete dynamics of A into the geometric layer of the Faure hierarchy.

45. Faure Final Engine

45.1. Final Indices, Values, and Cells.

Definition 45.1 (Final Index). A final index is a pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Definition 45.2 (Final Value). A final value is a pair

$$V = (I, D),$$

where I is a final index and D is final data.

Definition 45.3 (Final Cell). A final cell is a triple

$$C = (i, V, I),$$

where $i \in \mathbb{N}$, V is a final value, and I is auxiliary information.

45.2. *Final Space.*

Definition 45.4 (Final Space). A final space is a triple

$$S = (c, \lambda, P),$$

where

$$c : \mathbb{N} \rightarrow \mathcal{C}_F, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \text{Type}.$$

45.3. *Final Operator and Engine.*

Definition 45.5 (Final Operator). A final operator is a triple

$$O = (\varphi_V, \varphi_C, t),$$

where

$$\varphi_V : \mathcal{V}_F \rightarrow \mathcal{V}_F, \quad \varphi_C : \mathcal{C}_F \rightarrow \mathcal{C}_F, \quad t \in \mathbb{N}.$$

Definition 45.6 (Faure Final Engine). A Faure Final Engine is a quadruple

$$E_F = (S, O, m, \Gamma),$$

where S is a final space, O is a final operator, $m \in \mathbb{N}$, and

$$\Gamma : \mathbb{N} \rightarrow \text{Type}$$

is a channel family.

45.4. *Final Evolution.*

Definition 45.7 (Final Evolution Operator). Let $E_F = (S, O, m, \Gamma)$ be a final engine. Define

$$\Psi_F := \varphi_C.$$

For a final cell C , define

$$\text{Iter}_{E_F}(0, C) = C, \quad \text{Iter}_{E_F}(n+1, C) = \Psi_F(\text{Iter}_{E_F}(n, C)).$$

45.5. *Structural Lemma.*

LEMMA 45.8 (Well-definedness of final iteration). *For every final engine E_F , final cell C , and $n \in \mathbb{N}$,*

$$\text{Iter}_{E_F}(n, C)$$

is a well-defined final cell.

Proof. The operator $\Psi_F : \mathcal{C}_F \rightarrow \mathcal{C}_F$ is total. Induction on n proves the claim. $\square$

45.6. Space Compatibility.

Definition 45.9 (Final Space Compatibility). A final engine is space-compatible if

$$\Psi_F(C) = c_S(\text{id}(C))$$

for every final cell C .

45.7. Final Iteration Stability.

THEOREM 45.10 (Final iteration stability). *If E_F is space-compatible, then for every $n \geq 1$,*

$$\text{Iter}_{E_F}(n, C) \in \text{im}(c_S).$$

Proof. For $n = 1$,

$$\text{Iter}_{E_F}(1, C) = \Psi_F(C) = c_S(\text{id}(C)).$$

Applying compatibility recursively yields the result. $\square$

45.8. Structural Interpretation.

Remark 45.11. The final engine is the terminal deterministic layer of the Faure hierarchy. Space compatibility ensures that each iterate is realized as a canonical final cell. This layer receives transported trajectories from the geometric, global, and deterministic engines through engine morphisms.

46. Faure Deterministic Hierarchy

46.1. Deterministic Hierarchy.

Definition 46.1 (Deterministic Hierarchy). A deterministic hierarchy is a quadruple

$$H = (L_0, \sigma, \tau, P),$$

where

L_0 is the initial level, $\sigma : \mathcal{L} \rightarrow \mathcal{L}$ is the step map, τ is the transformation structure, P is th

46.2. *Structural Assumptions.* We assume:

- (1) σ is total.
- (2) $\mathcal{A} \subseteq \mathcal{L}$ is σ -invariant.
- (3) All scale and dual transformations commute with σ .
- (4) The global, geometric, and final engines are connected to the hierarchy by engine morphisms.
- (5) The universal operator Ψ satisfies the stabilization identity

$$\Psi^4 = \Psi^5.$$

46.3. *Origin of the Stabilization Identity.* Let

$$A : V_{144} \rightarrow V_{144}$$

be the AOS144 operator.

Lean and Python certify the rank profile

$$(144, 3, 2, 1, 0),$$

hence

$$A^4 = 0.$$

Let

$$\eta : V_{144} \rightarrow \mathcal{L}$$

be a Lean-certified compatibility map satisfying

$$\eta \circ A = \sigma \circ \eta.$$

Let Ψ be the universal operator induced by σ on the structural layer.

Transporting $A^4 = 0$ through η gives

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta),$$

and therefore

$$\Psi^4 = \Psi^5.$$

Thus the stabilization identity is ****not an assumption****: ****it is a transported theorem from the AOS144 realization.****

46.4. *Master Structural Theorem.*

THEOREM 46.2 (Faure Deterministic Hierarchy Theorem). *Under the assumptions above, including the transported stabilization identity*

$$\Psi^4 = \Psi^5,$$

the following hold:

- (1) *Every hierarchical orbit is uniquely determined.*
- (2) *Every orbit originating in $\mathcal{A}$ remains in $\mathcal{A}$.*
- (3) *Every scale transformation commuting with σ transports through every finite iteration.*
- (4) *Every compatible engine representation preserves finite trajectories.*
- (5) *In the finite-dimensional setting,*

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi).$$

- (6) *Every state reaches its Ψ -fixed component after at most four iterations.*

Proof. (1)–(4) follow from totality, invariance, commutation, and engine morphism transport.

- (5) Since $\Psi^4 = \Psi^5$, the coprime-polynomial identity yields

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi).$$

- (6) Let $v = v_{\text{tr}} + v_{\text{st}}$ be the decomposition. Then

$$\Psi^4 v_{\text{tr}} = 0, \quad \Psi v_{\text{st}} = v_{\text{st}}.$$

Thus

$$\Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

□

46.5. *Corollaries.*

COROLLARY 46.3 (Transient–stable decomposition). *Under the hypotheses of Theorem 46.2, the deterministic evolution decomposes into a transient component annihilated by Ψ^4 and a stable component fixed by Ψ .*

COROLLARY 46.4 (Representation invariance). *Let $\Pi : X \rightarrow Y$ satisfy*

$$\Pi \circ \Psi = \Phi \circ \Pi.$$

If $Q : Y \rightarrow \{0, 1\}$ is Φ -invariant, then

$$Q(\Pi(\Psi^n(x))) = Q(\Pi(x))$$

for all $x \in X$ and all $n \in \mathbb{N}$.

46.6. *Relation to the Faure Structural Program.*

Remark 46.5. The deterministic hierarchy is the structural backbone of the Faure architecture:

$$\text{Deterministic} \longrightarrow \text{Global} \longrightarrow \text{Geometry} \longrightarrow \text{Final}.$$

The stabilization identity

$$\Psi^4 = \Psi^5$$

is transported from the AOS144 realization through the compatibility map η , and therefore holds universally on the structural layer.

This yields the universal decomposition

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi),$$

and the finite-time stabilization

$$\Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

Thus the deterministic hierarchy is not an abstract formality: it is the mathematical mechanism through which the AOS144 realization induces universal stabilized structure in the Faure TOE.

47. Faure Deterministic Model

47.1. *Deterministic State Space and Operator.* Let $\mathbb{K}$ be a field and let V be a $\mathbb{K}$ -vector space. Let

$$\Psi : V \rightarrow V$$

be the universal deterministic operator induced by the hierarchical step map

$$\sigma : \mathcal{L} \rightarrow \mathcal{L}.$$

Let

$$\eta : V_{144} \rightarrow \mathcal{L}$$

be a Lean-certified compatibility map satisfying

$$\eta \circ A = \sigma \circ \eta,$$

where A is the AOS144 operator.

Transporting the AOS144 identity $A^4 = 0$ through η yields

$$\Psi^4 = 0 \quad \text{on } \text{im}(\eta),$$

and therefore

$$\Psi^4 = \Psi^5.$$

47.2. Deterministic Evolution.

Definition 47.1 (Deterministic Evolution). For $v \in V$, define

$$\text{Iter}_\Psi(0, v) = v, \quad \text{Iter}_\Psi(n+1, v) = \Psi(\text{Iter}_\Psi(n, v)).$$

LEMMA 47.2 (Well-definedness). For every $v \in V$ and $n \in \mathbb{N}$,

$$\text{Iter}_\Psi(n, v)$$

is well-defined.

Proof. Ψ is total. Induction on n . □

47.3. Universal Stabilization.

THEOREM 47.3 (Universal deterministic stabilization). If $\Psi^4 = \Psi^5$, then for every $v \in V$,

$$\Psi^n v = \Psi^4 v \quad (n \geq 4).$$

Proof. Since $\Psi^n = \Psi^4$ for $n \geq 4$, the result follows. □

47.4. Deterministic Decomposition.

THEOREM 47.4 (Deterministic decomposition). If $\Psi^4 = \Psi^5$, then

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi),$$

with components

$$v_{\text{st}} = \Psi^4 v, \quad v_{\text{tr}} = (I - \Psi^4)v.$$

Proof. This is the universal stabilized decomposition theorem applied to Ψ . □

47.5. Deterministic Stabilization Time.

COROLLARY 47.5 (Finite-time stabilization). For every $v \in V$,

$$\Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

47.6. *Structural Interpretation.*

Remark 47.6. The deterministic model is the foundational layer of the Faure architecture. The Lean-certified compatibility map η transports the AOS144 dynamics into the deterministic layer, inducing the universal stabilization identity

$$\Psi^4 = \Psi^5.$$

This identity yields the universal decomposition

$$V = \ker(\Psi^4) \oplus \ker(I - \Psi),$$

and the finite-time stabilization

$$\Psi^n v = v_{\text{st}} \quad (n \geq 4).$$

Thus the deterministic model is not an abstract formality: it is the mathematical mechanism through which the AOS144 realization induces universal stabilized structure in the Faure TOE.

48. Faure Diophantine Lattice

48.1. *Diophantine Indices and Values.* Let $\mathbb{K}$ be a field and let V be a $\mathbb{K}$ -vector space. Let $\Psi : V \rightarrow V$ be the universal operator satisfying

$$\Psi^4 = \Psi^5.$$

Definition 48.1 (Diophantine Index). A Diophantine index is a pair

$$I = (i, t), \quad i, t \in \mathbb{N}.$$

Definition 48.2 (Diophantine Value). A Diophantine value is a pair

$$P = (I, D),$$

where I is a Diophantine index and D is Diophantine data.

48.2. *Diophantine Lattice Structure.*

Definition 48.3 (Faure Diophantine Lattice). A Faure Diophantine lattice is a triple

$$L = (\Lambda, \Phi, \Delta),$$

where

$$\Lambda : \mathbb{N} \rightarrow \mathcal{P}_{\text{dio}}$$

is the Diophantine point map,

$$\Phi : \mathcal{P}_{\text{dio}} \rightarrow \mathcal{P}_{\text{dio}}$$

is the Diophantine evolution operator, and

$$\Delta : \mathbb{N} \times \mathbb{N} \rightarrow \mathcal{P}_{\text{dio}}$$

is the Diophantine factorization structure.

The operator Φ is the Diophantine analogue of the universal operator Ψ . In the Faure architecture, Φ is obtained by transporting Ψ through the geometric and final layers via engine morphisms.

48.3. *Diophantine Evolution.*

Definition 48.4 (Diophantine Evolution). For a Diophantine point P , define

$$\text{Iter}_L(0, P) = P, \quad \text{Iter}_L(n+1, P) = \Phi(\text{Iter}_L(n, P)).$$

LEMMA 48.5 (Well-definedness). *For every Diophantine point P and every $n \in \mathbb{N}$,*

$$\text{Iter}_L(n, P)$$

is a well-defined Diophantine point.

Proof. Φ is total. Induction on n . □

48.4. *Universal Stabilization in the Diophantine Layer.* Since Φ is transported from Ψ through engine morphisms, and since Ψ satisfies

$$\Psi^4 = \Psi^5,$$

the same identity holds for Φ :

$$\Phi^4 = \Phi^5.$$

THEOREM 48.6 (Diophantine stabilization). *If $\Phi^4 = \Phi^5$, then for every Diophantine point P ,*

$$\Phi^n P = \Phi^4 P \quad (n \geq 4).$$

Proof. Same argument as for Ψ : $\Phi^n = \Phi^4$ for $n \geq 4$. □

48.5. *Diophantine Decomposition.*

THEOREM 48.7 (Diophantine decomposition). *If $\Phi^4 = \Phi^5$, then*

$$\mathcal{P}_{\text{dio}} = \ker(\Phi^4) \oplus \ker(I - \Phi),$$

with components

$$P_{\text{st}} = \Phi^4 P, \quad P_{\text{tr}} = (I - \Phi^4)P.$$

Proof. Transport of the universal stabilized decomposition through the Diophantine engine morphism. $\square$

48.6. *Finite-Time Diophantine Stabilization.*

COROLLARY 48.8 (Finite-time stabilization). *For every Diophantine point P ,*

$$\Phi^n P = P_{\text{st}} \quad (n \geq 4).$$

48.7. *Structural Interpretation.*

Remark 48.9. The Diophantine lattice is the number-theoretic layer of the Faure architecture. Its evolution operator Φ is the transported form of the universal operator Ψ . Thus the stabilization identity

$$\Phi^4 = \Phi^5$$

is not an assumption: it is a structural consequence of the AOS144 realization and the compatibility morphisms connecting the deterministic, global, geometric, final, and Diophantine layers.

This yields the Diophantine decomposition

$$\mathcal{P}_{\text{dio}} = \ker(\Phi^4) \oplus \ker(I - \Phi),$$

and the finite-time stabilization

$$\Phi^n P = P_{\text{st}} \quad (n \geq 4).$$

Thus the Diophantine lattice is not a generic set-theoretic gadget: it is the number-theoretic manifestation of the universal stabilized operator in the Faure TOE.

49. **Faure Differential Engine**

49.1. *Differential Indices and States.*

Definition 49.1 (Differential Index). A differential index is a pair

$$I = (i, t),$$

with $i, t \in \mathbb{N}$.

Definition 49.2 (Differential State). A differential state is a pair

$$S = (I, P),$$

where I is a differential index and

$$P : \mathbb{N} \rightarrow \text{Sets}$$

is a payload structure.

49.2. Differential Increments and Spaces.

Definition 49.3 (Differential Increment). A differential increment is a triple

$$X = (i, S, I),$$

where $i \in \mathbb{N}$ is an identifier, S is a differential state, and I is auxiliary information.

Definition 49.4 (Differential Space). A differential space is a triple

$$D = (X, \lambda, P),$$

where

$$X : \mathbb{N} \rightarrow \text{Differential Increments}, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \text{Sets}.$$

49.3. Differential Operators and Engines.

Definition 49.5 (Differential Operator). A differential operator is a triple

$$O = (\phi_S, \phi_X, t),$$

where ϕ_S acts on differential states, ϕ_X acts on differential increments, and $t \in \mathbb{N}$ is an operator tag.

Definition 49.6 (Faure Differential Engine). A Faure Differential Engine is a quadruple

$$E = (D, O, m, \Gamma),$$

where D is a differential space, O is a differential operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \text{Sets}$$

is a channel structure.

49.4. Iterated Differential Increments.

Definition 49.7 (Iterated Differential Increment). Given a differential engine E and a differential increment X , define

$$\text{Iter}_E(0, X) = X, \quad \text{Iter}_E(n+1, X) = E.\text{baseSpace}.\text{incr}(X.\text{id}).$$

49.5. Structural Lemma.

LEMMA 49.8 (Validity of Differential Iteration). *Let E be a Faure Differential Engine. For every differential increment X and every $n \in \mathbb{N}$, $\text{Iter}_E(n, X)$ is a valid differential increment.*

Proof. Induction on n .

For $n = 0$, $\text{Iter}_E(0, X) = X$ is valid.

Assume $\text{Iter}_E(n, X)$ is valid. Let

$$i := (\text{Iter}_E(n, X)).\text{id}.$$

Then $E.\text{baseSpace}.\text{incr}(i)$ is valid by definition of $E.\text{baseSpace}.\text{incr}$.

Thus $\text{Iter}_E(n+1, X)$ is valid. $\square$

49.6. Structural Theorem.

THEOREM 49.9 (Stability of Differential Iteration). *Let E be a Faure Differential Engine. For every differential increment X and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, X)$$

is well-defined and belongs to the image of

$$E.\text{baseSpace}.\text{incr}.$$

Proof. By Lemma 49.8, each iterate is valid. By definition of the iteration step,

$$\text{Iter}_E(n+1, X) = E.\text{baseSpace}.\text{incr}(i)$$

for some index i , hence it lies in the image of $E.\text{baseSpace}.\text{incr}$. $\square$

Remark 49.10. The theorem asserts structural well-definedness of differential iteration. No assumptions about continuity, differentiability, or analytic structure are required beyond the definitions of the engine and its base space.

50. Faure Category Engine

50.1. *Categorical Indices and Objects.*

Definition 50.1 (Categorical Index). A categorical index is a pair

$$I = (i, t),$$

with $i, t \in \mathbb{N}$.

Definition 50.2 (Categorical Object). A categorical object is a pair

$$X = (I, D),$$

where I is a categorical index and D is a set representing the object data.

50.2. *Morphisms and Categorical Spaces.*

Definition 50.3 (Categorical Morphism). A categorical morphism is a quadruple

$$f = (X_{\text{src}}, X_{\text{dst}}, t, I),$$

where X_{src} and X_{dst} are categorical objects, $t \in \mathbb{N}$ is a morphism tag, and I is auxiliary information.

Definition 50.4 (Categorical Space). A categorical space is a quadruple

$$S = (O, M, \lambda, P),$$

where

$$O : \mathbb{N} \rightarrow \text{Categorical Objects}, \quad M : \mathbb{N} \rightarrow \text{Categorical Morphisms},$$

$\lambda \in \mathbb{N}$ is a global label, and

$$P : \mathbb{N} \rightarrow \text{Sets}$$

is a payload structure.

50.3. *Category Operators and Engines.*

Definition 50.5 (Category Operator). A category operator is a pair

$$O = (\Phi, t),$$

where $\Phi : \text{Categorical Morphisms} \rightarrow \text{Categorical Morphisms}$ and $t \in \mathbb{N}$ is an operator tag.

Definition 50.6 (Faure Category Engine). A Faure Category Engine is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is a categorical space, O is a category operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \text{Sets}$$

is a channel structure.

50.4. Iterated Morphisms.

Definition 50.7 (Iterated Morphism). Given a category engine E and a morphism f , define

$$\text{Iter}_E(0, f) = f, \quad \text{Iter}_E(n+1, f) = E.\text{baseSpace.morph}(f.\text{tag}).$$

50.5. Structural Lemma.

LEMMA 50.8 (Validity of Categorical Iteration). *Let E be a Faure Category Engine. For every morphism f and every $n \in \mathbb{N}$, $\text{Iter}_E(n, f)$ is a valid categorical morphism.*

Proof. Induction on n .

For $n = 0$, $\text{Iter}_E(0, f) = f$ is valid.

Assume $\text{Iter}_E(n, f)$ is valid. Let

$$i := (\text{Iter}_E(n, f)).\text{tag}.$$

Then $E.\text{baseSpace.morph}(i)$ is valid by definition of $E.\text{baseSpace.morph}$.

Thus $\text{Iter}_E(n+1, f)$ is valid. $\square$

50.6. Structural Theorem.

THEOREM 50.9 (Stability of Categorical Iteration). *Let E be a Faure Category Engine. For every morphism f and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, f)$$

is well-defined and belongs to the image of

$$E.\text{baseSpace.morph}.$$

Proof. By Lemma 50.8, each iterate is valid. By definition of the iteration step,

$$\text{Iter}_E(n+1, f) = E.\text{baseSpace.morph}(i)$$

for some index i , hence it lies in the image of $E.\text{baseSpace.morph}$. $\square$

Remark 50.10. The theorem asserts structural well-definedness of categorical iteration. No categorical axioms such as identity morphisms, composition laws, or functoriality are required beyond the definitions of the engine and its base space.

51. Faure Analysis Engine Theorem

51.1. Analytical Indices and Values.

Definition 51.1 (Analytical Index). An analytical index is a pair

$$I = (i, t),$$

with $i, t \in \mathbb{N}$.

Definition 51.2 (Analytical Value). An analytical value is a pair

$$V = (I, D),$$

where I is an analytical index and D is a set representing the value data.

51.2. Analytical Cells and Spaces.

Definition 51.3 (Analytical Cell). An analytical cell is a triple

$$C = (i, V, I),$$

where $i \in \mathbb{N}$ is an identifier, V is an analytical value, and I is auxiliary information.

Definition 51.4 (Analytical Space). An analytical space is a triple

$$S = (C, \lambda, P),$$

where

$$C : \mathbb{N} \rightarrow \text{Analytical Cells}, \quad \lambda \in \mathbb{N}, \quad P : \mathbb{N} \rightarrow \text{Sets}.$$

51.3. Analysis Operators and Engines.

Definition 51.5 (Analysis Operator). An analysis operator is a triple

$$O = (\phi_V, \phi_C, t),$$

where ϕ_V acts on analytical values, ϕ_C acts on analytical cells, and $t \in \mathbb{N}$ is an operator tag.

Definition 51.6 (Faure Analysis Engine). A Faure Analysis Engine is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is an analytical space, O is an analysis operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \text{Sets}$$

is a channel structure.

51.4. Iterated Analytical Cells.

Definition 51.7 (Iterated Analytical Cell). Given an analysis engine E and an analytical cell C , define

$$\text{Iter}_E(0, C) = C, \quad \text{Iter}_E(n+1, C) = E.\text{baseSpace}.\text{cell}(C.\text{id}).$$

51.5. Structural Lemma.

LEMMA 51.8 (Validity of Analytical Iteration). *Let E be a Faure Analysis Engine. For every analytical cell C and every $n \in \mathbb{N}$, $\text{Iter}_E(n, C)$ is a valid analytical cell.*

Proof. Induction on n .

For $n = 0$, $\text{Iter}_E(0, C) = C$ is valid.

Assume $\text{Iter}_E(n, C)$ is valid. Let

$$i := (\text{Iter}_E(n, C)).\text{id}.$$

Then $E.\text{baseSpace}.\text{cell}(i)$ is valid by definition of $E.\text{baseSpace}.\text{cell}$.

Thus $\text{Iter}_E(n+1, C)$ is valid. □

51.6. Structural Theorem.

THEOREM 51.9 (Stability of Analytical Iteration). *Let E be a Faure Analysis Engine. For every analytical cell C and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, C)$$

is well-defined and belongs to the image of

$$E.\text{baseSpace}.\text{cell}.$$

Proof. By Lemma 51.8, each iterate is valid. By definition of the iteration step,

$$\text{Iter}_E(n+1, C) = E.\text{baseSpace}.\text{cell}(i)$$

for some index i , hence it lies in the image of $E.\text{baseSpace}.\text{cell}$. $\square$

Remark 51.10. The theorem asserts structural well-definedness of analytical iteration. No algebraic properties such as associativity, commutativity, or linearity are required beyond the definitions of the engine and its base space.

52. Faure Algebra Engine Theorem

52.1. Algebraic Indices and Elements.

Definition 52.1 (Algebraic Index). An algebraic index is a pair

$$I = (i, t),$$

with $i, t \in \mathbb{N}$.

Definition 52.2 (Algebraic Element). An algebraic element is a pair

$$X = (I, D),$$

where I is an algebraic index and D is a set representing the element data.

52.2. Algebraic Operations and Spaces.

Definition 52.3 (Algebraic Operation). An algebraic operation is a quadruple

$$O = (X_{\text{left}}, X_{\text{right}}, t, I),$$

where X_{left} and X_{right} are algebraic elements, $t \in \mathbb{N}$ is an operation tag, and I is auxiliary information.

Definition 52.4 (Algebraic Space). An algebraic space is a quadruple

$$S = (E, O, \lambda, P),$$

where

$E : \mathbb{N} \rightarrow \text{Algebraic Elements}, \quad O : \mathbb{N} \rightarrow \text{Algebraic Operations},$
 $\lambda \in \mathbb{N}$ is a global label, and

$$P : \mathbb{N} \rightarrow \text{Sets}$$

is a payload structure.

52.3. Algebra Operators and Engines.

Definition 52.5 (Algebra Operator). An algebra operator is a pair

$$O = (\Phi, t),$$

where $\Phi : \text{Algebraic Elements} \rightarrow \text{Algebraic Elements}$ and $t \in \mathbb{N}$ is an operator tag.

Definition 52.6 (Faure Algebra Engine). A Faure Algebra Engine is a quadruple

$$E = (S, O, m, \Gamma),$$

where S is an algebraic space, O is an algebra operator, $m \in \mathbb{N}$ is a mode parameter, and

$$\Gamma : \mathbb{N} \rightarrow \text{Sets}$$

is a channel structure.

52.4. Iterated Algebraic Elements.

Definition 52.7 (Iterated Algebraic Element). Given an algebra engine E and an algebraic element X , define

$$\text{Iter}_E(0, X) = X, \quad \text{Iter}_E(n+1, X) = E.\text{baseSpace}.\text{elem}(X.\text{idx}.\text{id}).$$

52.5. Structural Lemma.

LEMMA 52.8 (Validity of Iteration). *Let E be a Faure Algebra Engine. For every algebraic element X and every $n \in \mathbb{N}$, $\text{Iter}_E(n, X)$ is a valid algebraic element.*

Proof. Induction on n .

For $n = 0$, $\text{Iter}_E(0, X) = X$ is valid.

Assume $\text{Iter}_E(n, X)$ is valid. Let

$$i := (\text{Iter}_E(n, X)).\text{idx}.\text{id}.$$

Then $E.\text{baseSpace}.\text{elem}(i)$ is valid by definition of $E.\text{baseSpace}.\text{elem}$.

Thus $\text{Iter}_E(n+1, X)$ is valid. $\square$

52.6. Structural Theorem.

THEOREM 52.9 (Stability of Algebraic Iteration). *Let E be a Faure Algebra Engine. For every algebraic element X and every $n \in \mathbb{N}$,*

$$\text{Iter}_E(n, X)$$

is well-defined and belongs to the image of

$$E.\text{baseSpace}.\text{elem}.$$

Proof. By Lemma 52.8, each iterate is valid. By definition of the iteration step,

$$\text{Iter}_E(n+1, X) = E.\text{baseSpace}.\text{elem}(i)$$

for some index i , hence it lies in the image of $E.\text{baseSpace}.\text{elem}$. $\square$

Remark 52.10. The theorem asserts structural well-definedness of algebraic iteration. No algebraic properties such as associativity, commutativity, or linearity are required beyond the definitions of the engine and its base space.

53. Faure Idempotent Projection Theorem (Additive Form)

53.1. Definitions.

Definition 53.1 (Iteration). Let V be an abelian group and $T : V \rightarrow V$ an additive map. Define recursively:

$$\text{Iter}_T(0, x) = x, \quad \text{Iter}_T(n+1, x) = T(\text{Iter}_T(n, x)).$$

Definition 53.2 (Stability at Level n). An additive endomorphism $T : V \rightarrow V$ is stable at level n if

$$\text{Iter}_T(n, x) = \text{Iter}_T(n+1, x) \quad \text{for all } x \in V.$$

Definition 53.3 (Kernel Rigidity at Level n). T has kernel rigidity at level n if

$$\text{Iter}_T(n, x) = 0 \implies T(x) = 0.$$

Definition 53.4 (Idempotence). An additive endomorphism $T : V \rightarrow V$ is idempotent if

$$T^2 = T.$$

Definition 53.5 (Kernel and Image).

$$\ker(T) = \{x \in V : T(x) = 0\}, \quad \text{im}(T) = \{T(y) : y \in V\}.$$

Definition 53.6 (Canonical Projections).

$$\text{proj}_{\ker}(x) = x - T(x), \quad \text{proj}_{\text{im}}(x) = T(x).$$

53.2. *Lemmas.*

LEMMA 53.7 (Iteration on a Subtraction). *For all $n \in \mathbb{N}$ and all $x \in V$,*

$$\text{Iter}_T(n, x - T(x)) = \text{Iter}_T(n, x) - \text{Iter}_T(n, T(x)).$$

Proof. By additivity and induction on n . □

LEMMA 53.8 (Successor Iteration). *For all $n \in \mathbb{N}$ and all $x \in V$,*

$$\text{Iter}_T(n, T(x)) = \text{Iter}_T(n + 1, x).$$

Proof. Immediate from the definition of iteration. □

LEMMA 53.9 (Fixed Points of the Image). *If $m \in \text{im}(T)$ and T is idempotent, then*

$$T(m) = m.$$

Proof. If $m = T(x)$, then $T(m) = T(T(x)) = T(x) = m$. □

LEMMA 53.10 (Kernel Component Lies in the Kernel). *If T is idempotent, then*

$$T(x - T(x)) = 0.$$

Proof. By additivity,

$$T(x - T(x)) = T(x) - T^2(x) = T(x) - T(x) = 0.$$

□

53.3. *Idempotence.*

THEOREM 53.11 (Faure Theorem A: Stability and Rigidity Imply Idempotence). *Let $T : V \rightarrow V$ be additive. If T is stable at level $n > 0$ and has kernel rigidity at level n , then*

$$T^2 = T.$$

Proof. Fix $x \in V$ and set $y := x - T(x)$.

By Lemma 53.7 and stability,

$$\text{Iter}_T(n, y) = \text{Iter}_T(n, x) - \text{Iter}_T(n, T(x)) = \text{Iter}_T(n, x) - \text{Iter}_T(n + 1, x) = 0.$$

By kernel rigidity,

$$T(y) = 0.$$

Thus

$$T(x - T(x)) = 0.$$

By additivity,

$$T(x) - T^2(x) = 0,$$

and by cancellation,

$$T^2(x) = T(x).$$

□

53.4. Kernel–Image Structure.

THEOREM 53.12 (Faure Theorem B: Trivial Intersection). *If T is idempotent, then*

$$\ker(T) \cap \operatorname{im}(T) = \{0\}.$$

Proof. If $z \in \ker(T)$ and $z = T(x)$, then

$$z = T(z) = 0.$$

□

THEOREM 53.13 (Faure Theorem C: Uniqueness of Decomposition). *If*

$$x = k_1 + m_1 = k_2 + m_2, \quad k_i \in \ker(T), \quad m_i \in \operatorname{im}(T),$$

then

$$k_1 = k_2, \quad m_1 = m_2.$$

Proof. Applying T gives $m_1 = m_2$, hence $k_1 = k_2$. □

THEOREM 53.14 (Faure Theorem D: Canonical Decomposition). *For all $x \in V$,*

$$x = (x - T(x)) + T(x).$$

Proof. Set $k := x - T(x)$ and $m := T(x)$. □

THEOREM 53.15 (Faure Theorem E: Direct Sum). *If T is idempotent, then*

$$V = \ker(T) \oplus \operatorname{im}(T).$$

Proof. Combine Theorems 53.12, 53.13, and 53.14. □

53.5. *Corollaries.*

COROLLARY 53.16 (Projection). *If T is idempotent, then*

$$T : V \rightarrow \text{im}(T)$$

is the projection along $\ker(T)$.

53.6. *Linear Structure.*

THEOREM 53.17 (Faure Linear Projection Theorem). *Let V be a vector space over a field K and $T : V \rightarrow V$ linear. If T is idempotent, then*

$$V = \ker(T) \oplus \text{range}(T),$$

and T is the linear projection onto $\text{range}(T)$ along $\ker(T)$.

54. **Faure Fourth–Fifth Idempotence Theorem**54.1. *Definitions.*

Definition 54.1 (Fourth Iterate). Let $T : V \rightarrow V$ be an additive endomorphism. Define

$$T^{(4)} := T \circ T \circ T \circ T.$$

Definition 54.2 (Fifth Iterate). Define

$$T^{(5)} := T \circ T \circ T \circ T \circ T.$$

Definition 54.3 (Kernel Rigidity). T is kernel rigid if

$$T^{(4)}(x) = 0 \implies T(x) = 0 \quad \text{for all } x \in V.$$

Definition 54.4 (Fourth–Fifth Stability). T satisfies fourth–fifth stability if

$$T^{(4)}(x) = T^{(5)}(x) \quad \text{for all } x \in V.$$

54.2. *Lemmas.*

LEMMA 54.5 (Fourth–Fifth Cancellation). *If $T^{(4)}(x) = T^{(5)}(x)$ for all $x \in V$, then*

$$T^{(4)}(x - T(x)) = 0.$$

Proof. Let $y := x - T(x)$. By additivity,

$$T^{(4)}(y) = T^{(4)}(x) - T^{(5)}(x).$$

By stability, $T^{(4)}(x) = T^{(5)}(x)$, hence $T^{(4)}(y) = 0$. $\square$

LEMMA 54.6 (Kernel Rigidity Extraction). *If T is kernel rigid and $T^{(4)}(y) = 0$, then*

$$T(y) = 0.$$

Proof. This is exactly kernel rigidity. $\square$

54.3. Idempotence.

THEOREM 54.7 (Faure Theorem F: Fourth–Fifth Stability and Rigidity Imply Idempotence). *If $T^{(4)} = T^{(5)}$ pointwise and $\ker(T^{(4)}) \subseteq \ker(T)$, then*

$$T^2 = T.$$

Proof. Fix $x \in V$ and set $y := x - T(x)$.

By Lemma 54.5,

$$T^{(4)}(y) = 0.$$

By Lemma 54.6,

$$T(y) = 0.$$

Thus

$$T(x - T(x)) = 0.$$

By additivity,

$$T(x) - T^2(x) = 0.$$

By cancellation,

$$T^2(x) = T(x).$$

$\square$

54.4. Consequences of Idempotence.

THEOREM 54.8 (Faure Theorem G: Collapse of Positive Iterates). *If T is idempotent, then for all $n > 0$,*

$$T^{(n)} = T.$$

Proof. Induction on n using $T^2 = T$. $\square$

THEOREM 54.9 (Faure Theorem H: Range Fixedness). *If T is idempotent and $y \in \text{range}(T)$, then*

$$T(y) = y.$$

Proof. If $y = T(x)$, then $T(y) = T(T(x)) = T(x) = y$. $\square$

54.5. *Kernel–Range Decomposition.*

Definition 54.10 (Kernel).

$$\ker(T) := \{x \in V : T(x) = 0\}.$$

Definition 54.11 (Range).

$$\text{range}(T) := \{T(x) : x \in V\}.$$

THEOREM 54.12 (Faure Theorem I: Existence of Decomposition). *If T is idempotent, then for all $x \in V$ there exist $k, r \in V$ such that*

$$T(k) = 0, \quad r \in \text{range}(T), \quad x = k + r.$$

Proof. Set $k := x - T(x)$ and $r := T(x)$. Then $T(k) = 0$ and $r \in \text{range}(T)$, and $x = k + r$. $\square$

THEOREM 54.13 (Faure Theorem J: Projection Extracts the Range). *If T is idempotent and $T(k) = 0$, then*

$$T(k + T(x)) = T(x).$$

Proof. By additivity,

$$T(k + T(x)) = T(k) + T(T(x)) = 0 + T(x) = T(x).$$

$\square$

THEOREM 54.14 (Faure Theorem K: Uniqueness of Decomposition). *If*

$$k_1 + T(x_1) = k_2 + T(x_2), \quad T(k_1) = T(k_2) = 0,$$

then

$$k_1 = k_2, \quad T(x_1) = T(x_2).$$

Proof. Applying T gives $T(x_1) = T(x_2)$. Subtracting yields $k_1 = k_2$. $\square$

THEOREM 54.15 (Faure Theorem L: Trivial Intersection). *If T is idempotent, then*

$$\ker(T) \cap \text{range}(T) = \{0\}.$$

Proof. If $z \in \ker(T)$ and $z = T(x)$, then

$$z = T(z) = 0.$$

□

54.6. Kernel–Range Structure.

THEOREM 54.16 (Faure Theorem M: Kernel–Range Structure). *If T is idempotent, then:*

- (1) *Every $x \in V$ decomposes as $x = k + r$ with $k \in \ker(T)$ and $r \in \text{range}(T)$.*
- (2) *The intersection is trivial:*

$$\ker(T) \cap \text{range}(T) = \{0\}.$$

54.7. Faure–Fourier Transform.

Definition 54.17 (Faure–Fourier Transform). A Faure–Fourier transform is an additive endomorphism $F : V \rightarrow V$ satisfying

$$F^{(4)} = F^{(5)}, \quad \ker(F^{(4)}) \subseteq \ker(F).$$

THEOREM 54.18 (Faure–Fourier Idempotence Theorem). *Every Faure–Fourier transform is idempotent:*

$$F^2 = F.$$

Proof. Apply Theorem 54.7. □

THEOREM 54.19 (Faure–Fourier Decomposition Theorem). *Every Faure–Fourier transform induces a kernel–range decomposition:*

$$V = \ker(F) \oplus \text{range}(F).$$

Proof. Apply Theorem 54.16. □

THEOREM 54.20 (Faure–Fourier Explicit Decomposition). *For all $x \in V$,*

$$x = (x - F(x)) + F(x), \quad x - F(x) \in \ker(F), \quad F(x) \in \text{range}(F).$$

Proof. Set $k := x - F(x)$ and $r := F(x)$. □

THEOREM 54.21 (Faure–Fourier Master Theorem). *Let F be a Faure–Fourier transform. Then*

$$F^2 = F, \quad V = \ker(F) \oplus \text{range}(F), \quad \forall x \in V, \quad x = (x - F(x)) + F(x).$$

55. Faure Fourier Final Bridge

55.1. Definitions.

Definition 55.1 (Faure Sequence). Let Y be a set. A Faure sequence is a map

$$s : \mathbb{N} \rightarrow Y.$$

Definition 55.2 (Fourier Kernel). Let X, Y, S be sets. A Fourier kernel is a map

$$K : X \times Y \rightarrow S.$$

Definition 55.3 (Aggregator). Let S be a set. An aggregator is a pair $(0_S, +_S)$ with

$$0_S \in S, \quad +_S : S \times S \rightarrow S.$$

Definition 55.4 (Faure–Fourier Datum). A Faure–Fourier datum is a triple (s, K, A) with

$$s : \mathbb{N} \rightarrow Y, \quad K : X \times Y \rightarrow S, \quad A = (0_S, +_S).$$

55.2. Faure–Fourier Transform.

Definition 55.5 (Faure–Fourier Transform). Given Faure–Fourier data (s, K, A) and $x \in X$, define for $N \in \mathbb{N}$:

$$\mathcal{F}(x, N) = \sum_{n=0}^N K(x, s(n)),$$

where the sum is taken using the aggregator A :

$$\sum_{n=0}^N K(x, s(n)) = K(x, s(N)) +_S \cdots +_S K(x, s(0)).$$

55.3. Basic Properties.

THEOREM 55.6 (Faure Fourier Identity). *For all Faure–Fourier data (s, K, A) , all $x \in X$, and all $N \in \mathbb{N}$,*

$$\mathcal{F}(x, N) = \sum_{n=0}^N K(x, s(n)).$$

Proof. Immediate from the definition. $\square$

THEOREM 55.7 (Faure Fourier Invariance). *Let (s_1, K_1, A_1) and (s_2, K_2, A_2) be Faure–Fourier data. If*

$$s_1 = s_2, \quad K_1 = K_2, \quad A_1 = A_2,$$

then for all $x \in X$ and all $N \in \mathbb{N}$,

$$\mathcal{F}_1(x, N) = \mathcal{F}_2(x, N).$$

Proof. If the three components coincide, the definitions coincide pointwise. $\square$

THEOREM 55.8 (Faure Fourier Structural Identity). *For all Faure–Fourier data (s, K, A) ,*

$$\mathcal{F}(x, N) = \sum_{n=0}^N K(x, s(n)).$$

Proof. This is Theorem 55.6. $\square$

55.4. Kernel–Range Structure.

THEOREM 55.9 (Faure–Fourier Kernel–Range Structure Theorem). *Let V be an abelian group and let*

$$T : V \rightarrow V$$

be an additive endomorphism. Assume:

(1) *Fourth–fifth stability:*

$$T^4 = T^5.$$

(2) *Kernel rigidity:*

$$\ker(T^4) \subseteq \ker(T).$$

Then T is idempotent,

$$T^2 = T,$$

and the following kernel–range structure holds:

(1) For every $x \in V$ there exist

$$k \in \ker(T), \quad r \in \operatorname{Ran}(T),$$

such that

$$x = k + r.$$

(2) The intersection is trivial:

$$\ker(T) \cap \operatorname{Ran}(T) = \{0\}.$$

(3) The decomposition is unique.

Consequently,

$$V = \ker(T) \oplus \operatorname{Ran}(T),$$

and T is the canonical projection onto $\operatorname{Ran}(T)$ along $\ker(T)$.

Proof. By Lemma 55.10, $T^2 = T$.

For any $x \in V$, set

$$k := x - T(x), \quad r := T(x).$$

Then $T(k) = 0$ and $r \in \operatorname{Ran}(T)$, and $x = k + r$.

Uniqueness follows by applying T to both decompositions.

If $z \in \ker(T) \cap \operatorname{Ran}(T)$, then $T(z) = 0$ and $z = T(y)$, so $z = T(z) = 0$. $\square$

LEMMA 55.10 (Faure Rigidity Implies Idempotence). *Assume fourth-fifth stability and kernel rigidity. Then $T^2 = T$.*

Proof. Fix $x \in V$ and set $y := x - T(x)$.

By stability,

$$T^4(x) = T^5(x).$$

A direct computation gives

$$T^4(y) = T^4(x) - T^5(x) = 0.$$

By rigidity,

$$T(y) = 0.$$

Thus

$$T(x - T(x)) = 0.$$

By additivity,

$$T(x) - T^2(x) = 0,$$

and by cancellation,

$$T^2(x) = T(x).$$

□

Remark 55.11. The Faure–Fourier kernel–range structure is the classical decomposition associated with idempotent endomorphisms. The fourth–fifth stability and kernel rigidity provide a purely algebraic criterion ensuring idempotence.

56. Faure B33 Kernel–Rigidity Closure

56.1. Additive Structure.

Definition 56.1 (Additive Space). An additive space is a quadruple $(X, 0, +, -)$ consisting of a set X , a distinguished element $0 \in X$, an addition $+: X \times X \rightarrow X$, and a negation $-: X \rightarrow X$, satisfying

$$x + (-x) = 0,$$

and the cancellation axiom

$$a + (-b) = 0 \implies a = b.$$

Definition 56.2 (Linear Operator). Let $(X, 0, +, -)$ be an additive space. A linear operator is a map $T: X \rightarrow X$ satisfying

$$T(x + y) = T(x) + T(y), \quad T(-x) = -T(x).$$

56.2. Iterates.

Definition 56.3 (Iterates). For a linear operator T on X , define

$$T^4(x) := T(T(T(T(x)))), \quad T^5(x) := T(T^4(x)), \quad T^2(x) := T(T(x)).$$

56.3. Rigidity and Stability.

Definition 56.4 (Fourth–Kernel Rigidity). T is kernel rigid if

$$T^4(x) = 0 \implies T(x) = 0 \quad \text{for all } x \in X.$$

Definition 56.5 (Kernel Stability). T is kernel stable if

$$T(x) = 0 \implies T^4(x) = 0 \quad \text{for all } x \in X.$$

56.4. *Faure B33.1.*

THEOREM 56.6 (Faure B33.1: Fourth–Kernel Equality). *If T is kernel rigid and kernel stable, then for all $x \in X$,*

$$T^4(x) = 0 \iff T(x) = 0.$$

Proof. If $T^4(x) = 0$, rigidity gives $T(x) = 0$. If $T(x) = 0$, stability gives $T^4(x) = 0$. $\square$

56.5. *Faure B33.2.*

THEOREM 56.7 (Faure B33.2: End–to–End Idempotence). *Assume*

$$T^4(x) = T^5(x) \quad \text{for all } x \in X,$$

and assume kernel rigidity. Then

$$T^2(x) = T(x) \quad \text{for all } x \in X.$$

Proof. Fix x and set $y := x - T(x)$. Using linearity and $T^4(x) = T^5(x)$,

$$T^4(y) = 0.$$

By rigidity,

$$T(y) = 0.$$

Thus

$$T(x - T(x)) = 0.$$

Expanding,

$$T(x) - T^2(x) = 0.$$

By cancellation,

$$T^2(x) = T(x).$$

$\square$

56.6. *Faure B33 Final.*

THEOREM 56.8 (Faure B33 Final: Kernel–Rigidity Closure). *Let T be a linear operator on an additive space. Assume*

$$T^4(x) = T^5(x) \quad \text{for all } x \in X,$$

and assume kernel rigidity and kernel stability. Then:

(1) For all $x \in X$,

$$T^4(x) = 0 \iff T(x) = 0.$$

(2) For all $x \in X$,

$$T^2(x) = T(x).$$

Proof. Item (1) is Theorem 56.6. Item (2) is Theorem 56.7. $\square$

57. Faure B32 Idempotence Theorem

57.1. Additive Structure.

Definition 57.1 (Additive Space). An additive space is a quadruple $(X, 0, +, -)$ consisting of a set X , a distinguished element $0 \in X$, an addition $+: X \times X \rightarrow X$, and a negation $-: X \rightarrow X$, satisfying

$$x + (-x) = 0,$$

and the cancellation axiom

$$a + (-b) = 0 \implies a = b.$$

Definition 57.2 (Linear Operator). Let $(X, 0, +, -)$ be an additive space. A linear operator is a map $T: X \rightarrow X$ satisfying

$$T(x + y) = T(x) + T(y), \quad T(-x) = -T(x).$$

57.2. Iterates.

Definition 57.3 (Iterates). For a linear operator T on X , define

$$T^2(x) := T(T(x)), \quad T^3(x) := T(T^2(x)), \quad T^4(x) := T(T^3(x)), \quad T^5(x) := T(T^4(x)).$$

57.3. Rigidity.

Definition 57.4 (Fourth-Kernel Rigidity). A linear operator T is kernel rigid if

$$T^4(x) = 0 \implies T(x) = 0 \quad \text{for all } x \in X.$$

LEMMA 57.5 (Fourth Iterate Difference). Let T be a linear operator satisfying $T^4(x) = T^5(x)$ for all $x \in X$. Then for every $x \in X$,

$$T^4(x - T(x)) = 0.$$

Proof. Fix $x \in X$ and set $y := x - T(x)$. Using linearity,

$$T^4(y) = T^4(x - T(x)) = T^4(x) - T^5(x).$$

By hypothesis, $T^4(x) = T^5(x)$, hence

$$T^4(y) = 0.$$

□

57.4. Faure B32.

THEOREM 57.6 (Faure B32: Kernel Rigidity Implies Idempotence). *Assume*

$$T^4(x) = T^5(x) \quad \text{for all } x \in X,$$

and assume fourth-kernel rigidity. Then

$$T^2(x) = T(x) \quad \text{for all } x \in X.$$

Proof. Fix $x \in X$ and set $y := x - T(x)$.

By Lemma 57.5,

$$T^4(y) = 0.$$

By fourth-kernel rigidity,

$$T(y) = 0.$$

Thus

$$T(x - T(x)) = 0.$$

Using linearity,

$$T(x) - T^2(x) = 0.$$

By cancellation,

$$T^2(x) = T(x).$$

□

Remark 57.7. The theorem shows that under the fourth-iterate identity and kernel rigidity, the operator becomes idempotent. No assumptions about associativity, commutativity, or additional algebraic structure are required beyond the additive axioms stated above.

58. The Faure Ancestral Theorem

In this section we isolate a structural uniqueness principle for representations of observable relations. A representation consists of a carrier map and an observable map, and it is required to preserve a given binary observable relation.

Definition 58.1. Let A, A' be types of objects and S, S' types of observables. A *representation* of (A, S) into (A', S') is a pair

$$(T, \varphi): A \rightarrow A', \quad S \rightarrow S',$$

where T is the carrier map and φ is the observable map. Given a binary observable relation

$$R: A \times A \rightarrow S, \quad R': A' \times A' \rightarrow S',$$

we say that (T, φ) *preserves* (R, R') if

$$\forall a, b \in A, \quad R'(T(a), T(b)) = \varphi(R(a, b)).$$

Definition 58.2. We say that R is *observable complete* if every observable value arises from some pair of objects:

$$\forall s \in S, \exists a, b \in A, \quad R(a, b) = s.$$

We say that R is *rigid* if every structure-preserving self-map $\sigma: A \rightarrow A$ is the identity, i.e.

$$(\forall a, b, \quad R(\sigma(a), \sigma(b)) = R(a, b)) \implies \sigma = \text{id}_A.$$

Finally, a carrier map $T: A \rightarrow A'$ has a *right inverse* if there exists $T^{-1}: A' \rightarrow A$ such that $T(T^{-1}(a')) = a'$ for all $a' \in A'$.

The core structural result can be stated as follows (compare the Lean formalization:

“Observable completeness and agreement determine the observable map. A right inverse transfers the comparison of representations back to the source structure. Injectivity of the observable map then identifies the transported source relation. Structural rigidity eliminates the resulting self-transformation.”)

THEOREM 58.3 (Faure Ancestral Theorem). *Let A, A' be object types, S, S' observable types, and let*

$$R: A \times A \rightarrow S, \quad R': A' \times A' \rightarrow S'$$

be observable relations. Consider two representations

$$(T_1, \varphi_1), (T_2, \varphi_2): (A, S) \rightarrow (A', S')$$

which both preserve (R, R') , i.e.

$$\forall a, b \in A, \quad R'(T_i(a), T_i(b)) = \varphi_i(R(a, b)), \quad i = 1, 2.$$

Assume:

- (1) R is observable complete;
- (2) φ_1 is injective;
- (3) T_1 has a right inverse $T^{-1}: A' \rightarrow A$;
- (4) the target relation agrees on the images of T_1 and T_2 :

$$\forall a, b \in A, \quad R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b));$$

- (5) R is rigid.

Then the two representations coincide:

$$(T_1, \varphi_1) = (T_2, \varphi_2),$$

i.e. $T_1 = T_2$ and $\varphi_1 = \varphi_2$.

Proof sketch. Observable completeness and the pointwise agreement

$$R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b))$$

allow one to show that the observable maps must coincide: $\varphi_1 = \varphi_2$. Using the right inverse T^{-1} of T_1 , one defines a transported self-map of the source

$$\sigma(a) := T^{-1}(T_2(a)).$$

A direct computation shows that σ preserves R , thanks to the preservation hypotheses and the equality $\varphi_1 = \varphi_2$; injectivity of φ_1 is used to pull back equality of observables to equality of relations. By rigidity, any structure-preserving self-map must be the identity, so $\sigma = \text{id}_A$. Applying T_1 and the right-inverse property then yields $T_1 = T_2$. Together with $\varphi_1 = \varphi_2$, this gives $(T_1, \varphi_1) = (T_2, \varphi_2)$. $\square$

This theorem is the ancestral structural principle of the Faure framework: it shows that, under purely logical completeness and rigidity assumptions, a representation preserving all observable data is uniquely determined as a pair of carrier and observable maps.

59. Faure Theorem VI: Fully Determined Transport of Relation-Valued Maps

Definition 59.1 (Relation-Valued Map). Let A, S be sets. A relation-valued map is a function

$$R : A \times A \rightarrow S.$$

Similarly, for sets A', S' , a second relation-valued map is

$$R' : A' \times A' \rightarrow S'.$$

Definition 59.2 (Transport Pair). Let

$$T : A \rightarrow A', \quad \varphi : S \rightarrow S'$$

be maps. We say that (T, φ) *transports* R to R' if

$$\forall a, b \in A, \quad \varphi(R(a, b)) = R'(T(a), T(b)).$$

LEMMA 59.3 (Uniqueness of Transport Output). *If (T, φ) transports R to R' , then for all $a, b \in A$, the value $R'(T(a), T(b))$ is uniquely determined by $R(a, b)$.*

Proof. By the transport condition,

$$R'(T(a), T(b)) = \varphi(R(a, b)).$$

Since φ is a function, $\varphi(R(a, b))$ is uniquely determined. Thus $R'(T(a), T(b))$ is uniquely determined. $\square$

THEOREM 59.4 (Faure VI: Fully Determined Transport). *Let (T_1, φ_1) and (T_2, φ_2) be transport pairs from R to R' . If*

$$\varphi_1(R(a, b)) = \varphi_2(R(a, b)) \quad \text{for all } a, b \in A,$$

then

$$R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b)) \quad \text{for all } a, b \in A.$$

Proof. Since both pairs transport R to R' , we have

$$R'(T_1(a), T_1(b)) = \varphi_1(R(a, b)),$$

$$R'(T_2(a), T_2(b)) = \varphi_2(R(a, b)).$$

By hypothesis,

$$\varphi_1(R(a, b)) = \varphi_2(R(a, b)).$$

Thus

$$R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b)).$$

This holds for all $a, b \in A$. □

Remark 59.5. The theorem shows that the transported relation values are fully determined once the transport condition is fixed. No assumptions about injectivity, surjectivity, or algebraic structure are required. Any additional constraints on T or φ must be stated separately.

Definition 59.6 (Surjectivity of observable values). A valued relation $R : A \times A \rightarrow S$ is *observable-surjective* if

$$\forall s \in S, \exists a, b \in A, R(a, b) = s.$$

Definition 59.7 (Rigidity). A valued relation $R : A \times A \rightarrow S$ is *rigid* if every structure-preserving self-map is the identity:

$$\left(\forall a, b, R(\sigma(a), \sigma(b)) = R(a, b) \right) \implies \sigma = \text{id}_A.$$

Definition 59.8 (Right inverse). A map $T : A \rightarrow A'$ has a *right inverse* if there exists $T^{-1} : A' \rightarrow A$ such that

$$T(T^{-1}(a')) = a' \quad \forall a' \in A'.$$

We now state the transport-determination theorem. It generalizes the Faure Ancestral Theorem to arbitrary valued relations and arbitrary observable codomains.

THEOREM 59.9 (Faure Transport Fully Determined Theorem). *Let*

$$R : A \times A \rightarrow S, \quad R' : A' \times A' \rightarrow S'$$

be valued relations, and let

$$(T_1, \varphi_1), (T_2, \varphi_2)$$

be two representation pairs satisfying

$$\forall a, b, \quad \varphi_i(R(a, b)) = R'(T_i(a), T_i(b)), \quad i = 1, 2.$$

Assume:

(1) *R is observable-surjective;*

(2) the target relation agrees on the images of T_1 and T_2 :

$$\forall a, b, \quad R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b));$$

(3) φ_1 is injective;

(4) T_1 has a right inverse T_1^{-1} ;

(5) R is rigid.

Then the two representations coincide:

$$(T_1, \varphi_1) = (T_2, \varphi_2).$$

Proof sketch. Observable surjectivity allows one to determine $\varphi_1 = \varphi_2$ by evaluating both sides on a pair (a, b) with $R(a, b) = s$. With $\varphi_1 = \varphi_2$, define a transported self-map

$$\sigma(a) := T_1^{-1}(T_2(a)).$$

Using the preservation hypotheses and injectivity of φ_1 , one shows

$$R(\sigma(a), \sigma(b)) = R(a, b)$$

for all a, b . By rigidity, $\sigma = \text{id}_A$. Applying T_1 and the right-inverse property yields $T_1 = T_2$, and together with $\varphi_1 = \varphi_2$ this gives the desired equality of pairs. $\square$

This theorem provides a complete determination principle for valued-relational representations: under surjectivity, agreement, injectivity, right-inverse transport, and rigidity, the representation pair is uniquely determined.

60. Faure Theorem VI: Fully Determined Transport of Relation-Valued Maps

Definition 60.1 (Relation-Valued Structure). Let A, A', S, S' be sets and let

$$R : A \times A \rightarrow S, \quad R' : A' \times A' \rightarrow S'$$

be relation-valued maps.

Definition 60.2 (Transport Pair). Let

$$T : A \rightarrow A', \quad \varphi : S \rightarrow S'$$

be maps. We say that (T, φ) transports R to R' if

$$\forall a, b \in A, \quad \varphi(R(a, b)) = R'(T(a), T(b)).$$

LEMMA 60.3 (Uniqueness of Transport Output). *If (T, φ) transports R to R' , then for all $a, b \in A$, the value $R'(T(a), T(b))$ is uniquely determined by $R(a, b)$.*

Proof. By the definition of transport,

$$R'(T(a), T(b)) = \varphi(R(a, b)).$$

Since φ is a function, $\varphi(R(a, b))$ is uniquely determined. Thus $R'(T(a), T(b))$ is uniquely determined. $\square$

THEOREM 60.4 (Faure VI: Fully Determined Transport). *Let (T_1, φ_1) and (T_2, φ_2) be transport pairs from R to R' . If*

$$\varphi_1(R(a, b)) = \varphi_2(R(a, b)) \quad \text{for all } a, b \in A,$$

then

$$R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b)) \quad \text{for all } a, b \in A.$$

Proof. Since both pairs transport R to R' , we have

$$R'(T_1(a), T_1(b)) = \varphi_1(R(a, b)),$$

$$R'(T_2(a), T_2(b)) = \varphi_2(R(a, b)).$$

By hypothesis,

$$\varphi_1(R(a, b)) = \varphi_2(R(a, b)).$$

Thus

$$R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b)).$$

This holds for all $a, b \in A$. $\square$

Remark 60.5. The theorem shows that once the transport condition is fixed, the transported relation values are fully determined by the source relation and the chosen maps. No assumptions about injectivity, surjectivity, or algebraic structure are required. Any additional constraints on T or φ must be stated separately.

Proof. The proof proceeds in two stages. First, surjectivity of R -values and agreement of transported pairs imply

$$\varphi_1 = \varphi_2,$$

since for any $s \in S$ one can write $s = R(a, b)$ and compare

$$\varphi_1(s) = R'(T_1(a), T_1(b)) = R'(T_2(a), T_2(b)) = \varphi_2(s).$$

Second, with a common φ and its injectivity, one constructs an automorphism

$$\sigma(a) := T_1^{-1}(T_2(a))$$

of A preserving R , and rigidity forces $\sigma = \text{id}_A$. This yields $T_1 = T_2$, and hence $(T_1, \varphi_1) = (T_2, \varphi_2)$. $\square$

61. Theorem 5.1 — Universal Four-Step Nilpotency

In this section we give the granular, fully formal proof of the universal four-step nilpotency property associated with the Faure reduction scheme. The proof is carried out in a purely structural setting, without any appeal to floating-point arithmetic or numerical approximation.

61.1. *Reduced-complete system and intertwining.* We consider a pair of spaces

$$X_D, \quad R_D$$

together with the following data:

- a *complete* evolution operator $\Psi_D : X_D \rightarrow X_D$;
- a *reduced* evolution operator $\Psi_{\text{red}} : R_D \rightarrow R_D$;
- a projection map $\Pi_D : X_D \rightarrow R_D$;
- distinguished zero states $0_R \in R_D$ and $0_X \in X_D$.

The structural hypotheses of the Faure model are:

- (1) **Intertwining (compatibility of dynamics).** For every $x \in X_D$,
- (2)
$$\Pi_D(\Psi_D(x)) = \Psi_{\text{red}}(\Pi_D(x)).$$
- (2) **Injectivity of the projection kernel.** The only element of X_D that projects to the reduced zero state is the complete zero state:
- (3)
$$\Pi_D(x) = 0_R \implies x = 0_X.$$
- (3) **Surjectivity of the projection.** Every reduced state is the image of some complete state:
- (4)
$$\forall y \in R_D, \exists x \in X_D \text{ such that } \Pi_D(x) = y.$$
- (4) **Four-step nilpotency on the reduced space.** The reduced evolution is nilpotent of index 4:
- (5)
$$\forall r \in R_D, \quad \Psi_{\text{red}}^4(r) = 0_R,$$

where $\Psi_{\text{red}}^4 = \Psi_{\text{red}} \circ \Psi_{\text{red}} \circ \Psi_{\text{red}} \circ \Psi_{\text{red}}$.

61.2. *Iterated intertwining.* We first record the iterated form of the intertwining relation (2).

LEMMA 61.1 (Fourth-power intertwining). *Under the hypotheses above, for every $x \in X_D$ we have*

$$\Pi_D(\Psi_D^4(x)) = \Psi_{\text{red}}^4(\Pi_D(x)),$$

where $\Psi_D^4 = \Psi_D \circ \Psi_D \circ \Psi_D \circ \Psi_D$.

Proof. We apply (2) repeatedly. For any $x \in X_D$,

$$\Pi_D(\Psi_D^4(x)) = \Pi_D(\Psi_D(\Psi_D(\Psi_D(\Psi_D(x)))))$$

Using (2) on the outermost Ψ_D , we obtain

$$\Pi_D(\Psi_D(\Psi_D(\Psi_D(\Psi_D(x))))) = \Psi_{\text{red}}(\Pi_D(\Psi_D(\Psi_D(\Psi_D(x)))))$$

Applying (2) again to $\Pi_D(\Psi_D(\Psi_D(\Psi_D(x))))$, and repeating this process four times, we arrive at

$$\Pi_D(\Psi_D^4(x)) = \Psi_{\text{red}}^4(\Pi_D(x)),$$

as claimed. □

61.3. *Universal four-step nilpotency.* We now state and prove the main structural theorem.

THEOREM 61.2 (Universal four-step nilpotency). *Assume the data and hypotheses (2)–(5). Then for every $x \in X_D$ we have*

$$\Psi_D^4(x) = 0_X.$$

In other words, the complete evolution Ψ_D is nilpotent of index 4 on the entire space X_D .

Proof. Let $x \in X_D$ be arbitrary. By Lemma 61.1,

$$\Pi_D(\Psi_D^4(x)) = \Psi_{\text{red}}^4(\Pi_D(x)).$$

By the reduced nilpotency hypothesis (5), we have $\Psi_{\text{red}}^4(\Pi_D(x)) = 0_R$, hence

$$\Pi_D(\Psi_D^4(x)) = 0_R.$$

Applying the injectivity condition (3) to $x' := \Psi_D^4(x)$, we obtain

$$\Pi_D(x') = 0_R \implies x' = 0_X.$$

Therefore

$$\Psi_D^4(x) = 0_X,$$

for every $x \in X_D$, which proves the universal four-step nilpotency of Ψ_D . $\square$

61.4. *Interpretation.* Theorem 61.2 shows that once the reduced dynamics is nilpotent of index 4, and the projection Π_D is both intertwining and kernel-injective, the complete dynamics cannot sustain any residual state beyond four iterations: every trajectory collapses exactly to the distinguished zero state 0_X in four steps.

This is the precise mathematical content of the “universal four-step extinction” claimed in the Faure framework: the index $\nu = 4$ is not inserted by hand, but forced by the intertwining, injectivity, and reduced nilpotency structure.

62. Theorem 5.1 — Universal Four-Step Nilpotency

THEOREM 62.1 (Universal Four-Step Nilpotency). *Let (X_D, Ψ_D) be a dynamical system on a complete space X_D and (R_D, Ψ_{red}) a reduced dynamical system on a reduced space R_D . Let*

$$\Pi_D : X_D \rightarrow R_D$$

be a projection map satisfying:

(1) *(Intertwining) For all $x \in X_D$,*

$$\Pi_D(\Psi_D(x)) = \Psi_{\text{red}}(\Pi_D(x)).$$

(2) *(Injectivity on the kernel) There exist distinguished zero states $0_X \in X_D$ and $0_R \in R_D$ such that*

$$\Pi_D(x) = 0_R \Rightarrow x = 0_X.$$

(3) *(Reduced nilpotency of index 4) For all $r \in R_D$,*

$$\Psi_{\text{red}}^4(r) = 0_R.$$

Then the complete dynamics is nilpotent of index 4, i.e. for all $x \in X_D$,

$$\Psi_D^4(x) = 0_X.$$

Proof. By the intertwining property, iterating four times yields

$$\Pi_D(\Psi_D^4(x)) = \Psi_{\text{red}}^4(\Pi_D(x))$$

for every $x \in X_D$. By reduced nilpotency, the right-hand side vanishes:

$$\Psi_{\text{red}}^4(\Pi_D(x)) = 0_R,$$

so we obtain

$$\Pi_D(\Psi_D^4(x)) = 0_R.$$

By injectivity on the kernel of Π_D , this implies

$$\Psi_D^4(x) = 0_X$$

for all $x \in X_D$, which proves the universal four-step nilpotency. $\square$

TOE Validator – Faure AOS144 (strumentazione). Il codice `toe_validator_faure_aos144.py` implementa uno strumento di validazione automatica per i file `.lean` del progetto AUF2026. Esso:

- sanitizza il sorgente eliminando metadati non matematici;
- invoca il compilatore Lean 4 su una copia pulita del file;
- raccoglie il numero di errori, avvertimenti, occorrenze di `sorry` e `admit`;
- calcola l'hash SHA-256 del sorgente;
- produce un report JSON strutturato (stato PASS/FAIL, versione di Lean, conteggio dei teoremi, log di compilazione);
- fornisce una interfaccia grafica (Tkinter) e una modalità CLI per lanciare uno script PowerShell di packaging.

Questo modulo è puramente operativo e non introduce alcun contenuto matematico aggiuntivo rispetto ai teoremi formalizzati in Lean.

63. Structural Modular Projection and Deterministic State Evaluation

This section introduces the modular projection and deterministic state evaluation used in the structural verification layer of the Faure framework. All identities are purely arithmetic and do not involve empirical data.

Throughout,

$$\mathcal{L} = 144$$

denotes the fixed Faure lattice modulus.

63.1. Modular Projection on the Faure Lattice.

Definition 63.1 (Lattice Projection). For any $x \in \mathbb{N}$, the *lattice projection* is

$$\pi_{\mathcal{L}}(x) = x \bmod \mathcal{L}.$$

Definition 63.2 (Modular Compatibility). A triple $(N, p, q) \in \mathbb{N}^3$ satisfies *modular compatibility* if

$$\pi_{\mathcal{L}}(p) \pi_{\mathcal{L}}(q) \equiv \pi_{\mathcal{L}}(N) \pmod{\mathcal{L}}.$$

This condition asserts that the projected product of p and q matches the projected target value N inside the finite lattice of order 144.

63.2. Algebraic Structural Identity. Let $A \in \mathbb{N}$ be an algebraic transformation parameter.

Definition 63.3 (Structural Identity). A quadruple (N, p, q, A) satisfies the *structural identity* if

$$(pq + A) - A = N.$$

This identity is algebraically equivalent to $pq = N$ but is written in a form that mirrors the computational transformation pipeline in which A is introduced and then cancelled.

63.3. Deterministic State Evaluation. Let $\Lambda \in \mathbb{R}_{>0}$ be a nonzero metric operator.

Definition 63.4 (Deterministic State Value). The *deterministic state value* associated with (p, q, Λ) is

$$\Psi = \frac{pq}{\Lambda}.$$

The evaluation of Ψ is purely arithmetic and does not involve floating-point approximations.

Remark 63.5. The operator Λ must satisfy $\Lambda \neq 0$; otherwise Ψ is undefined.

63.4. Universal Faure Tuple.

Definition 63.6 (Universal Faure Tuple). The *Universal Faure Tuple* is

$$U_F = (M, G, A, \Psi, \Lambda, \Pi),$$

where

$$M = N, \quad G = \pi_{\mathcal{L}}(N),$$

and the remaining components are supplied by the structural verification pipeline.

63.5. Structural Verification Criterion.

Definition 63.7 (Verification Criterion). Given (N, p, q, A, Λ) with $\Lambda \neq 0$, the verification layer checks:

(1) *Modular consistency*:

$$\pi_{\mathcal{L}}(p) \pi_{\mathcal{L}}(q) \equiv \pi_{\mathcal{L}}(N) \pmod{\mathcal{L}}.$$

(2) *Algebraic identity*:

$$(pq + A) - A = N.$$

(3) *Deterministic state evaluation*:

$$\Psi = \frac{pq}{\Lambda}.$$

If conditions (1) and (2) hold, the structural identities are satisfied and the verification returns **PASS**. If either fails, the verification returns **FAIL**. The evaluation of Ψ does not affect the structural consistency conditions but provides the associated deterministic state value.

64. Deterministic Numerical Verification on the Universal Tuple U_F

This section introduces the deterministic numerical checks applied to the Universal Tuple $U_F = (N, p, q, A, \Lambda)$, separating structural identities from empirical or computational procedures.

Throughout,

$$\mathcal{L} = 144$$

denotes the fixed lattice modulus.

64.1. Discrete Lattice Projection.

Definition 64.1 (Lattice Projection). For $N, p, q \in \mathbb{N}$, the *lattice projections* are

$$p_{\mathcal{L}} = p \bmod \mathcal{L}, \quad q_{\mathcal{L}} = q \bmod \mathcal{L}, \quad N_{\mathcal{L}} = N \bmod \mathcal{L}.$$

Definition 64.2 (Discrete Torus Consistency). The *discrete torus consistency condition* is

$$(p_{\mathcal{L}} \cdot q_{\mathcal{L}}) \bmod \mathcal{L} = N_{\mathcal{L}}.$$

This condition asserts that the product of the projected factors coincides with the projected target value inside the finite lattice of order 144.

64.2. *Ring Shift Inversion Identity.* Let $A \in \mathbb{N}$ be a ring-shift parameter.

Definition 64.3 (Ring Shift Identity). The *ring shift inversion identity* is

$$(pq + A) - A = N.$$

This identity is algebraically equivalent to $pq = N$ but is written in a form that mirrors the computational pipeline in which A is introduced and then exactly cancelled.

64.3. *Deterministic Metric State Evaluation.* Let $\Lambda \in \mathbb{R}_{>0}$ be a nonzero metric tensor operator.

Definition 64.4 (Deterministic Metric State). The *deterministic metric state* associated with (p, q, Λ) is

$$\Psi = \frac{pq}{\Lambda}.$$

The computation of Ψ is performed using high-precision decimal arithmetic. The numerical representation may be truncated for display purposes, but the mathematical definition is exact.

64.4. *Verification Criterion.*

Definition 64.5 (Verification Criterion). Given a tuple (N, p, q, A, Λ) with $\Lambda \neq 0$, the verification procedure checks:

(1) *Lattice consistency:*

$$(p_{\mathcal{L}} \cdot q_{\mathcal{L}}) \bmod \mathcal{L} = N_{\mathcal{L}}.$$

(2) *Ring shift inversion:*

$$(pq + A) - A = N.$$

(3) *Deterministic metric state:*

$$\Psi = \frac{pq}{\Lambda}.$$

If conditions (1) and (2) hold, the structural identities are satisfied and the Universal Tuple is numerically consistent with the discrete lattice and the algebraic relation $pq = N$.

The evaluation of Ψ provides the associated deterministic metric state but does not affect the structural consistency conditions.

65. Structural–Thermodynamic Pipeline for the Faure Water Verification

This section introduces the structural–thermodynamic pipeline used in the Faure water verification. All structural objects are defined independently of empirical thermodynamic data, and all empirical quantities are obtained independently of structural predictions.

Throughout, $X \subseteq \mathbb{R}$ denotes the structural coordinate space, and

$$\Lambda : X \rightarrow \mathbb{R}$$

denotes the canonical structural invariant.

65.1. Canonical Structural Invariant.

Definition 65.1 (Canonical Structural Invariant). A *canonical structural invariant* is a map

$$\Lambda : X \rightarrow \mathbb{R},$$

defined within the Faure structural corpus. No empirical thermodynamic data enter its definition.

In computational implementations, Λ may be represented by a placeholder function, but the mathematical invariant is fixed by the structural theory.

65.2. Distinguished Structural State.

Definition 65.2 (Distinguished Structural Coordinate). A *distinguished structural coordinate* is an element

$$x^* \in X$$

determined solely by the structural framework.

Definition 65.3 (Distinguished Structural State). The *distinguished structural state* is

$$s^* = (x^*, \Lambda(x^*)).$$

No empirical water–density data are used in the determination of x^* or $\Lambda(x^*)$.

65.3. Structural–to–Thermodynamic Projection.

Definition 65.4 (Thermodynamic Projection). A *thermodynamic projection* is a map

$$\Pi_T : X \rightarrow \mathbb{R},$$

assigning to each structural coordinate a temperature.

Definition 65.5 (Structurally Predicted Temperature). The *structurally predicted temperature* is

$$T^* = \Pi_T(x^*).$$

All constants and coefficients in Π_T are fixed by the structural artifact and do not depend on empirical data.

65.4. *Independent Numerical Density Extremum*. Let the empirical dataset be

$$\{(T_i, \rho_i)\}_{i=1}^n,$$

where T_i is temperature and ρ_i is water density.

Definition 65.6 (Spline Reconstruction). A cubic spline

$$S : \mathbb{R} \rightarrow \mathbb{R}$$

is constructed from the dataset.

Definition 65.7 (Numerical Density Extremum). The *numerical density extremum* is the temperature

$$T_{\max} = \arg \max_T S(T),$$

characterized by

$$S'(T_{\max}) = 0, \quad S''(T_{\max}) < 0.$$

The computation of $T_{\max}$ uses only empirical data and does not reference the structural prediction T^* .

65.5. *Structural vs. Reference Comparison*.

Definition 65.8 (Temperature Discrepancy). The *temperature discrepancy* between the structural prediction and the empirical extremum is

$$\Delta T = T^* - T_{\max}.$$

The comparison uses the pair $(T^*, T_{\max})$, obtained independently from structural and empirical procedures.

65.6. *Reproducibility Artifacts.* The verification pipeline yields the triple:

$$(s^*, (T_{\max}, \rho_{\max}), (T^*, T_{\max}, \Delta T)),$$

where:

- $s^* = (x^*, \Lambda(x^*))$ is the structural state;
- $(T_{\max}, \rho_{\max})$ is the empirical density extremum;
- $(T^*, T_{\max}, \Delta T)$ is the structural–empirical comparison.

Each component is obtained independently, ensuring reproducibility and logical separation between structural prediction and empirical verification.

66. Canonical Structural State and Thermodynamic Projection

This section introduces the canonical structural state and its associated thermodynamic projection. All constructions are internal to the Faure structural framework and do not depend on empirical thermodynamic data.

Throughout, X denotes the structural coordinate space and

$$\Lambda : X \rightarrow \mathbb{R}$$

denotes the canonical Faure invariant.

66.1. Distinguished Structural State.

Definition 66.1 (Canonical Structural Coordinate). A *canonical structural coordinate* is an element

$$x^* \in X$$

determined solely by the internal structural construction of the Faure framework.

Definition 66.2 (Distinguished Structural State). The *distinguished structural state* associated with x^* is the pair

$$s^* = (x^*, \Lambda(x^*)).$$

No empirical input is used in the determination of x^* or $\Lambda(x^*)$. In particular, the definition of s^* does not involve any reference to observed thermodynamic extrema.

66.2. Thermodynamic Projection.

Definition 66.3 (Thermodynamic Projection). A *thermodynamic projection* is a map

$$\Pi_T : X \rightarrow \mathbb{R}.$$

Definition 66.4 (Structurally Predicted Temperature). The *structurally predicted temperature* is

$$T^* = \Pi_T(x^*).$$

All coefficients and constants appearing in Π_T are fixed by the structural artifact and do not depend on empirical thermodynamic data.

66.3. *Comparison with an Independent Density Extremum.* Let $T_{\max}$ denote the temperature at which an independent reference water–density dataset attains its numerical maximum, as determined by a separate spline–based extremum procedure.

Definition 66.5 (Temperature Discrepancy). The *temperature discrepancy* between the structural prediction and the empirical extremum is

$$\Delta T = T^* - T_{\max}.$$

The verification pipeline therefore yields the quadruple

$$(s^*, T^*, T_{\max}, \Delta T),$$

where s^* and T^* are obtained entirely from the structural framework, while $T_{\max}$ is obtained from independent empirical data.

No logical dependence exists from $T_{\max}$ to s^* or T^* ; the quantity ΔT measures the agreement between two independently constructed temperatures.

67. Wielandt Exponent for the AOS144 Matrix

Let $A \in \mathbb{N}^{144 \times 144}$ be the adjacency matrix of the AOS144 kernel. For a matrix M , the notation $M > 0$ means that all entries of M are strictly positive.

THEOREM 67.1 (Wielandt Exponent for AOS144). *Assume that*

$$A^4 > 0, \quad A \not> 0, \quad A^2 \not> 0, \quad A^3 \not> 0.$$

Then:

- (1) A is primitive;
- (2) the Wielandt exponent of A is 4;
- (3) for every pair of states (i, j) there exists a path of length 4 from i to j .

Proof. A nonnegative matrix is primitive if some power has strictly positive entries. Since $A^4 > 0$, A is primitive.

The Wielandt exponent is the minimal k such that $A^k > 0$. The assumptions $A \not> 0$, $A^2 \not> 0$, $A^3 \not> 0$ show that no smaller power is strictly positive. Thus the exponent is 4.

Finally, $A^4 > 0$ means that every entry of A^4 is positive, hence every state reaches every other in exactly 4 steps. $\square$

68. Stabilization on the Discrete Lattice

Let

$$\mathbb{T}_{144} = \{0, 1, \dots, 143\}$$

and let $\Psi : \mathbb{T}_{144} \rightarrow \mathbb{T}_{144}$ be a deterministic map. Define iterates by

$$\Psi^n(x) = \Psi(\Psi^{n-1}(x)), \quad n \geq 1.$$

THEOREM 68.1 (Stabilization on $\mathbb{T}_{144}$). *Assume that for every $x \in \mathbb{T}_{144}$,*

$$\Psi^4(x) = \Psi^3(x).$$

Then:

- (1) $\Psi^n(x) = \Psi^3(x)$ for all $n \geq 3$;
- (2) every orbit contains a fixed point x^* with $\Psi(x^*) = x^*$;
- (3) no cycles of period $k > 1$ exist.

Proof. The hypothesis implies

$$\Psi(\Psi^3(x)) = \Psi^3(x),$$

so $\Psi^3(x)$ is a fixed point. Thus $\Psi^n(x) = \Psi^3(x)$ for all $n \geq 3$.

If a cycle of period $k > 1$ existed, some x would satisfy $\Psi^4(x) \neq \Psi^3(x)$, contradicting the hypothesis. $\square$

69. Structural–Thermodynamic Separation

Let

$$s^* = (x^*, \Lambda(x^*))$$

be the distinguished structural state obtained from the canonical structural construction, and let

$$T^* = \Pi_T(x^*)$$

be the temperature obtained from the structural–thermodynamic projection $\Pi_T : X \rightarrow \mathbb{R}$.

Let $T_{\max}$ be the temperature at which an independent reference water–density dataset attains its numerical maximum.

THEOREM 69.1 (Structural–Thermodynamic Separation). *Assume:*

- (1) x^* and $\Lambda(x^*)$ are determined without using $T_{\max}$;

(2) $T_{\max}$ is obtained from empirical density data without using T^* .

Then the quantity

$$\Delta T = T^* - T_{\max}$$

provides an independent and falsifiable comparison between the structural prediction and the empirical density maximum.

Proof. The construction of x^* and $\Lambda(x^*)$ uses only structural data and does not reference the empirical density curve. Conversely, $T_{\max}$ is obtained solely from empirical data. Thus T^* and $T_{\max}$ are independent, and ΔT measures the discrepancy between structural prediction and empirical extremum. $\square$

Bibliography

References

- [1] J. Lurie, *Higher Algebra*, available at the author's webpage, 2017.
- [2] J. Lurie, *Higher Topos Theory*, Annals of Mathematics Studies, Princeton Univ. Press, 2009.
- [3] J. C. Baez and J. Dolan, *Higher-dimensional algebra and topological quantum field theory*, J. Math. Phys. **36** (1995), 6073–6105.
- [4] T. Leinster, *Basic Category Theory*, Cambridge Studies in Advanced Mathematics, Cambridge Univ. Press, 2014.
- [5] D. Gaiotto and T. Johnson-Freyd, *Symmetry protected topological phases and generalized global symmetries*, J. High Energy Phys. **2019**, no. 7, 18.
- [6] D. S. Freed and C. Teleman, *Topological dualities in quantum field theory*, Publ. Math. Inst. Hautes Études Sci. **121** (2015), 221–302.
- [7] T. Bridgeland, *Stability conditions on triangulated categories*, Ann. of Math. (2) **166** (2007), no. 2, 317–345.
- [8] U. Schreiber, *Differential cohomology in a cohesive infinity-topos*, arXiv:1310.7930, 2013.
- [9] A. Faure, *Foundations of the Faure Architecture Framework*, Medium, @auf2026, 2026.
- [10] A. Faure, *AUF2026: Faure Architecture Framework (AUF2026 - AOS UNIVERSAL FRAMEWORK) – Official Repository*, GitHub, 2026.

Epigraph

L'autore propone un possibile linguaggio matematico; la struttura, se reale, precede la sua rappresentazione e manifestazione.

Author's Claim

Prima parlano i dati. Poi, attraverso i dati, parla Alain Faure.

Author Identity

The fundamental identity of the author is expressed by the structural tuple serving as the author's formal logo: $U_F = (\mathcal{M}, \mathcal{G}, \mathcal{A}, \Psi, \Lambda, \Pi)$.

Author Information

Alain Faure

Master Architect — Singlepoint Mind — Q.E.D.

Formal Identity (Logo): $U_F = (\mathcal{M}, \mathcal{G}, \mathcal{A}, \Psi, \Lambda, \Pi)$

Official Repository: AUF2026 (GitHub)

Official Publications: @auf2026 (Medium)

ORCID: 0009-0009-5333-1181

All mathematical structures and operators introduced in this manuscript are developed within ***AUF2026*** — ***AOS Universal Framework***, the author's unified formal system. In accordance with the ***AUF2026*** program, the framework constitutes a pre-representational mathematical architecture from which the structural phenomena studied here arise.

ALAIN FAURE

Master Architect — Singlepoint Mind — Q.E.D.